



第2章 线性表

何谓线性结构

➡ 线性结构是一个数据元素的有序（次序）集合。

例1、26个大写英文字母组成的字母表
(A, B, C, ..., Z)



例2、一副扑克的点数
(2, 3, 4, ..., J, Q, K, A)



何谓线性结构

- 例3、日常生活中看到的学生花名册、学生的成绩单、通讯录、单位的职工工资表以及图书馆的图书目录等等，
- 在计算机内处理这些信息时，都是用由一行行结构相同的数据元素构成的表来进行表示的。对这些表经常进行的操作是修改、查找、插入和删除。

学号	姓名	数学	物理	英语
880001	丁一	90	85	95
880002	马二	80	85	90
880003	张三	95	90	99
880004	李四	70	84	86
.....				

何谓线性结构

- 例4、货轮装货时集装箱由下而上摆放起来，卸货时再从上至下依次取出
- 如果用计算机模拟这一过程，集装箱之间的逻辑关系是线性结构，但处理集装箱的摆放顺序需要遵循后装先取（**后进先出**，Last In First Out, LIFO）的原则进行处理。



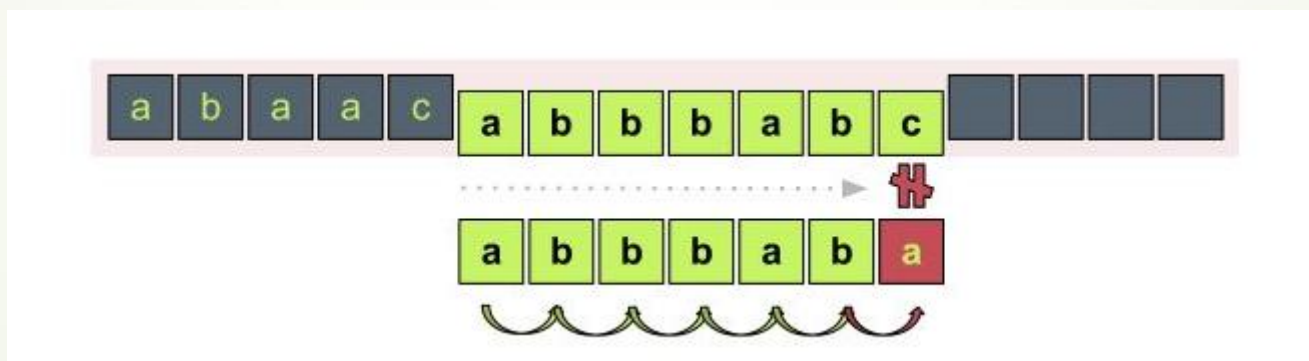
何谓线性结构

- 例5、日常生活中排队购物、汽车进站出站、到银行办理业务等事务处理过程
- 如果用计算机模拟，一般情况是一个需要遵循先来先服务（先进先出，First In First Out, FIFO）的处理原则的线性结构。



何谓线性结构

- 例6、文本编辑软件对文本的处理
- 可以认为是对一个大字符串进行建立、插入、删除、取子串、字符串匹配等操作，而字符串的逻辑结构是**以字符为数据元素**的线性结构。



线性表的逻辑结构

形式定义：由 n ($n \geq 0$) 个数据元素组成的**有序序列**。

$\text{Linear_list} = (D, R)$

其中： $D = \{a_i \mid a_i \in D_0, i=1, 2, \dots, n \quad n \geq 0\}$

$R = \{N\} \quad N = \{ \langle a_{i-1}, a_i \rangle \mid a_{i-1}, a_i \in D_0, i=2, 3, \dots, n \}$

D_0 为某个数据对象

或者简记为： **$(a_1, a_2, \dots, a_i, \dots, a_n)$** $n \geq 0$

(n 为**表长**。当 $n=0$ ，称为**空表**)

线性表的特点：在数据元素的非空有限集中

- 数据元素间是线性关系，数据元素在表中的**位置**只取决于其**序号**
- 存在**唯一**的一个被称作“**第一个**”的数据元素和**唯一**的一个被称作“**最后一个**”的数据元素
- 除第一个外，每个数据元素均**只有一个前驱**；除最后一个外，每个数据元素均**只有一个后继**

线性结构的定义：

若结构是非空有限集，则有且仅有一个开始结点和一个终端结点，并且所有结点都最多只有一个直接前趋和一个直接后继。

简言之，线性结构反映结点间的逻辑关系是 一对一 的关系

可表示为： $(a_1, a_2, \dots, a_n)$

- 线性表（第二章）
- 栈和队列（第三章）
- 字符串（第四章）
- 数组（矩阵）和广义表（第五章）

线性结构

方面 层次	数据表示	数据处理
抽象	逻辑结构	基本运算
实现	存储结构	算法
评价	不同数据结构的比较及算法分析	

何谓线性结构

四个特征：

- 非空集合中必存在唯一的一个“第一元素”
- 非空集合中必存在唯一的一个“最后元素”
- 非空集合中除最后元素之外，其它数据元素均有唯一的“后继”
- 非空集合中除第一元素之外，其它数据元素均有唯一的“前驱”

常见的线性结构：

- 线性表、栈、队列（循环队列）、数组、串

数据元素间的关系：逻辑位置先后关系

第一部分 线性表



教学目标

1. 了解线性表的特点
2. 掌握顺序表的定义、查找、插入和删除
3. 掌握链表的定义、创建、查找、插入和删除
4. 能够从时间和空间复杂度的角度比较线性表的两种存储结构实现的不同特点及其适用场合

第一部分 线性表



教学内容

2.1 线性表

2.2 线性表的顺序存储与实现

2.3 线性表的链式存储

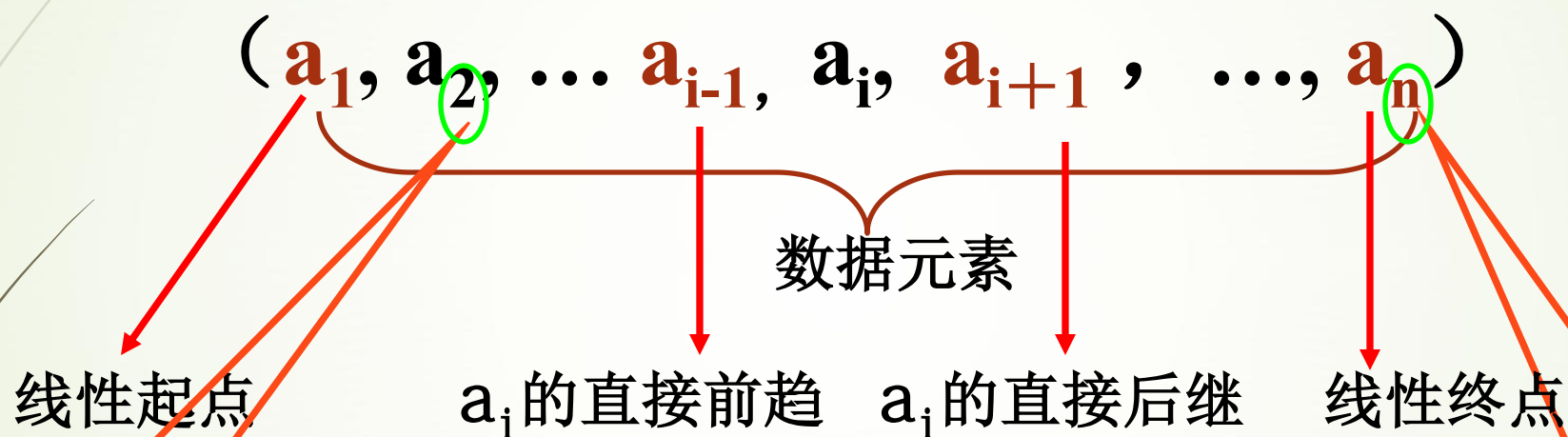
2.4 顺序表和链表的比较

线性表知识点小结

2.1 线性表的定义和特点



定义： 具有相同数据类型的 $n(n \geq 0)$ 个数据元素的有限序列



下标，是元素的序号，表示元素在表中的位置

$n=0$ 时称为**空表**；

即**空表**指表中的元素个数为空，而不是一个不存在的表

n 为元素总个数，即表长

例1 分析26个英文字母组成的英文表

(A, B, C, D,, Z)

数据元素都是字母; 数据元素间关系是线性

例2 分析学生情况登记表

学号	姓名	性别	年龄	班级
041810205	于春梅	女	18	04级计算机1班
041810260	何仕鹏	男	20	04级计算机2班
041810284	王 爽	女	19	04级计算机3班
041810360	王亚武	男	18	04级计算机4班
:	:	:	:	:

数据元素都是记录; 数据元素间关系是线性

同一线性表中的元素必定具有相同特性

2.2 案例引入



案例2.1：在计算机中实现一元多项式的运算

$$P_n(x) = p_0 + p_1x + p_2x^2 + \dots + p_nx^n$$

对照线性表的特点，上面的表达式中有哪些信息需要表示，哪些信息不需要表示

（每一项的指数*i*隐含在其系数 p_i 的序号中， x 不需要每次都表示）

线性表 $P = (p_0, p_1, p_2, \dots, p_n)$

$$P(x) = 10 + 5x - 4x^2 + 3x^3 + 2x^4$$

表达式指数 (数组下标 <i>i</i>)	0	1	2	3	4
系数 $p[i]$	10	5	-4	3	2

用顺序存储结构实现

（每一项的指数*i*隐含在其系数 p_i 的序号中）

$$R_n(x) = P_n(x) + Q_m(x)$$



线性表 $R = (p_0 + q_0, p_1 + q_1, p_2 + q_2, \dots, p_m + q_m, p_{m+1}, \dots, p_n)$

$$S(x) = 1 + 3x^{10000} + 2x^{20000}$$

按照上例，我们要建一个 $n = 20001$ 的顺序存储结构的线性表，并且其中只有 p_0 、 p_{10000} 、 p_{20000} 有值，其他都是空的；
这样的线性表很浪费，该怎么办？用什么样的线性表来表示？



●顺序存储结构存在问题

✓存储空间分配不灵活

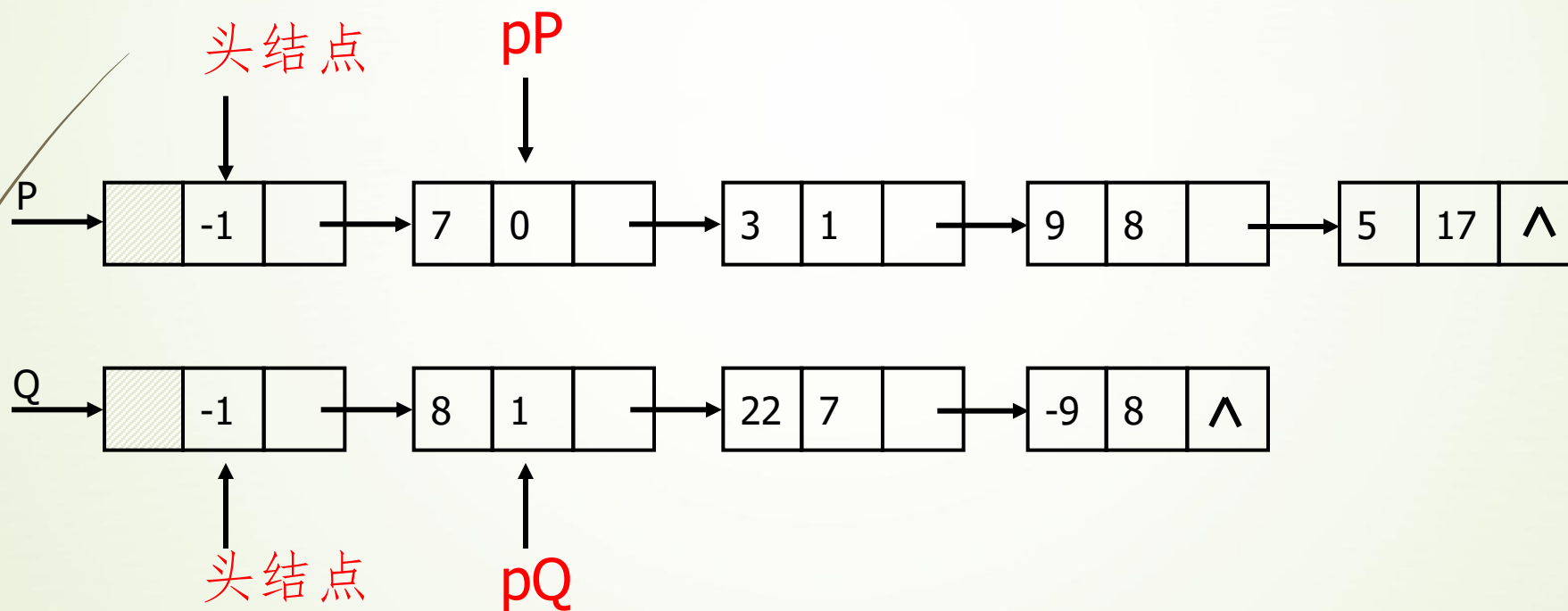
✓运算的空间复杂度高



试试链式存储结构

$$P(x) = 7 + 3x + 9x^8 + 5x^{17}$$

$$Q(x) = 8x + 22x^7 - 9x^8$$



案例2.2：稀疏多项式的运算

$$P(x) = 7 + 3x + 9x^8 + 5x^{17}$$

下标i	0	1	2	3
系数	7	3	9	5
指数	0	1	8	17



$$Q(x) = 8x + 22x^7 - 9x^8$$

下标i	0	1	2
系数	8	22	-9
指数	1	7	8

$$R(x) = 7 + 11x + 22x^7 + 5x^{17}$$

下标i	0	1	2	3
系数	7	11	22	5
指数	0	1	7	17

$$P_n(x) = p_1 x^{e_1} + p_2 x^{e_2} + \dots + p_m x^{e_m}$$

线性表 $P = ((p_1, e_1), (p_2, e_2), \dots, (p_m, e_m))$



$$R_n(x) = P_n(x) + Q_m(x)$$

线性表的遍历运算

依次访问线性表的每个元素。

线性表的加运算

● 创建一个新线性表R来存储计算结果：

● 分别从头遍历比较 $P_n(x)$ 的线性表P和 $Q_m(x)$ 的线性表Q中的每一项

✓ 指数相同，对应系数相加，若其和不为零，则在R中增加一个新项，继续遍历两个表

✓ 指数不相同，则将指数较小的项复制到R中，继续遍历指数小的线性表

● 一个多项式已遍历完毕时，遍历另一个表，将表中剩余项依次复制到R中即可

多项式相加

$$P(x) = 7 + 3x + 9x^8 + 5x^{17}$$

$$Q(x) = 8x + 22x^7 - 9x^8$$

●采用 链式存储结构 后的加 运算

✓直接在P 线性表上进行运算:

✓指数相同，相加后非0则替换值；为0则删除当前结点；继续遍历P和Q的下一个结点再进行比较

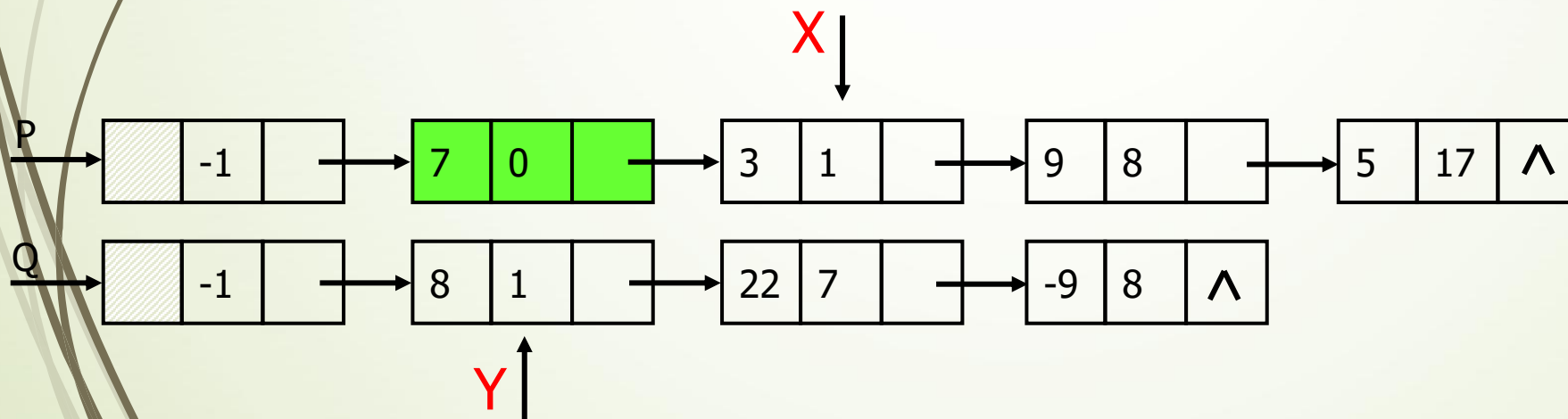
✓指数不同时

✓P的指数小，则遍历到P的下一个继续比较；（X指针）

✓Q的指数小，则新建一个结点，复制Q的指数值插入到P的当前结点前，遍历Q的下一个结点进行比较；（Y指针）

✓Q链表遍历结束，计算结束；

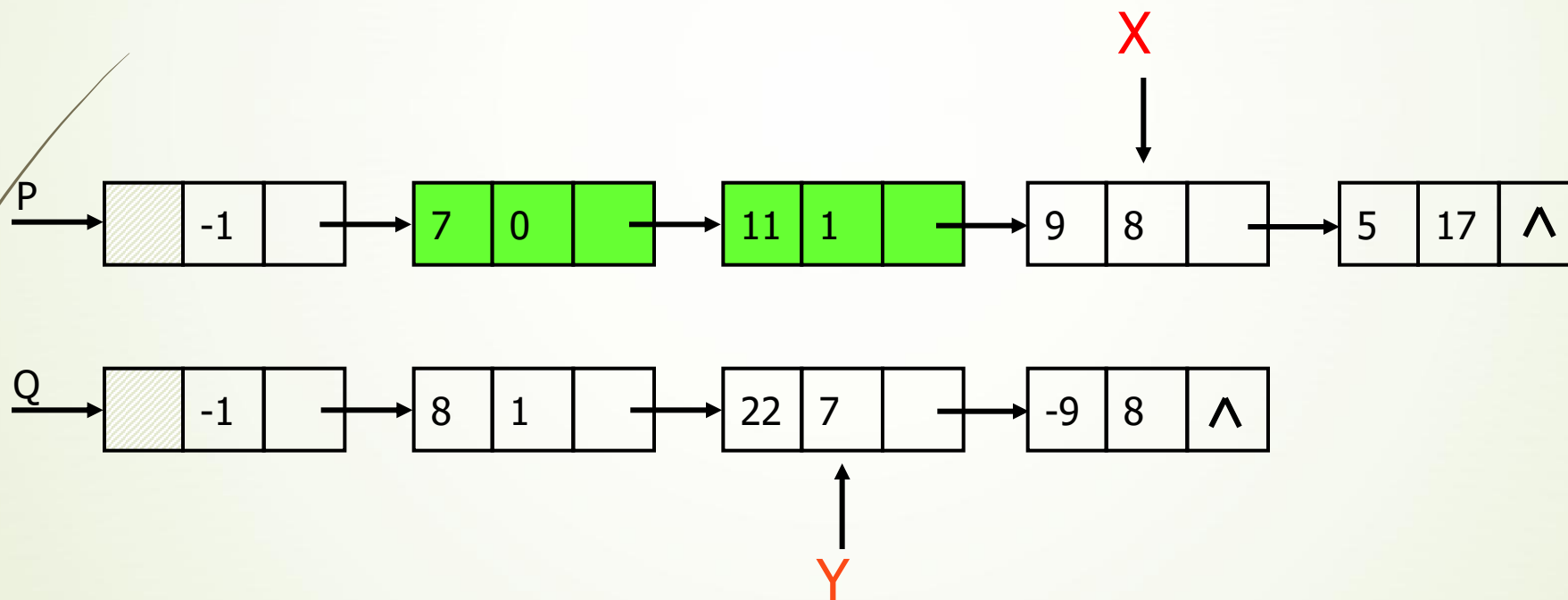
✓P链表遍历到结束，遍历Q链表将Q中的剩余结点依次插入到P的尾部。



多项式相加

$$P(x) = 7 + 3x + 9x^8 + 5x^{17}$$

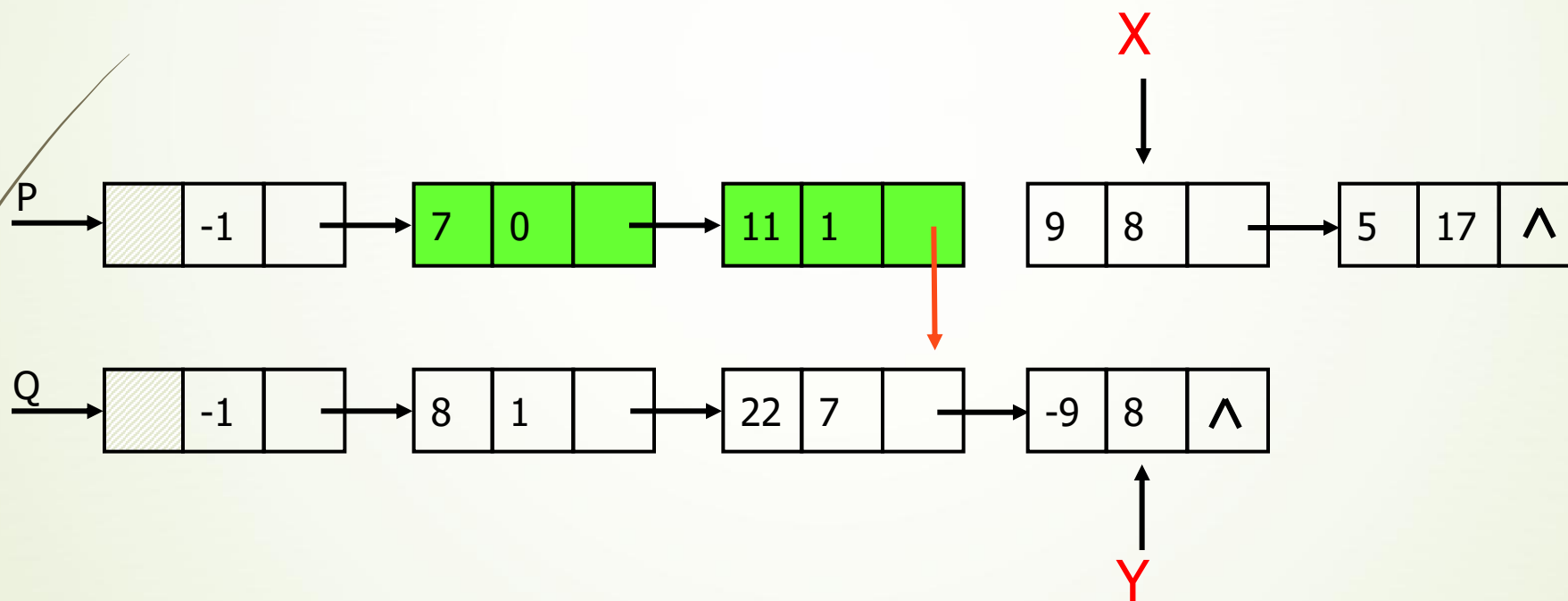
$$Q(x) = 8x + 22x^7 - 9x^8$$



多项式相加

$$P(x) = 7 + 3x + 9x^8 + 5x^{17}$$

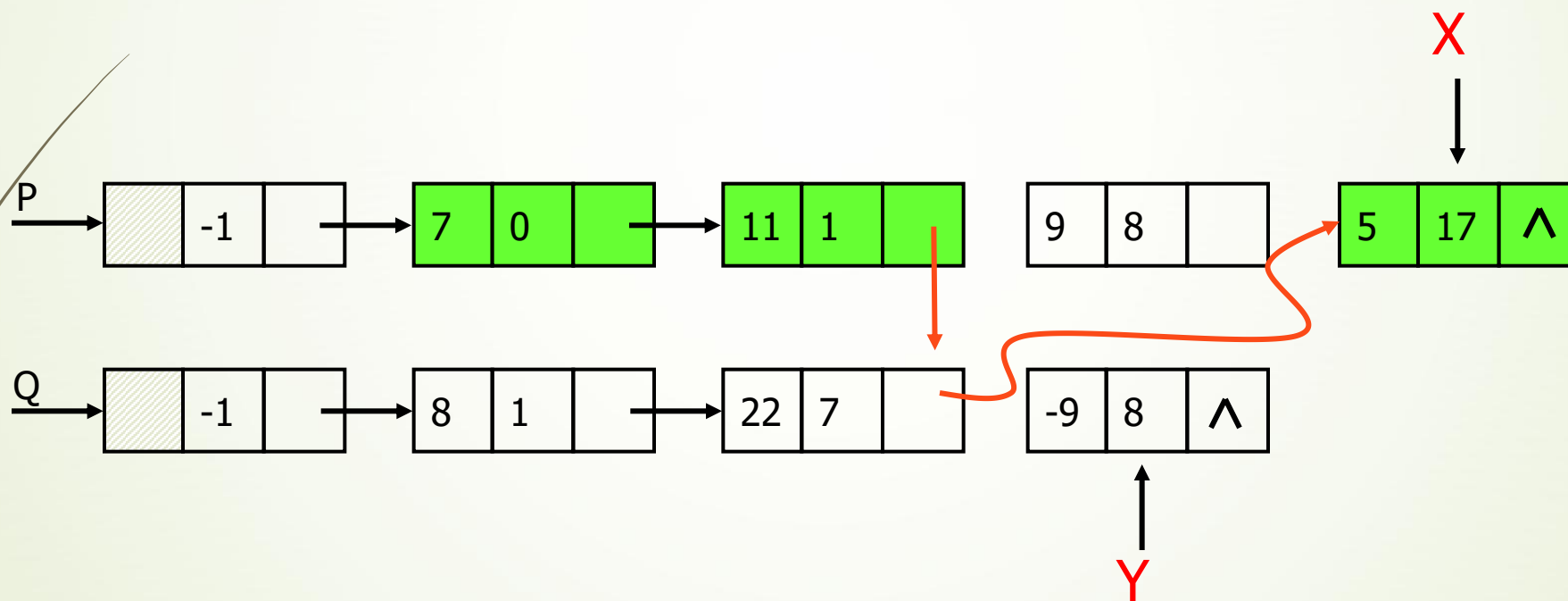
$$Q(x) = 8x + 22x^7 - 9x^8$$



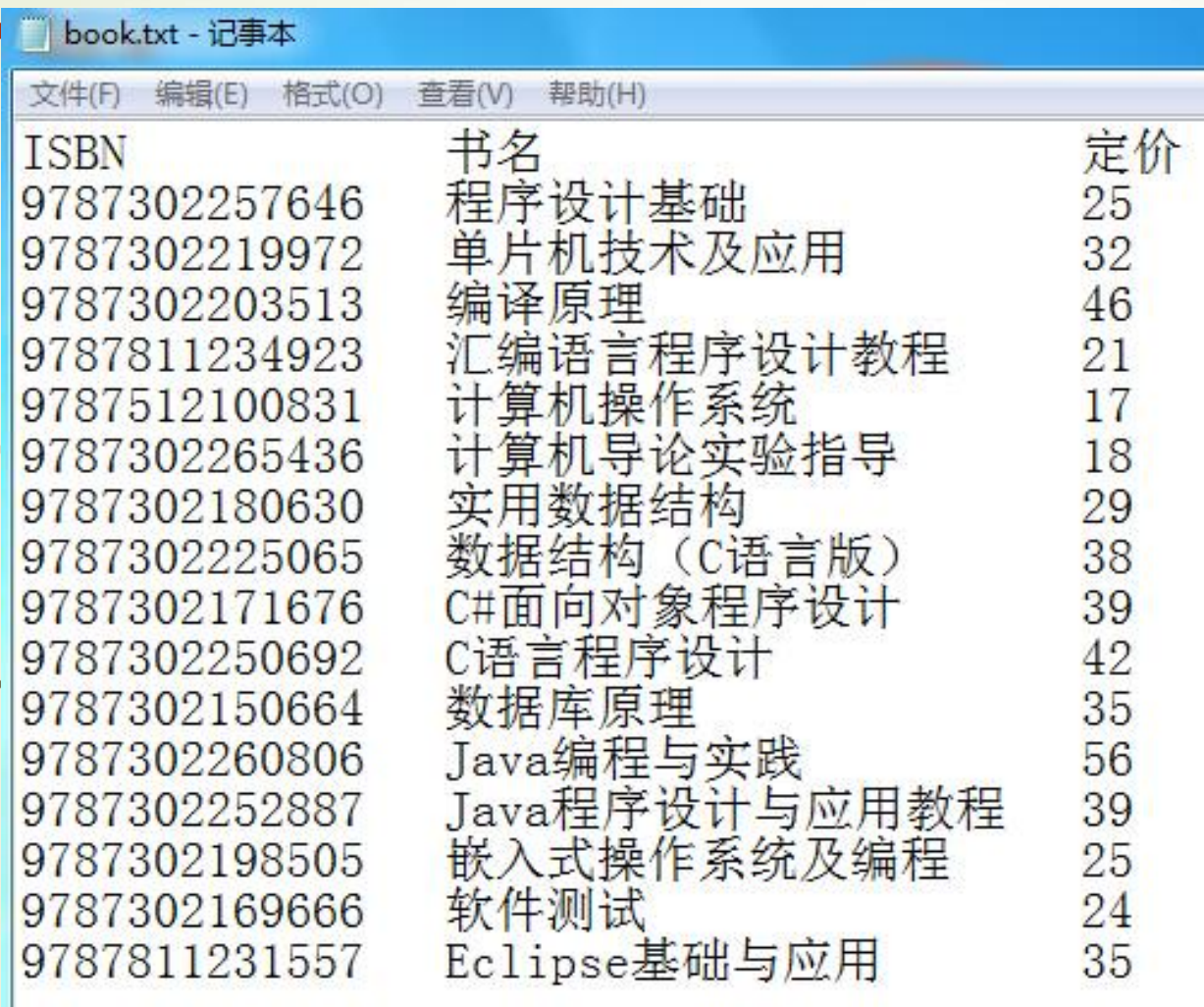
多项式相加

$$P(x) = 7 + 3x + 9x^8 + 5x^{17}$$

$$Q(x) = 8x + 22x^7 - 9x^8$$



案例2.3：图书信息管理系统



ISBN	书名	定价
9787302257646	程序设计基础	25
9787302219972	单片机技术及应用	32
9787302203513	编译原理	46
9787811234923	汇编语言程序设计教程	21
9787512100831	计算机操作系统	17
9787302265436	计算机导论实验指导	18
9787302180630	实用数据结构	29
9787302225065	数据结构（C语言版）	38
9787302171676	C#面向对象程序设计	39
9787302250692	C语言程序设计	42
9787302150664	数据库原理	35
9787302260806	Java编程与实践	56
9787302252887	Java程序设计与应用教程	39
9787302198505	嵌入式操作系统及编程	25
9787302169666	软件测试	24
9787811231557	Eclipse基础与应用	35

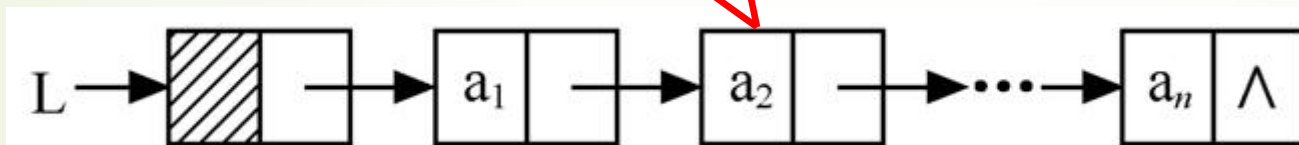
- (1) 查找
- (2) 插入
- (3) 删除
- (4) 修改
- (5) 排序
- (6) 计数

图书表的顺序存储

elem[0]	elem[1]	elem[2]	...	elem[length-1]	空闲区
a_1	a_2	a_3	...	a_{length}	



图书表的链式存储





- 线性表中数据元素的类型可以为简单类型（单个数据项），也可以为复杂类型（多个数据项）。
- 许多实际应用问题所涉的基本操作有很大相似性，不必要为每个具体应用单独编写一个程序。
- 从具体应用中**抽象出共性的逻辑结构和基本操作（抽象数据类型）**，然后实现其存储结构和基本操作。
- 将数据元素的类型差异忽略，抽象为一个结点。

2.3 线性表的类型定义



线性表的重要基本操作

1. 初始化
2. 取值，按位置取值，
3. 查找，按位置查找，按值查找
4. 插入，指定位置插入，尾插入
5. 删除
6. 计数
7. 排序（第7章）

线性表的ADT定义

- **ADT List {**
- 数据对象：
 - $D = \{a_i \mid a_i \in \text{ElemSet}, i=1,2,\dots,n, n \geq 0\}$
- 数据关系：
 - $R = \{ \langle a_{i-1}, a_i \rangle \mid a_i \in D, i=2,\dots,n \}$
 - 序偶，即有序对，可以看作是含有两个元素的集合。
但它与一般集合不同的是序偶具有确定的次序。

线性表的ADT定义

基本操作：

- ➡ (1) 线性表初始化： `InitList(&L)`
- ➡ (2) 销毁线性表： `DestroyList(&L)`
- ➡ (3) 将线性表置空： `ClearList(&L)`
- ➡ (4) 判断线性表是否为空： `ListEmpty(L)`
- ➡ (5) 求线性表的长度： `ListLength (L)`
- ➡ (6) 按位置取表中某个元素： `GetElem(L,i,&e)`

线性表的ADT定义

➡ 基本操作：

- ➡ (7) 按值查找： `LocateElem(L,e,compare())`
- ➡ (8) 前驱： `PriorElem(L,cur_e, &pre_e)`
- ➡ (9) 后继： `NextElem(L,cur_e,&next_e)`
- ➡ (10) 插入操作： `ListInsert (&L,i,e)`
- ➡ (11) 删除操作： `ListDelete (&L,i,&e)`
- ➡ (12) 遍历操作： `ListTraverse (L,visit())`

在遍历操作中，用visit()函数
指针作为参数
即遍历到某个结点时，由
visit()函数来具体对结点的数
据项进行操作

例2-1

- 现假设以线性表 LA 和 LB 分别表示集合 A 和 B，求两个集合的并集，使 $A = A \cup B$ 。
- 审题后得到题目要求：
 - 集合合并于A上，即将B中不存在于A中的元素加入到A中
 - 集合用线性表的逻辑结构，即将存在于线性表 LB 中而不存在于线性表 LA 中的数据元素插入到线性表 LA 中去。

例2-1

想到的一个方法为：

1. 从线性表 **LB** 中取出一个数据元素；
2. 依值在线性表 **LA** 中进行查询；
3. 若不存在，则将它插入到 **LA** 中。

重复上述三步直至 处理完**LB**中所有的元素。

改成计算机的算法思路：

ListLength(Lb)

对每个**LB**中的数据元素

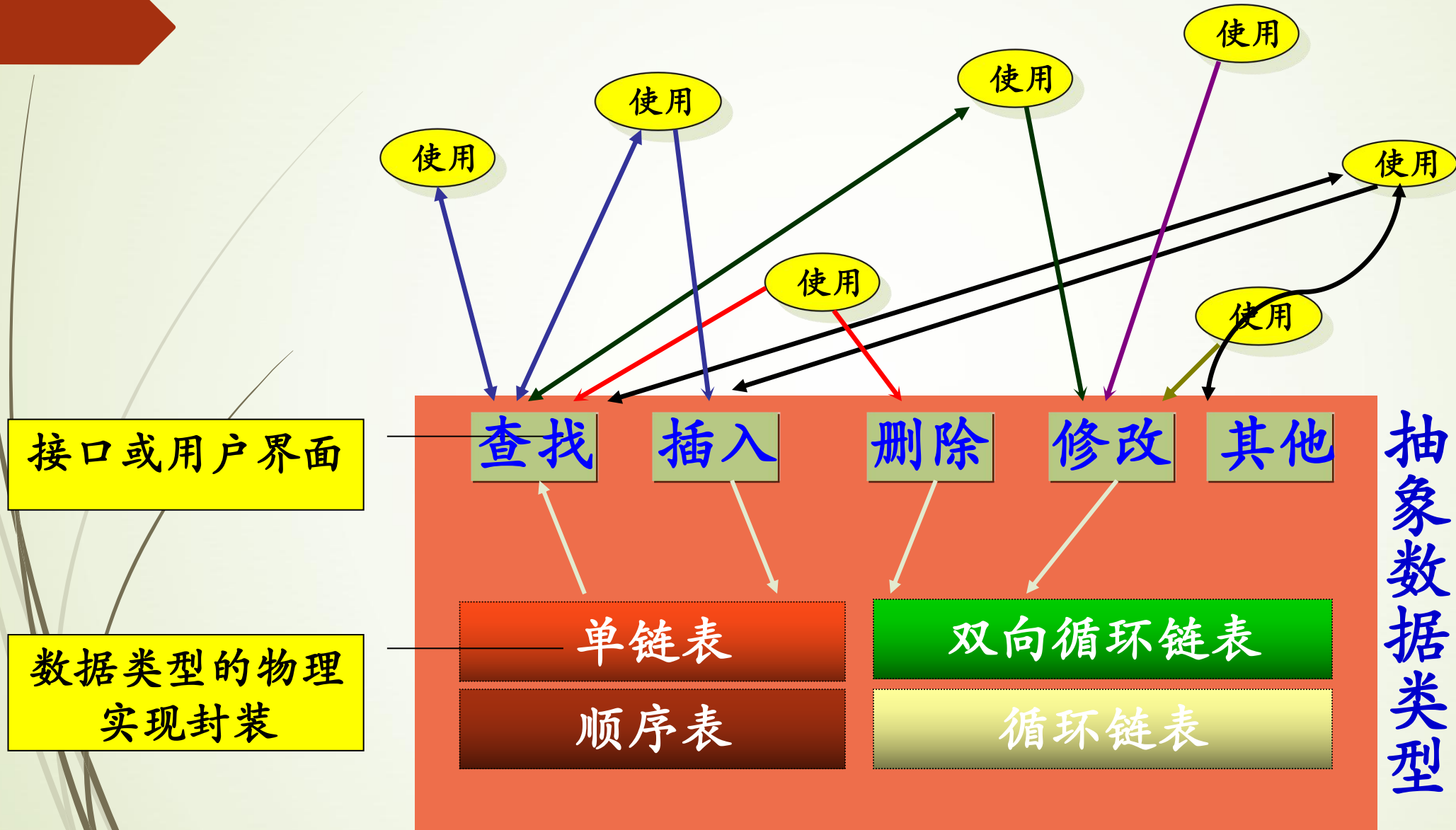
- 1、取出其数据元素的值，
- 2、在**LA**中查找是否存在
- 3、若不存在，则插入到**LA**的尾部

GetElem(Lb, i, &e)

LocateElem(La, e)

ListInsert(La, ListLength(La)+1, e)

ADT的好处：信息隐蔽和数据封装，使用与实现相分离



2.2 线性表的顺序表示和实现



线性表的顺序表示又称为**顺序存储结构**或**顺序映像**。

顺序存储定义：把逻辑上相邻的数据元素存储在物理上相邻的存储单元中的存储结构。

简言之，**逻辑上相邻，物理上也相邻**

顺序存储方法：用一组地址连续的存储单元依次存储线性表的元素，可通过**数组V[n]**来实现。



顺序存储

存储地址	存储内容
1	元素1
2	元素2
.....	
i	元素i
.....	
n	元素n

← 线性表的基地址 L_0

第i个元素的地址:

$$\text{Loc}(\text{元素}i) = L_0 + (i-1) * m$$

其中m为数据元素的大小

即, $m = \text{sizeof}(\text{数据元素})$

了相应的描述功能。以下为其作用示意图。

(1) 预定义常量和类型:

```
// 函数结果状态代码
#define TRUE      1
#define FALSE    0
#define OK       1
#define ERROR    0
#define INFEASIBLE -1
#define OVERFLOW -2
// Status 是函数的类型,其值是函数结果状态代码
typedef int Status;
```

(2) 数据结构的表示(存储结构)用类型定义(**typedef**)描述。数据元素类型约定为 ElemType,由用户在使用该数据类型时自行定义。

```
typedef int ElemType;
```

(3) 基本操作的算法都用以下形式的函数描述:

```
函数类型 函数名(函数参数表) {
    // 算法说明
    语句序列
} // 函数名
```

顺序存储的线性表的数据结构定义

// 假设已经定义了数据元素类型，即ElemType，例如：typedef int ElemType;
方法0:

```
ElemType SqList[100];    //100个ElemType类型的数据元素的表，未初始化
```

优点：顺序存储的特征非常明显。

缺点：1) 表长度用数值定义，不规范（用宏定义替换数值）；
2) 表的大小在变量定义时固定死了，不能用于大于100个数据元素的表；
3) 缺一个表示当前长度的变量。

方法1:

```
#define MAXSIZE 100      //最大长度  
Elemtype SqList[MAXSIZE]; //MAXSIZE个ElemType类型的数据元素的表  
                        //SqList的当前长度用变量nListLength表示  
int nListLength; //线性表SqList当前长度，初始化是置为0，表示空表
```

优点：顺序存储的特征非常明显。

缺点：表长度用有含有的宏表示出来了；
表的大小在变量定义时固定死了，不能用于大于100个数据元素的表；
两个变量是配合使用的关系，需要在注释里说明清楚。

顺序存储的线性表的数据结构定义

// 不在直接定义变量来表示表，先定义通用的表示变量的类型：

方法2：

```
typedef struct {  
    ElemType  elem[MAXSIZE];           //数组形式的定义，在声明变量是表空间已经有了  
                                           //而且是固定大小的  
    unsigned int  nListLength;         //线性表的当前长度  
    unsigned int  nListSize;           //线性表的最大长度  
}SqListType;
```

优点：

顺序存储的特征非常明显；

补充了表当前长度的表示，补充了表最大容量的表示。

在结构体内把存储位置、当前长度、最大长度表示在一起，阅读理解方便

缺点：

表的大小在类型定义时固定死了，不能适应通用的要求

顺序存储的线性表的数据结构定义

方法3:

```
typedef struct {  
    ElemType *elem;           //指向顺序存储的线性表的基地址  
    unsigned int uListLength;  //线性表的当前长度  
    unsigned int uListSize;    //线性表的最大长度  
}SqListType;
```

缺点: 1) 由于elem为指针类型, 不能直观的看出是一个顺序存储结构
2) 定义变量时, 未分配顺序存储空间, 需用语句分配空间, 容易忘记分配空间而导致空指针错误。(数据结构都需要定义初始化函数)
3) 退出函数或主程序时, 需要记得对已分配内存的变量进行释放, 避免造成内存丢失。(数据结构都需要定义销毁函数)

优点: 可以方便的适应不同大小的表容量的要求: 在初始化的操作中, 将elem指向一个实实在在的存储区域

例如: SqListType sqList; //定义变量,

sqList.elem = (ElemType*)malloc(sizeof(ElemType) * sqList.uListSize); //动态分配内存
free(sqList.elem); // 释放内存。 c中使用malloc和free, c++使用new和delete

顺序存储的线性表的数据结构定义

方法3:

```
typedef struct {  
    ElemType *elem;           //指向顺序存储的线性表的基地址  
    unsigned int uListLength;  //线性表的当前长度  
    unsigned int uListSize;    //线性表的最大长度  
}SqListType;
```

初始化时静态分配内存:

SqListType sqList; //定义表变量

ElemType aList[MAXSIZE]; //定义静态内存变量; 顺序存储, 编译时分配

将静态内存和表变量的指针联系起来:

sqList.elem = &aList; //运行时使用, 无需释放

教材上的定义

```
typedef struct
```

```
{ ElemType *elem; //存储空间基地址
```

```
int length; //当前长度
```

```
int listsize; //表最大大小，即对elem进行内存分配后可容纳的
```

```
//ElemType类型数据元素的个数，对教材定义增加的。
```

```
} SqList;
```

定义一个顺序表的存储变量： **SqList L;**

有时用一个指向SqList 类型的指针对实现顺序表的操作更为方便： **SqList *pL;**

pL是一个指针变量，通过“pL=(**SqList ***)malloc(sizeof(**SqList**));”操作来进行顺序表变量的存储空间分配，pL中存放的是**顺序表结构体（SqList）的地址**。

再对pL->listsize初始化， pL->listsize = nInput;

再通过pL->elem = (ElemType *)malloc(sizeof(ElemType) * pL->listsize)来给顺序表分配所需大小的连续的内存区域。

顺序表的第一个数据元素：*(L->elem) 或 L->elem[0]

顺序表的第一个数据元素的地址： L->elem 或 &(L->elem[0])。

例子：图书表的顺序存储结构类型定义

```
typedef struct          //图书信息定义
{
    char no[ISBNSTRINGSIZE];          //图书ISBN
    char name[BOOKNAMESTRINGSIZE];    //图书名字
    float price;                       //图书价格
}BookInfoType;

typedef struct
{
    BookInfoType *elem;                //存储空间的基地址
    unsigned int  uListLength;          //图书表中当前图书个数
    unsigned int  uListSize;            //图书表的最大长度
}SqBookListType;    //图书表的顺序存储结构类型为SqBookListType
```

线性表的重要基本操作

1. 初始化
2. 取值
3. 查找
4. 插入
5. 删除

重要基本操作的算法实现

1.1 初始化线性表L （参数用引用方式），时间复杂度为O（1）

```
bool InitList_Sq(SqList &L, int nListSize )
//构造一个内容为空，长度为nListSize 的顺序表L
{
    if( nListSize <=0) return false;

    //为线性表分配顺序存储空间
    L.elem = (ElemType*)malloc( sizeof(ElemType)*nListSize);
    if(!L.elem) return false;           //存储分配失败
    L.length=0;                          //空表长度为0
    L.listsize= nListSize;               //最大表长
    return true;
}
```

重要基本操作的算法实现

1.2 初始化线性表L （参数用指针方式），时间复杂度为O(1)

```
bool InitList_Sq(SqList *pL, int nListSize )
```

```
//构造一个内容为空，长度为nListSize 的顺序表L
```

```
{
```

```
    if( !pL) return false;
```

```
    if( nListSize <=0) return false;
```

```
    //为线性表分配顺序存储空间
```

```
    pL->elem = (ElemType*)malloc( sizeof(ElemType)*nListSize);
```

```
    if(!pL->elem) return false;           //存储分配失败
```

```
    pL->length=0;                          //空表长度为0
```

```
    pL->listsize= nListSize;              //最大表长
```

```
    return true;
```

```
}
```

这里，L是NULL，
为什么不对L
malloc一下，分
配一个内存空间后
继续初始化呢？

重要基本操作的算法实现

1.3 初始化线性表L（返回一个线性表指针），时间复杂度为O（1）

```
SqList * InitList_Sq(int nListSize )
```

```
//构造一个内容为空，长度为nListSize 的顺序表L
```

```
{
```

```
    if( nListSize <=0) return NULL;
```

```
    SqlList *pL=(SqlList*)malloc(sizeof(SqlList));//分配线性表结构体空间
```

```
    if(!pL) return NULL;
```

```
    //为线性表分配顺序存储空间
```

```
    pL->elem = (ElemType*)malloc( sizeof(ElemType)*nListSize);
```

```
    if(!pL->elem) { free(pL); return NULL }; //存储分配失败
```

```
    pL->nLength=0; //空表长度为0
```

```
    pL->nListSize = nListSize; //最大表长
```

```
    return L;
```

```
}
```

对比一下下面两个函数声明，思考一下喜欢用那个

```
FILE *__cdecl fopen(const char *_FileName, const char *_Mode)
```

```
errno_t __cdecl fopen_s(FILE **_Stream, const char *_FileName, const char *_Mode)
```

用fopen操作文件

```
FILE* fp;  
fp = fopen("test.txt","a");  
if (!fp)  
    exit(-1);  
fprintf(fp,"hello world!\n");  
fclose(fp);
```

用fopen_s操作文件

```
FILE* fp;  
fopen_s(&fp, "test.txt", "a");  
if (!fp)  
    exit(-1);  
fprintf(fp, "hello world!\n");  
fclose(fp);
```

左侧代码，在VS2019编译器中报错：“错误 C4996 ‘fopen’: This function or variable may be unsafe.Consider using fopen_s instead.”;按错误提示信息进行修改，修改为右侧代码可编译通过。

微软编译器的警告，主要是针对那些传统C库的函数，这些C库函数内部是不进行参数检测的（会导致越界），微软担心使用这些会造成内存越界读写造成安全隐患，就改写了一下同样功能的函数，进行了参数的检测。

对比一下下面两个函数声明，思考一下喜欢用那个

```
FILE *__cdecl fopen(const char *_FileName, const char *_Mode)
```

```
errno_t __cdecl fopen_s(FILE **_Stream, const char *_FileName, const char *_Mode)
```

用fopen操作文件

```
FILE* fp;
```

```
if((fp=fopen("testa.txt", "r")) == NULL)
```

```
    exit(-1);
```

```
fprintf(fp, "hello world!\n");
```

```
fclose(fp);
```

用fopen_s操作文件

```
FILE* fp;
```

```
fopen_s(&fp, "test.txt", "a");
```

```
if (!fp)
```

```
    exit(-1);
```

```
fprintf(fp, "hello world!\n");
```

```
fclose(fp);
```

对比一下下面两个函数声明，思考一下喜欢用那个

```
FILE *__cdecl fopen(const char *_FileName, const char *_Mode)
```

```
errno_t __cdecl fopen_s(FILE **_Stream, const char *_FileName, const char *_Mode)
```

用fopen操作文件

```
FILE* fp;
```

```
if(!(fp = fopen("test.txt","a")))
```

```
    exit(-1);
```

```
fprintf(fp,"hello world!\n");
```

```
fclose(fp);
```

用fopen_s操作文件

```
FILE* fp;
```

```
fopen_s(&fp, "test.txt", "a");
```

```
if (!fp)
```

```
    exit(-1);
```

```
fprintf(fp, "hello world!\n");
```

```
fclose(fp);
```

对比一下下面两个函数声明，思考一下喜欢用那个

```
FILE *__cdecl fopen(const char *_FileName, const char *_Mode)
```

```
errno_t __cdecl fopen_s(FILE **_Stream, const char *_FileName, const char *_Mode)
```

用fopen操作文件

```
FILE* fp;
```

```
if(!(fp = fopen("test.txt","a")))
```

```
    exit(-1);
```

```
fprintf(fp,"hello world!\n");
```

```
fclose(fp);
```

用fopen_s操作文件

```
FILE* fp;
```

```
errno_t errNo=fopen_s(&fp, "test.txt", "a");
```

```
if (errNo)
```

```
    exit(-1);
```

```
fprintf(fp, "hello world!\n");
```

```
fclose(fp);
```

对比一下下面两个函数声明，思考一下喜欢用那个

```
FILE *__cdecl fopen(const char *_FileName, const char *_Mode)
```

```
errno_t __cdecl fopen_s(FILE **_Stream, const char *_FileName, const char *_Mode)
```

用fopen操作文件

```
FILE* fp;
```

```
if(!(fp = fopen("test.txt","a")))
```

```
    exit(-1);
```

```
fprintf(fp,"hello world!\n");
```

```
fclose(fp);
```

用fopen_s操作文件

```
FILE* fp;
```

```
if(fopen_s(&fp, "test.txt", "a"))
```

```
    exit(-1);
```

```
fprintf(fp, "hello world!\n");
```

```
fclose(fp);
```

补充：几个简单基本操作的算法实现

销毁线性表L

时间复杂度均为O(1)

```
void DestroyList(SqList &L)
{
    //释放存储空间
    if (L.elem) free(L.elem);
    L.elem = NULL;
    L.length=L.listsize=0;
}
```

销毁线性表L

时间复杂度均为O(1)

```
void DestroyList(SqList * pL)
{
    if (!pL) return;
    //释放存储空间
    if (pL->elem) free(pL->elem);
    pL -> elem = NULL;
    pL->length=pL->listsize=0;
}
```

void ClearList(SqList &L) //清空线性表L，简单方式

```
{
    L.length=0; //将线性表的长度置为0
                //并未真正清空L.elem占用内存的内容，有风险
    //将L.elem的每个存储单元置为0
    memset(L.elem, 0, sizeof(ElemType) * L.listsize);
}
```

ClearList的时间复杂度?

补充：几个简单基本操作的算法实现 时间复杂度均为0 (1)

清空线性表L，安全方式

```
void ClearList(SqList &L)
```

```
{
```

```
    L.length=0;    //将线性表的长度置为0
```

```
                //并未真正清空L.elem占用内存的内容，有风险
```

```
    //将L.elem的每个存储单元置为0
```

```
    memset(L.elem, 0, sizeof(ElemType) * L.listsize);
```

```
}
```

```
//memset的时间复杂度，看看gcc的memset实现源码
```

```
/* Public domain.  */
```

```
#include <stddef.h>
```

```
void * memset (void *dest, int val, size_t len)
```

```
{
```

```
    unsigned char *ptr = dest;
```

```
    while (len-- > 0)
```

```
        *ptr++ = val;
```

```
    return dest;
```

```
}
```

memset是
O(n)的时间复杂度

补充：几个简单基本操作的算法实现 时间复杂度均为O(1)

求线性表L的长度

```
int GetLength(SqList L)
{
    return (L.length);
}
```

```
int GetLength(SqList * pL)
{
    if(pL) return (pL->length);
    else return 0;
}
```

判断线性表L是否为空

```
int IsEmpty(SqList L)
{
    if (L.length==0)
        return 1;
    else
        return 0;
}
```

```
int IsEmpty(SqList *pL)
{
    if(pL)
    {
        if (pL->length==0)
            return 1;
    }
    return 0;
}
```

补充：几个简单基本操作的算法实现

扩展线性表存储空间

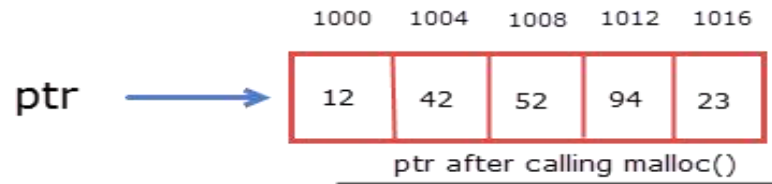
```
bool ExtendList_Sq(SqList &L, int nIncrementSize)
{
    ElemType *newbase=(ElemType *)realloc(L.elem,
        (L.listsize + nIncrementSize)*sizeof(ElemType));
    if (! newbase) return false;

    L.elem=newbase;
    L.listsize += nIncrementSize;
    return true;
} // ExtendList_Sq
```

最好 $O(1)$ 最坏 $O(n)$

补充：几个简单基本操作的算法实现

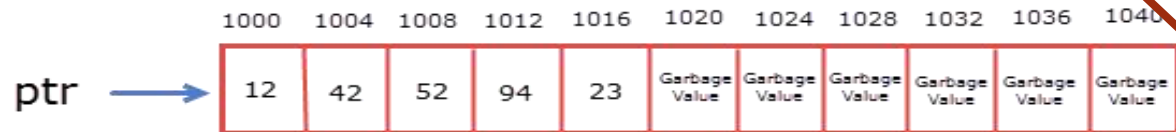
```
p = (int*)malloc(5*sizeof(int));
```



```
p = (int*)realloc(p, 11*sizeof(int));
```

Now two conditions may arise:

1st case: If sufficient memory is available after address 1016, then the address of ptr doesn't change.



2nd case: If sufficient memory is not available after address 1016, then the realloc() function allocates memory somewhere else in the heap and copies the all content from old memory block to the new memory block. In this case the address of ptr changes.



```
int nIncrementSize)
```

```
)realloc(L.elem,  
ze)*sizeof(ElemType));
```

最好 $O(1)$ 最坏 $O(n)$

线性表的重要基本操作

1. 初始化

2. 取值

3. 查找

4. 插入

5. 删除

2. 取值 (根据位置*i*获取相应位置数据元素的内容)

获取线性表L中的某个数据元素的内容

```
int GetElem(SqList L,int i,ElemType &e)
{
    if (i<1 || i>L.length) return 0;
    //判断i值是否合理, 若不合理, 返回0
    e=L.elem[i-1]; //第i-1的单元存储着第i个数据, c语言的数组从0开始
    return 1;
}
```

补充: 随机存取和顺序存取

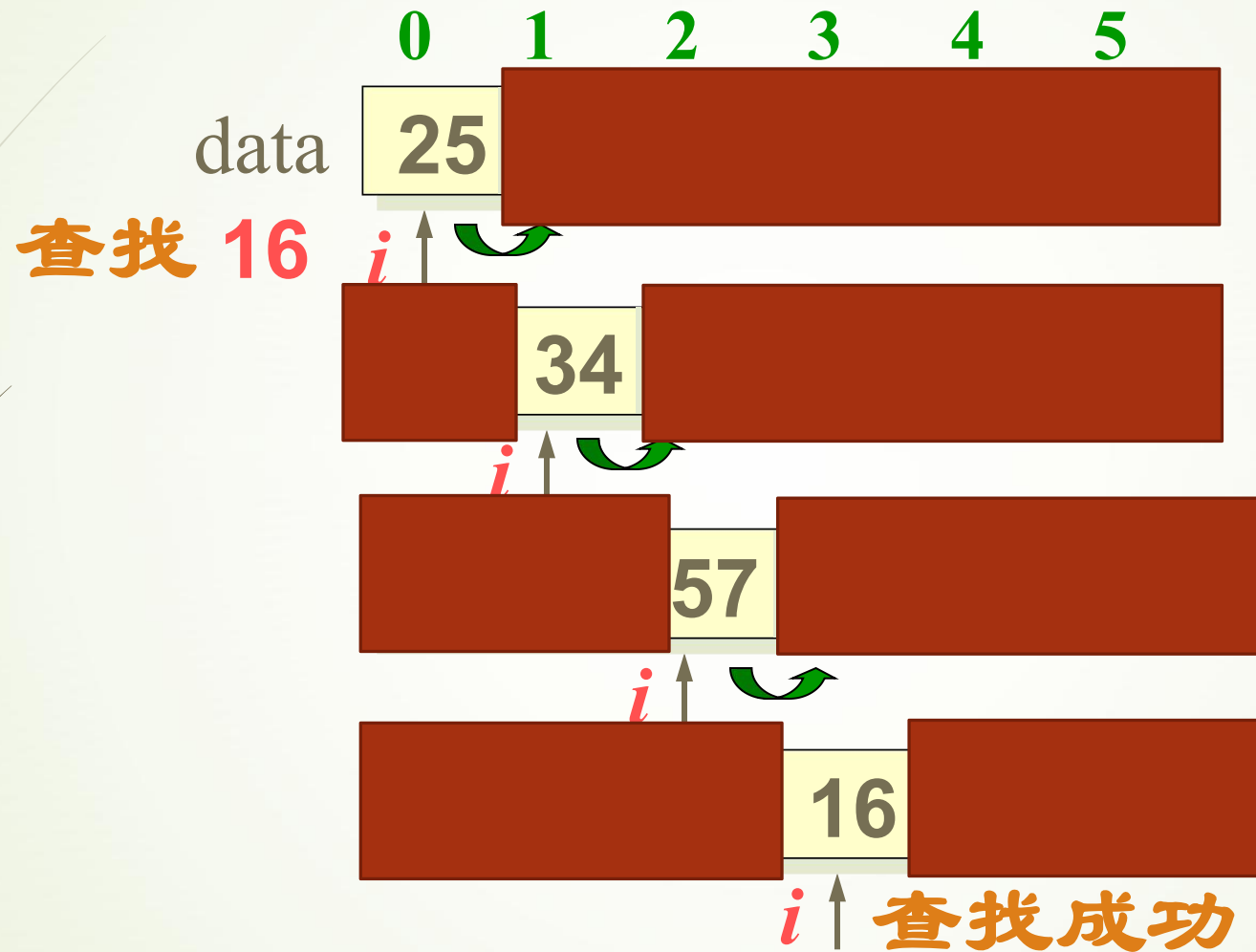
对于一个磁带存储, 我们查找一个数据时, 只能在磁带绕动的同时, 一个数据一个数据的找, 这是顺序存取;

相对的, 对于一个内存, 可以通过地址直接取到数据, 这叫做随机存取

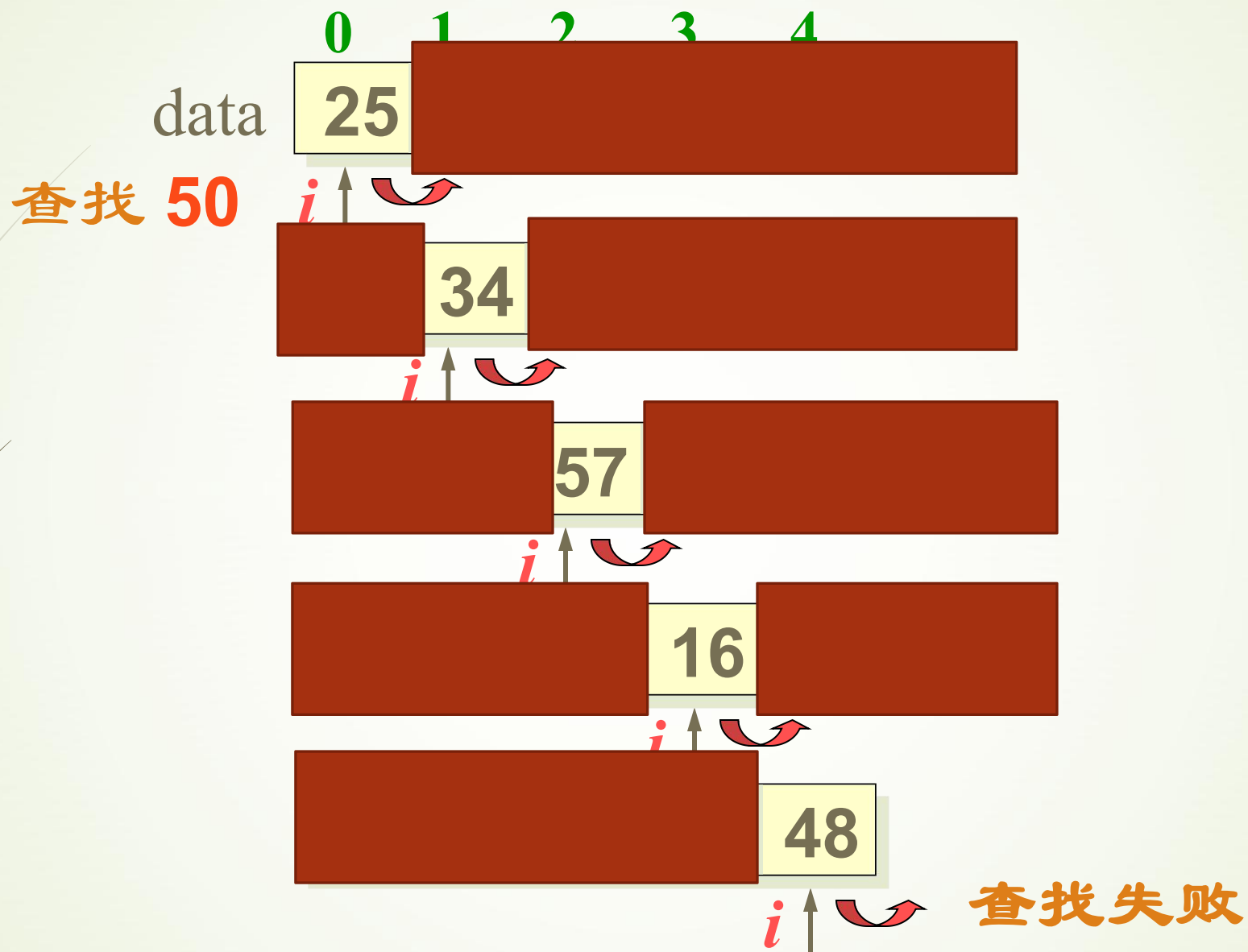
随机存取, $O(1)$

3. 查找（根据指定数据获取数据所在的位置）

顺序查找图示



3. 查找 (根据指定数据获取数据所在的位置)



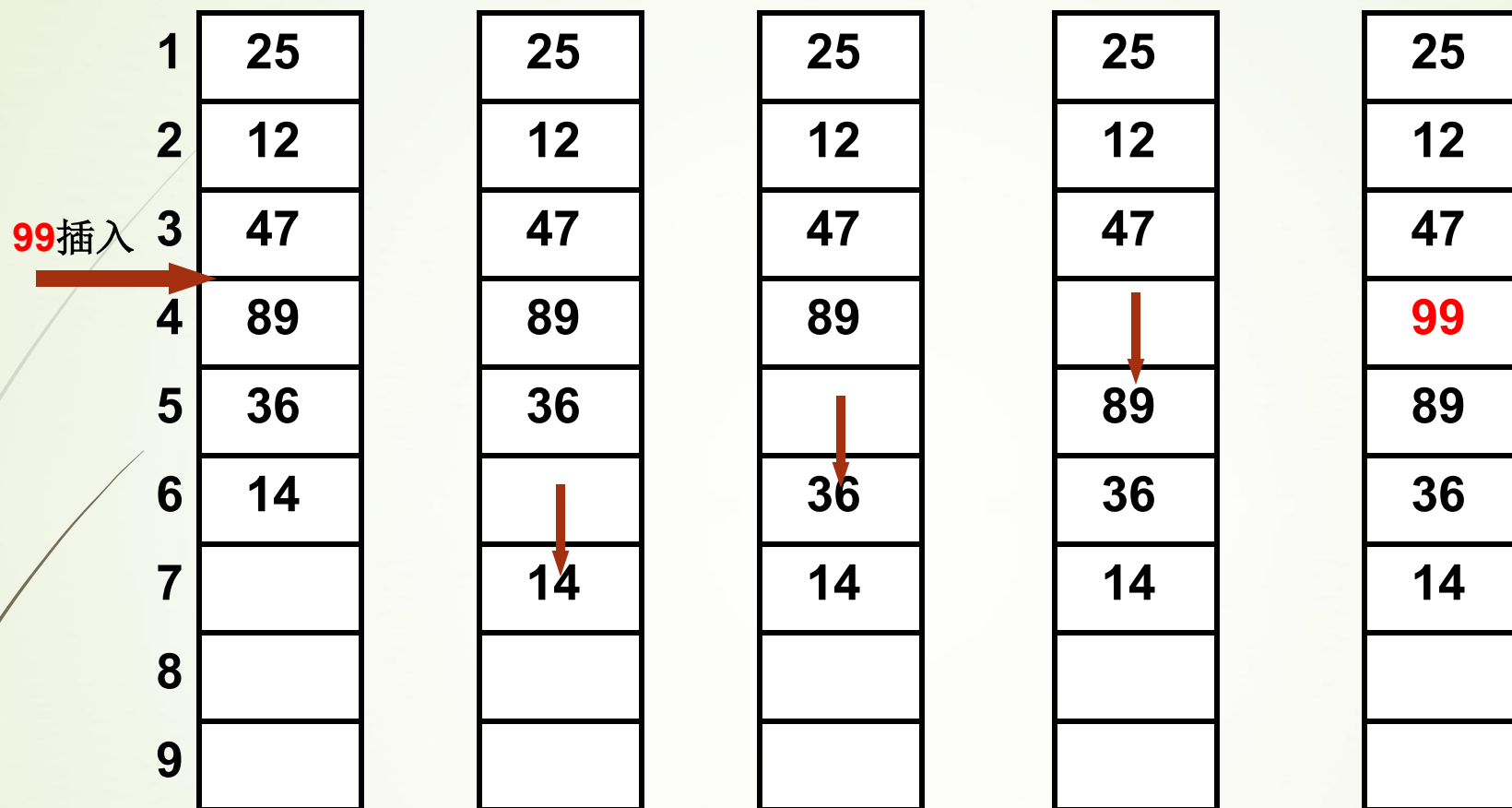
3. 查找 (根据指定数据获取数据所在的位置)

在线性表L中查找值为e的数据元素

```
int LocateElem(SqList L, ElemType e)
{
    for (i=0; i< L.length; i++)
        if (L.elem[i]==e) //ElemType应是比较类型
            return i+1;
    return 0;
}
```

查找算法时间效率分析 ? ? ? **$O(n)$**

4. 插入（插在第 i 个结点之前）



插第 4 个结点之前，移动 $6-4+1$ 次

插在第 i 个结点之前，移动 $n-i+1$ 次

- 插入运算思路:

将 $a_i \sim a_n$ 顺序向下移动, 为新元素让出位置

将 x 置入空出的第 i 个位置

修改 $L.length$ (相当于修改表长)

- 思路代码化:

```
if (i < 1 || i > L.uLength + 1)
```

超出范围, 返回false

```
if (L.uLength >= L.uListsize)
```

满, 返回false //或增加分配空间

倒着处理从 a_n 到 a_i 依次后移一个位置

将 x 置入空出的第 i 个位置

修改 $L.uLength$

【算法描述】

4. 在线性表L中第i个数据元素之前插入数据元素e，空间不够时失败

```
bool ListInsert_Sq(SqList &L,int nPos ,ElemType e){
    if(nPos <1 || nPos >L.length+1)
        return false;                //i值不合法
    if(L.length==L.listsize) //当前存储空间已满，无法插入e
        return false;    //不扩充存储空间，返回false

    for(j=L.length-1; j>= nPos-1; j--)
        L.elem[j+1]=L.elem[j];        //插入位置及之后的元素后移
    L.elem[nPos-1]=e;                  //将新元素e放入第i个位置
    ++L.length;                        //表长增1
    return true;
}
```

【算法描述】

4. 在线性表L中第i个数据元素之前插入数据元素e，空间不够时动态扩充

```
bool ListInsert_Sq(SqList &L,int nPos ,ElemType e){
    if(nPos <1 || nPos >L.length+1)
        return false;                //i值不合法
    if(L.length==L.listsize) //当前存储空间已满，无法插入e
        if( !ExtendList_Sq(L, L.uListSize*10/100+1))//扩存储空间
            return false;            //扩充失败，返回false
    for(j=L.length-1; j>= nPos-1; j--)
        L.elem[j+1]=L.elem[j];        //插入位置及之后的元素后移
    L.elem[nPos-1]=e;                  //将新元素e放入第i个位置
    ++L.length;                        //表长增1
    return true;
}
```

本算法中注意以下问题：

① 顺序表中数据区域有**listsize**个存储单元，所以在向顺序表中做插入时先检查表空间是否满了，在表满的情况下除非扩展空间否则不能再做插入。

② 要检验插入位置的有效性，这里 **nPos** 的有效范围是：
 $1 \leq nPos \leq n+1$ ，其中 n 为原表长。

③ 注意数据的移动方向，先移动最后1个，从后向前移动。

④ 表长的修改。

【算法分析】

算法时间主要耗费在移动元素的操作上

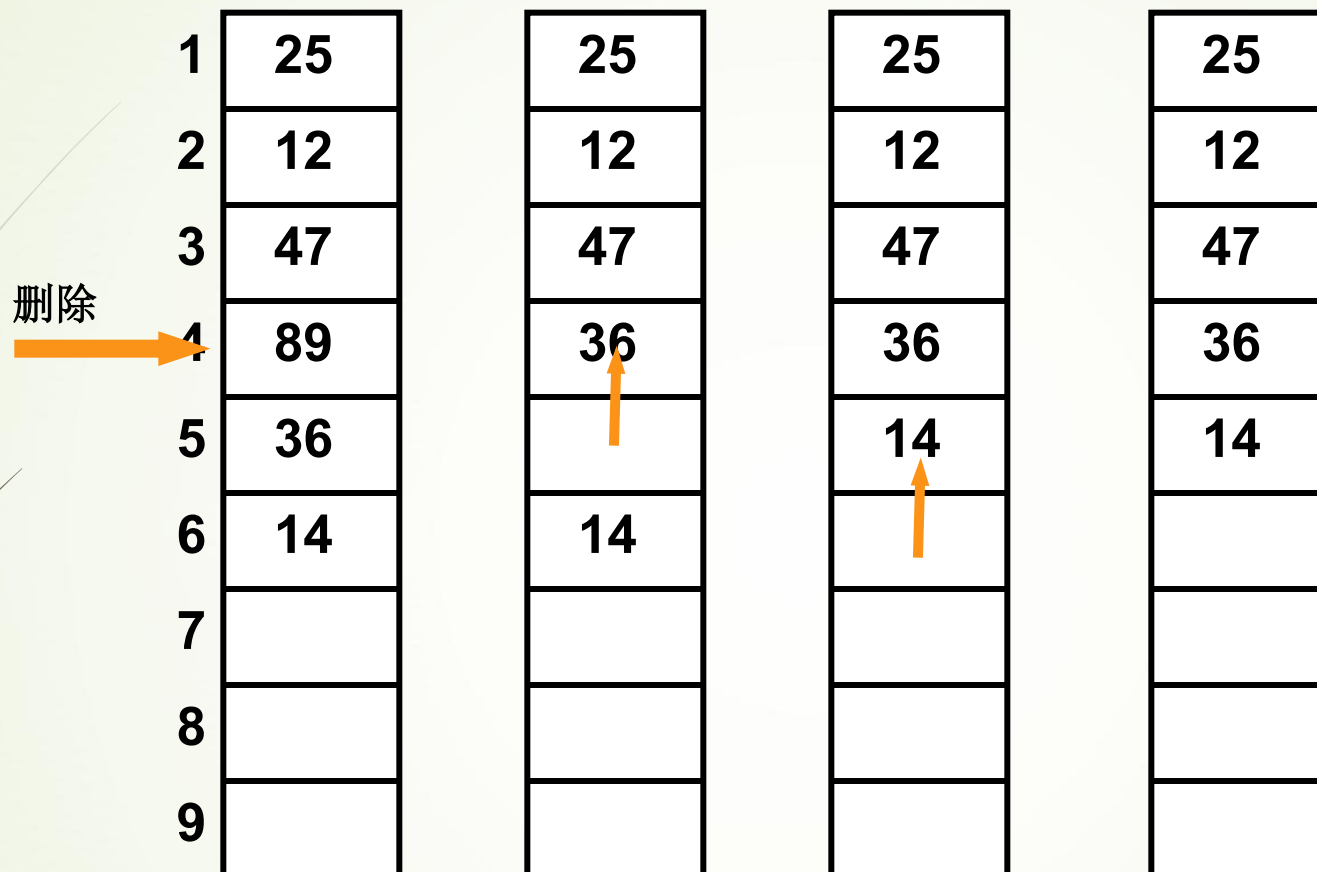
若插入在尾结点之后，则根本无需移动（特别快）；

若插入在首结点之前，则表中元素全部后移（特别慢）；

若要考虑在各种位置插入（共 $n+1$ 种可能）的平均移动次数，该如何计算？

$$\begin{aligned} \text{AMN} &= \frac{1}{n+1} \sum_{i=1}^{n+1} (n - i + 1) = \frac{1}{n+1} (n + \cdots + 1 + 0) \\ &= \frac{1}{(n+1)} \frac{n(n+1)}{2} = \frac{n}{2} \end{aligned}$$

5. 删除（删除第 i 个结点）



删除第 4 个结点，移动 $6-4$ 次

删除第 i 个结点，移动 $n-i$ 次

➤ 删除元素运算基本思路:

1) 将 $a_{i+1} \sim a_n$ 顺序向前移动

2) 修改L.uLength

➤ 思路代码化:

If ($i < 1 || i > n$) //检查空表及删除位置的合法性

不存在直接返回

将 $a_{i+1} \sim a_n$ 顺序向前移动

修改L.uLength

【算法描述】

5. 将线性表L中第i个数据元素删除

```
bool ListDelete_Sq(SqList &L, int i)
```

```
{
```

```
    if((i<1)|| (i>L.length))
```

```
        return false; //i值不合法
```

```
    for (j=i; j<=L.length-1; j++)
```

```
        L.elem[j-1]=L.elem[j]; //被删除元素之后的元素前移
```

```
    --L.length;
```

```
    //表长减1
```

```
    return true;
```

```
}
```

本算法中注意以下问题：

① 删除第 i 个元素， i 的取值为 $1 \leq i \leq n$ ，否则第 i 个元素不存在，因此，要检查删除位置的有效性。

② 当表空时不能做删除

$((i < 1) \parallel (i > L.length))$ 包括了表空的检查和删除位置的检查。

③ 删除 a_i 之后，该数据已不存在。如果需要，先取出，再做删除。

④ 修改表长。

【算法分析】

算法时间主要耗费在移动元素的操作上

若删除尾结点，则根本无需移动（特别快）；

若删除首结点，则表中 $n-1$ 个元素全部前移（特别慢）；

若要考虑在各种位置删除（共 n 种可能）的平均移动次数，该如何计算？

$$AMN = \frac{1}{n} \sum_{i=1}^n (n-i) = \frac{1}{n} \frac{(n-1)n}{2} = \frac{n-1}{2}$$

查找、插入、删除算法的平均时间复杂度为
 $O(n)$

显然，顺序表的空间复杂度 $S(n)=O(1)$
(没有占用辅助空间)

顺序表（顺序存储结构）的特点

- (1) 利用数据元素的存储位置表示线性表中相邻数据元素之间的前后关系，即线性表的**逻辑结构与存储结构一致**
- (2) 在访问线性表时，可以快速地计算出任何一个数据元素的存储地址。因此可以粗略地认为，**访问每个元素所花时间相等**

这种存取元素的方法被称为**随机存取法**

顺序表的优缺点

优点:

- ✓ 存储密度大 (结点本身所占存储量/结点结构所占存储量)
- ✓ 可以随机存取表中任一元素

缺点:

- ✓ 在插入、删除某一元素时, 需要移动大量元素
- ✓ 浪费存储空间
- ✓ 属于静态存储形式, 数据元素的个数不便于自由扩充

为克服这一缺点



链表

- ➡ 两个有序顺序表的合并：有顺序表A和B，其数据元素均按从小到大升序排列，编写一个算法将它们合并成顺序表C，要求C的元素也是从小到大升序排列。

```
bool merge(SqList A, SqList B, SqList &C)
{
    //这里要求C是已经初始化好的
    if(C.uListSize < (A.length + B.length) )
    {
        return false;
    }

    int i,j,k;
    i=0; j=0; k=0;
    while ( i<A.length && j<B.length )
    {
        /*将A和B的当前元素较小者复制到表C*/
        if (A.elem[i]<=B.elem[j])
        {
            C.elem[k++]=A.elem[i++];
        }
        else
        {
            C.elem[k++]=B.elem[j++];
        }
    }

    while (i<A.length) /*将A中剩余元素复制到表C*/
    {
        C.elem[k++]= A.elem[i++];
    }
    while (j<B.length) /*将B中剩余元素复制到表C */
    {
        C.elem[k++]= B.elem[j++];
    }
    C.uLength=k;
    return true;
}
```

算法的时间性能是 $O(m+n)$ ，其中
m是A的表长，n是B的表长。
即 $O(n)$ ，线性时间复杂度

//和教材 P44-2.16 稍有差别

void MergeList_Sq(SqList La, SqList Lb, SqList &Lc)

//将数据元素按值非递减排列的顺序表La和Lb合并到顺序表Lc

//Lc的元素也按值非递减排列

```
01 {
02     ElemType*pa=La.elem; ElemType*pb=Lb.elem;
03     Lc.listsize =Lc.length =La.length+Lb.length;
04     //这里要求Lc是未初始化的顺序表
05     pc=Lc.elem=(ElemType *)malloc(Lc.listsize*sizeof(ElemType));
06     if (!pc) exit(OVERFLOW);
07
08     ElemType* pa_last= La.elem+La.length-1;
09     ElemType* pb_last= Lb.elem+Lb.length-1;
10     while (pa<=pa_last && pb<=pb_last ) {
11         if (*pa<=*pb)
12             *pc++ = *pa++;
13         else
14             *pc++ = *pb++;
15     }
16     while (pa<=pa_last) *pc++ = *pa++;
17     while (pb<=pb_last) *pc++ = *pb++;
18 }//MergeList_Sq    O(La.length+Lb.length)
```

10到15行的while循环为什么不用如下方式:

```
while ((pa-La.elem)<=La.length &&
        pb-Lb.elem<=Lb.length){
    if (*pa<=*pb)
        *pc++ = *pa++;
    else
        *pc++ = *pb++;
}
```

这段代码中while的条件比较表达式比教材代码11行的表达式多两次减法计算。16、17行是同一原因

//来自教材 P26-2.7

void MergeList_Sq(SqList La, SqList Lb, SqList &Lc)

//将数据元素按值非递减排列的顺序表La和Lb合并到顺序表Lc

//Lc的元素也按值非递减排列

```
01 {
02     ElemType*pa=La.elem; ElemType*pb=Lb.elem;
03     Lc.listsize =Lc.length =La.length+Lb.length;
04     //这里要求Lc是未初始化的顺序表
05     pc=Lc.elem=(ElemType *)malloc(Lc.listsize*sizeof(ElemType));
06     if (!pc) exit(OVERFLOW);
07
08     ElemType* pa_last= La.elem+La.length-1;
09     ElemType* pb_last= Lb.elem+Lb.length-1;
10     while (pa<=pa_last && pb<=pb_last ) {
11         if (*pa<=*pb)
12             *pc++ = *pa++;
13         else
14             *pc++ = *pb++;
15     }
16     while (pa<=pa_last) *pc++ = *pa++;
17     while (pb<=pb_last) *pc++ = *pb++;
18 }//MergeList_Sq    O(La.length+Lb.length)
```

和用数组方式实现的代码对比一下:

```
while ( i<A.length && j<B.length ) {
    if (A.elem[i]<=B.elem[j])
        C.elem[k++]=A.elem[i++];
    else
        C.elem[k++]=B.elem[j++];
}
```

代码逻辑结构一样，具体操作变量方式不同。
为什么很多早期的C专家喜欢用指针代替数组？

顺序表应用举例-1

应用1：将顺序表 $(a_1, a_2, \dots, a_n)$ 重新排列为以 a_1 为界的两部分： a_1 前面的值均比 a_1 小， a_1 后面的值都比 a_1 大。（规则）

➡ 基本思路：

25	30	20	60	10	35	15		
15	10	20	25	30	60	35		

假设当前就是满足以 a_1 为界的，如何证实，就对每个元素依次验证一下：

0) 取出 a_1 的值到 x

1) 当前数据元素的值比 x 大时，假设成立，不必改变位置，回到1) 继续比较下一个

2) 当前数据元素的值比 x 小时，假设不成立，需要调整表让假设成立

调整方法：前面的都是符合规则的，将当前元素抽出放在最前面即可

算法：将当前元素上（前）面的元素都依次向下（后）移动一个位置，然后把它置于最上方。回到1) 继续进行验证。

【算法2-5】 顺序表划分算法

```
void part(SeqList &L)
{
    int i,j;
    ElemType x,y;
    x=L.elem[0];          /*将基准数据元素置入 x 中*/
    for (i=1; i<L.uLength; i++)
    {
        if (L.elem[i] < x)    /*当前数据元素小于基准*/
        {
            y=L.elem[i];
            for (j=i-1;j>=0;j--) /*前面所有数据元素向后移动*/
                L.elem[j+1] = L.elem[j];
            L.elem[0]=y;      /*将当前元素置入最前面位置*/
        }
    }
}
```

最坏情况下总的移动次数为：

$$\sum_{i=2}^n (i - 1 + 2) = \sum_{i=2}^n (i + 1) = \frac{n * (n + 3)}{2}$$

即最坏情况下移动数据时间性能为 $O(n^2)$ 。

【算法2-5】 顺序表划分算法

最坏情况下移动数据时间性能为 $O(n^2)$ 。

如何优化？即，如何减少移动的次数？

```
void part(SeqList &L)
{
    int i,j;
    ElemType x,y;
    x=L.elem[0];          /*将基准数据元素置入 x 中*/
    for (i=1; i<L.uLength; i++)
    {
        if (L.elem[i] < x)    /*当前数据元素小于基准*/
        {
            y=L.elem[i];
            for (j=i-1;j>=0;j--) /*前面所有数据元素向后移动*/
                L.elem[j+1] = L.elem[j];
            L.elem[0]=y;      /*将当前元素置入最前面位置*/
        }
        对这段代码进行优化，减少移动次数，如何做？
    }
}
```

C语言相关知识

在本课程中，数据的存储结构是用C语言的数据类型描述（定义）的，主要用到下列数据类型：

- 数组
- 指针
- 结构
- 结构指针

C语言相关知识

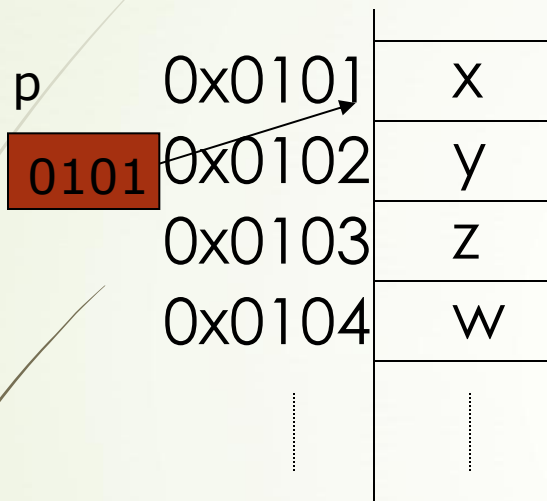
1 数组

```
char a[100];
```

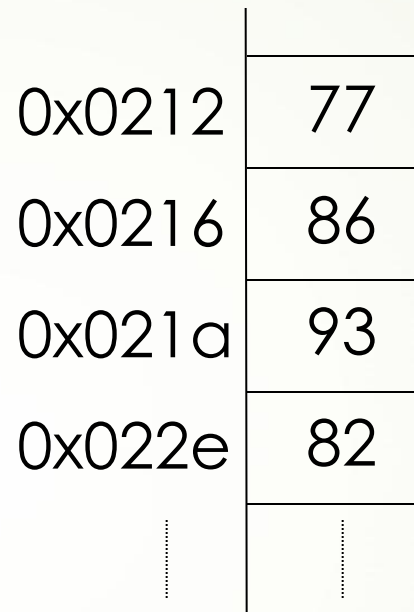
```
int b[100];
```

数组a中每个元素占用一个字节空间，数组b中每个元素占用四个字节空间。

数组



```
char * p; char ch;  
p = 0x0101;  
ch = *p;  
p++;  
ch = *p;
```



```
int * p; int num;  
p = 0x0212;  
num = *p;  
p++;  
num = *p;
```

结构定义的一般形式

```
struct 结构名 {  
    类型  变量名;  
    类型  变量名;  
    .....  
} 结构变量;
```

结构类型变量（结构变量）由一组类型可以不同的数据元素组成

结构定义的一般形式

```
struct 结构名 {
```

```
    类型  变量名;
```

```
    类型  变量名;
```

```
    .....
```

```
};
```

```
struct 结构名 结构变量;
```

结构

{	80001
	zh
	90
{	80002
	li
	96
{	80003
	wa
	84
	⋮

```
struct record {  
    int  number;  
    char name[2];  
    int  score;  
}
```

```
struct record people[100];
```

typedef（定义别名）：

```
typedef float real;
```

```
typedef struct {  
    char number;  
    char name;  
    char score;  
} study;
```

```
study people[100];  
study *p;
```

区分类型定义和变量定义

结构

`typedef 结构定义 结构类型名;`

`结构类型名 结构变量名;`

例 一本书可以用有2个成员（数据域）的结构变量存储。

```
typedef struct {  
    int no;  
    char title[40];  
}BookType;  
BookType book1;
```

结构指针

指针类型变量（指针变量）用于存储变量地址（或称指向该变量）
(只介绍本课程用到的指向结构变量指针类型)

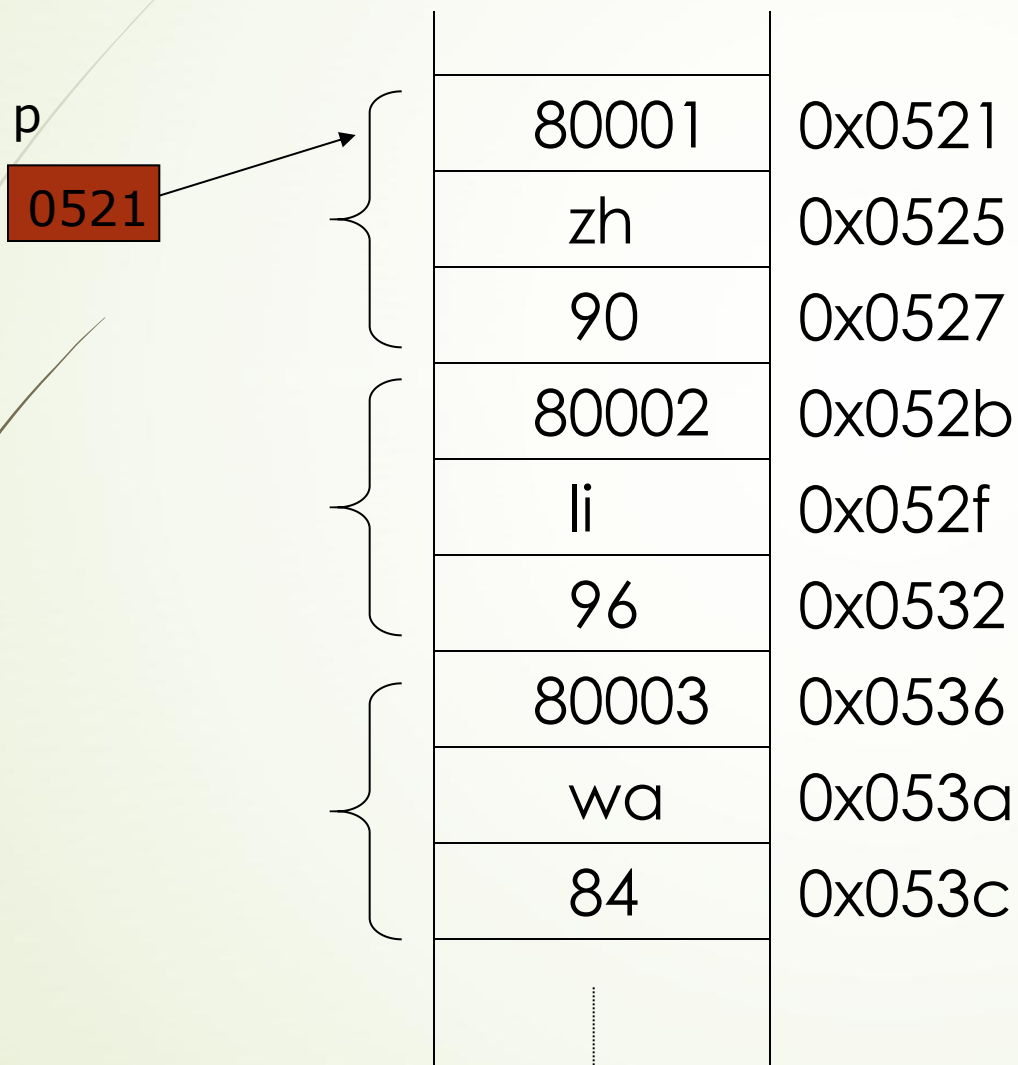
*typedef 结构定义 *指针的类型名;*

指针的类型名 指针变量名

- 例

```
typedef struct {  
    int no;  
    char title;  
}*BookPtr;  
BookPtr pbook;
```

结构指针



```
struct record {  
    int  number;  
    char name[2];  
    int  score;  
}
```

```
struct record *  
p;
```

```
p = 0x0521;  
x = (*p).score;  
p++;  
x = (*p).score;
```

```
y = p->score;
```

```
p = 0x0536;  
int i;  
i=*((int *)p)
```

1.5 C语言相关知识

数组的**类型定义**和**变量定义**

typedef 数组元素类型名 数组类型名[常量表达式];

typedef int Scores[30];

数组类型变量（数组变量）由一组类型相同的数据元素组成

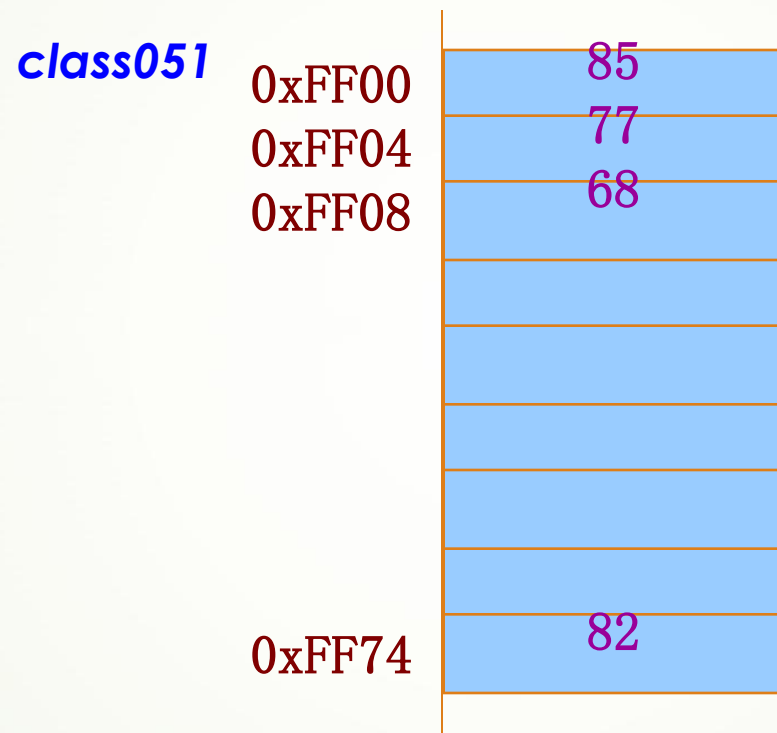
数组类型名 数组变量名;

例 某班30个学生的数学成绩，可以用有30个分量的整型数组变量存储。

typedef int Scores[30];

Scores class051;

- 该数组在内存中的存储示意图



在函数参数中使用变量

- C中只允许值传递，C++中允许值传递和引用传递
- 例：我们写一个交换两个整数的值的函数

C的方法

```
1. void Exchg2(int *px, int *py)
2. {
3.     int tmp = *px;
4.     *px = *py;
5.     *py = tmp;
6.     printf("*px = %d, *py = %d.\n", *px, *py);
7. }
8. main()
9. {
10.    int a = 4, b = 6;
11.    Exchg2(&a, &b);
12.    printf("a = %d, b = %d.\n", a, b);
13.    return(0);
14. }
```

输出结果为：

*px = 6, *py = 4.
a = 6, b = 4.

C++多了一个方法

```
1. void Exchg3(int &x, int &y)
2. {
3.     int tmp = x;
4.     x = y;
5.     y = tmp;
6.     printf("x = %d, y = %d.\n", x, y);
7. }
8. main()
9. {
10.    int a = 4, b = 6;
11.    Exchg3(a, b);
12.    printf("a = %d, b = %d.\n", a, b);
13.    return(0);
14. }
```

输出结果为：

x = 6, y = 4
a = 6, b = 4

值传递：将调用者的变量的地址值传递到函数里面去

```
1. void Exchg2(int *px, int *py)
2. {
3.     int tmp = *px;
4.     *px = *py;
5.     *py = tmp;
6.     printf("*px = %d, *py = %d.\n", *px, *py);
7. }
8. main()
9. {
10.    int a = 4, b = 6;
11.    Exchg2(&a, &b);
12.    printf("a = %d, b = %d.\n", a, b);
13.    return(0);
14. }
```

输出结果为：

x = 6, y = 4

a = 6, b = 4

main函数在调用Exchg2时，是main中的变量a和b的地址分别传递给了函数Exchg的参数px和py。有了变量的内存地址之后，就可以改变内存地址所保存的内容了。

在函数内对*px，*py进行操作时，也就是对a，b进行的操作。

引用传递：将调用者的变量自身传递到函数里面去

```
1. void Exchg3(int &x, int &y)
2. {
3.     int tmp = x;
4.     x = y;
5.     y = tmp;
6.     printf("x= %d,y = %d\n", x, y);
7. }
8. main()
9. {
10.    int a = 4,b =6;
11.    Exchg3(a, b);
12.    printf("a= %d, b = %d\n", a, b);
13.    return(0);
14. }
```

输出结果为：

x = 6,y = 4

a= 6, b = 4

Exchg3的两个参数，x、y前都有一个“&”，调用Exchg3时函数会将a、b分别代替了x、y了，我们称：**x、y分别引用了a、b变量**。这样函数里操作的其实就是a、b本身了，因此a和b的值可在函数里被修改。

在函数参数中使用变量

- C中只允许值传递，C++中允许值传递和引用传递
- 例：我们写一个交换两个整数的值的函数

在C和C++都是无效方法

```
1. void Exchg1(int x, int y)
2. {
3.     int tmp;
4.     tmp = x;
5.     x = y;
6.     y = tmp;
7.     printf("x = %d, y = %d\n", x, y);
8. }
9. main()
10. {
11.     int a = 4, b = 6;
12.     Exchg1(a, b);
13.     printf("a = %d, b = %d\n", a, b);
14.     return(0);
15. }
```

输出结果为：
x = 6, y = 4
a = 4, b = 6

C/C++语言中的函数参数采用**值传递**：意味着将用调用者变量a、b的值对函数参数x、y变量进行了赋值；是**单向传递**(由函数外向函数内传，不会反向再传回去)。

在函数内，对参数变量x、y的值进行任意的修改，都不会影响到调用者变量a、b**自身**的值。

在函数参数中使用变量

- C中只允许值传递，那么如何在函数内修改调用者的变量值？
- 通过值传递的方式，将需要修改的变量的内存地址传进去。

```
1. void Exchg2(int *px, int *py)
2. {
3.     int tmp = *px;
4.     *px = *py;
5.     *py = tmp;
6.     printf("*px = %d, *py = %d.\n", *px, *py);
7. }
8. main()
9. {
10.     int a = 4, b = 6;
11.     Exchg2(&a, &b);
12.     printf("a = %d, b = %d.\n", a, b);
13.     return(0);
14. }
```

在函数中，我们要改变的不是这个地址值；而是通过这个地址值来找到调用者变量的内存，进而通过改变这个内存的内容而改变调用者变量的值

main函数在调用Exchg2时，main函数中的变量a和b的地址被传入，这样在Exchg2中通过px和py来记录a和b的地址，进一步通过*px和*py来读写a和b的内存内容。用指针类型变量来完成这种值传递的参数传递。

下面三个函数，分别是：值传递（变量值）、引用传递、值传递（变量的指针值）

假设，以H和e为参数，分别调用三个函数，调用前：e== '#';(即字符'#'); H.uListLength == 0; H.uListSize==10; H.elem == 0x1234;从0x1234的地址开始的10个字符值,均为0xcd(即 H.elem[0]==0xcd; H.elem[1]==0xcd; ... H.elem[9]==0xcd;) **typedef char ElemType;**

<pre>bool ListTailInsertPassByValue(Sqlist L, ElemType e) { if((L.elem) && (L.uListLength < L.uListSize)) { L.elem[L.uListLength] = e; L.uListLength ++; return true; } else return false; }</pre>	<pre>bool ListTailInsertPassByRef(Sqlist &L, ElemType e) { if((L.elem) && (L.uListLength < L.uListSize)) { L.elem[L.uListLength] = e; L.uListLength ++; return true; } else return false; }</pre>	<pre>bool ListTailInsertPassByPoint(Sqlist *L, ElemType e) { if(!L) return false; if((L->elem) && (L->uListLength < L->uListSize)) { L->elem[L->uListLength] = e; L->uListLength ++; return true; } else return false; }</pre>
---	--	--

函数返回前L.elem[0]的值		函数返回前L.elem[0]的值		函数返回前L->elem[0]的值	
函数返回前L.uListLength的值		函数返回前L.uListLength的值		函数返回前L->uListLength的值	
调用函数后H.elem[0]的值		调用函数后H.elem[0]的值		调用函数后H.elem[0]的值	
调用函数后H.uListLength的值		调用函数后H.uListLength的值		调用函数后H.uListLength的值	
调用函数后H.elem的值		调用函数后H.elem的值		调用函数后H.elem的值	

下面三个函数，分别是：值传递（变量值）、引用传递、值传递（变量的指针值）

假设，以H和e为参数，分别调用三个函数，调用前：e== '#';(即字符'#'); H.uListLength == 0; H.uListSize==10; H.elem == 0x1234;从0x1234的地址开始的10个字符值,均为0xcd(即 H.elem[0]==0xcd; H.elem[1]==0xcd; ... H.elem[9]==0xcd;) **typedef char ElemType;**

<pre>bool ListTailInsertPassByValue(Sqlist L, ElemType e) { if((L.elem) && (L.uListLength < L.uListSize)) { L.elem[L.uListLength] = e; L.uListLength ++; return true; } else return false; }</pre>	<pre>bool ListTailInsertPassByRef(Sqlist &L, ElemType e) { if((L.elem) && (L.uListLength < L.uListSize)) { L.elem[L.uListLength] = e; L.uListLength ++; return true; } else return false; }</pre>	<pre>bool ListTailInsertPassByPoint(Sqlist *L, ElemType e) { if(!L) return false; if((L->elem) && (L->uListLength < L->uListSize)) { L->elem[L->uListLength] = e; L->uListLength ++; return true; } else return false; }</pre>
---	--	--

函数返回前L.elem[0]的值	'#'	函数返回前L.elem[0]的值	'#'	函数返回前L->elem[0]的值	'#'
函数返回前L.uListLength的值	1	函数返回前L.uListLength的值	1	函数返回前L->uListLength的值	1
调用函数后H.elem[0]的值	'#'	调用函数后H.elem[0]的值	'#'	调用函数后H.elem[0]的值	'#'
调用函数后H.uListLength的值	0	调用函数后H.uListLength的值	1	调用函数后H.uListLength的值	1
调用函数后H.elem的值	0x1234	调用函数后H.elem的值	0x1234	调用函数后H.elem的值	0x1234

问题：我不明白的是当指针作为参数的时候，不是直接可以修改吗

- 1、这里提到的“可以修改”，是对指针指向的变量进行修改，可以带到函数外；对指针本身值的修改，按照值传递的调用原则，是只能在本函数内用，而无法带到函数外的。
- 2、在值传递原则下，传递到函数内的是指针变量的值，这个值保存在函数的参数变量中；我们对这个参数变量的值做的任何修改，在值传递情况下（单向传递情况下），不会对调用者的变量值有任何改变；我们对这个参数变量值对应的内存(即指向的值)的任何修改，都会带出函数，这种通过内存地址(即指针值)对调用者的变量值的修改是直接马上生效的修改。

```
void abc2(linkhead H, Inode*e)
{
    H = (linkhead)malloc(sizeof(node));
    if(H)
    {
        (*H).data = 100;
    }
}
```

错误！
最常见在函数内
初始化一个调用
者的指针错误!!!
如何改正？

```
void abc1(linkhead H, Inode*e)
{
    if(H)
    {
        linkhead S = (linkhead)malloc(sizeof(node));
        H = S;
        (*H).data = 100;
    }
}
```

错误！
导致内存丢失(S丢失了),
还无法为H->data赋值

```
void abc3(linkhead H, Inode*e)
{
    if(H)
    {
        (*H).data = 100;
    }
}
```

正确

问题：我不明白的是当指针作为参数的时候，不是直接可以修改吗

- 1、这里提到的“可以修改”，是对指针指向的变量进行修改，可以带到函数外；对指针本身值的修改，按照值传递的调用原则，是只能在本函数内用，而无法带到函数外的。
- 2、在值传递原则下，传递到函数内的是指针变量的值，这个值保存在函数的参数变量中；我们对这个参数变量的值做的任何修改，在值传递情况下（单向传递情况下），不会对调用者的变量值有任何改变；我们对这个参数变量值对应的内存（即指向的值）的任何修改，都会带出函数，这种通过内存地址（即指针值）对调用者的变量值的修改是直接马上生效的修改。

```
void abc2(linkhead H, Inode*e)
{
    H = (linkhead)malloc(sizeof(node));
    if(H)
    {
        (*H).data = 100;
    }
}
```

错误！
最常见在函数内
初始化一个调用
者的指针错误!!!
如何改正？

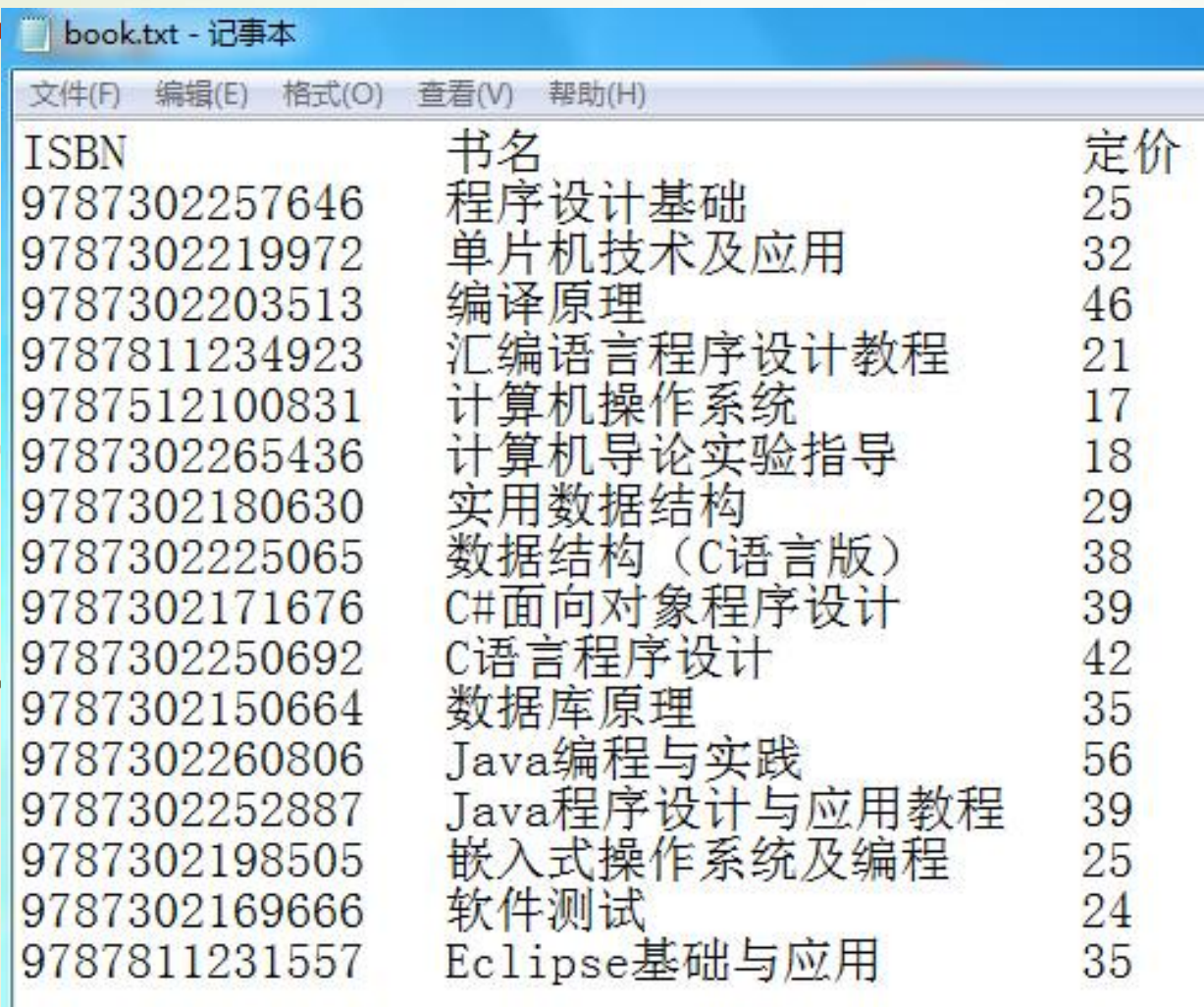
```
void abc4(linkhead * H, Inode * e )
{
    (*H) = (linkhead)malloc(sizeof(node));
    if(*H)
    {
        (*H)->data=100;
    }
}
```



第2章 线性表

线性表的链式存储

案例2.3：图书信息管理系统



ISBN	书名	定价
9787302257646	程序设计基础	25
9787302219972	单片机技术及应用	32
9787302203513	编译原理	46
9787811234923	汇编语言程序设计教程	21
9787512100831	计算机操作系统	17
9787302265436	计算机导论实验指导	18
9787302180630	实用数据结构	29
9787302225065	数据结构（C语言版）	38
9787302171676	C#面向对象程序设计	39
9787302250692	C语言程序设计	42
9787302150664	数据库原理	35
9787302260806	Java编程与实践	56
9787302252887	Java程序设计与应用教程	39
9787302198505	嵌入式操作系统及编程	25
9787302169666	软件测试	24
9787811231557	Eclipse基础与应用	35

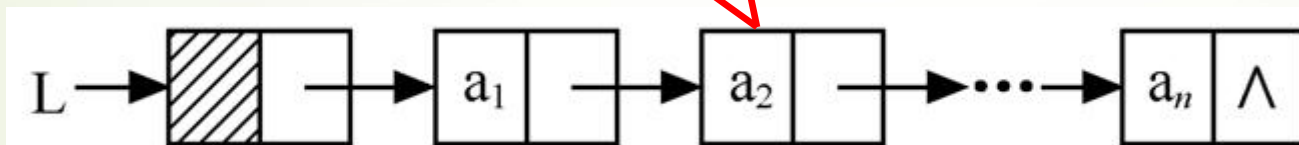
- (1) 查找
- (2) 插入
- (3) 删除
- (4) 修改
- (5) 排序
- (6) 计数

图书表的顺序存储

elem[0]	elem[1]	elem[2]	...	elem[length-1]	空闲区	
a_1	a_2	a_3	...	a_{length}		



图书表的链式存储



2.3 线性表的链式存储

结点在存储器中的位置是任意的，即逻辑上相邻的数据元素在物理上不一定相邻

为建立起数据元素之间的线性关系，对每个数据元素 a_i ，除了存放数据元素的自身的信息 a_i 之外，还需要和 a_i 一起存放其后继元素 a_{i+1} 所在的存储单元的地址，这两部分信息组成一个“结点”：

data	next
------	------

各结点由两个域组成：

数据域：存储元素数值数据

指针域：存储直接后继结点的存储位置

线性表的链式表示又称为**非顺序映像**或**链式映像**。

	数据域	指针域
112		
111		
110	a2	109
109	a3	103
108		
107		
106	a1	110
105		
104		
103	a4	0
102		
101		
100		

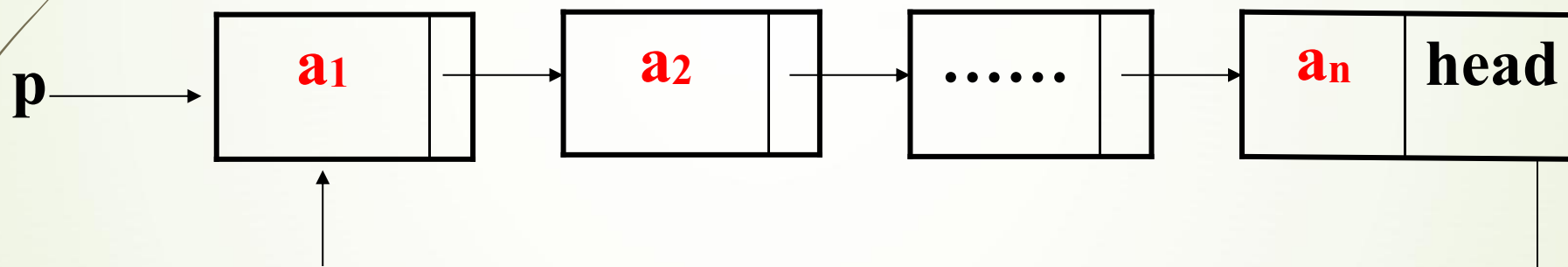
与链式存储有关的术语

- 1、**结点**：数据元素的存储映像。由数据域和指针域两部分组成
- 2、**链表**： n 个结点由**指针链**组成一个链表。它是线性表的链式存储映像，称为线性表的链式存储结构

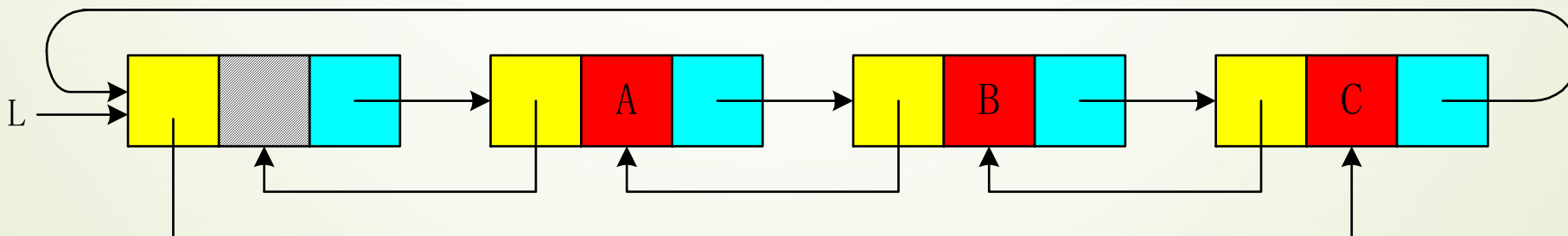
3、单链表、双链表、循环链表：

- 结点只有一个指针域的链表，称为单链表或线性链表
- 有两个指针域的链表，称为双链表
- 首尾相接的链表称为循环链表

循环链表示意图：



双向循环链表示意图：

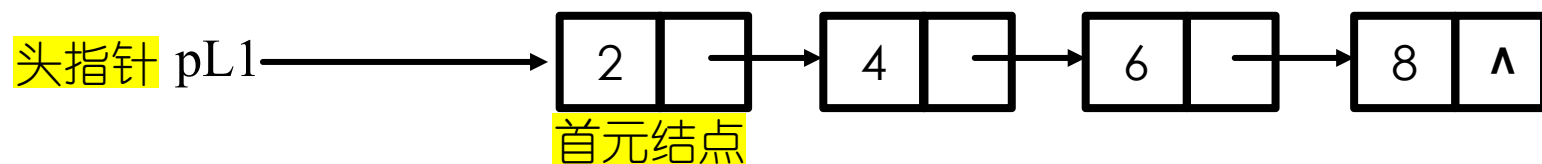


带头结点的和不带头结点的单链表

结点	
data	next
数据域	指针域

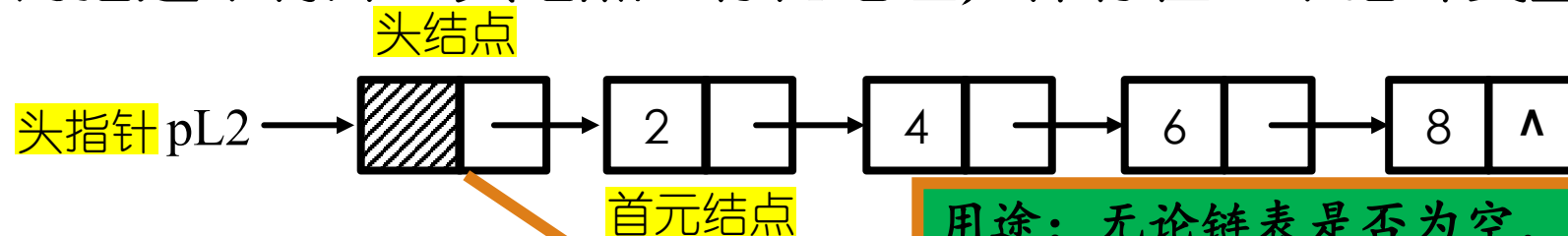
对于逻辑的线性表(2,4,6,8)，采用链式存储方式存储时：

- 1、需要有4个结点，每个结点的数据域存储对应的数据元素
- 2、每个结点要保存“后继元素 a_{i+1} 的存储地址”，“后继元素 a_{i+1} 的存储地址”保存在结点的指针域；无后继元素时，结点指针域保存 NULL 值（NULL即为0，也写做null）
- 3、还需要用一个变量把这个“链”记录下来，即需要保存第一个结点的存储地址，此地址值可以保存在一个指针类型变量pL1中（称为pL1指向这个链表）



不带头结点单链表

- 4、也可以把第一个结点的存储地址保存在一个特殊的结点（称为头结点）的指针域；再把这个特殊的头结点的存储地址，保存在一个指针类型的变量pL2中



带头结点单链表

用途：无论链表是否为空，头指针始终保持值不变；插入和删除第一个数据元素时不必对头指针进行特殊处理。

如何实现？

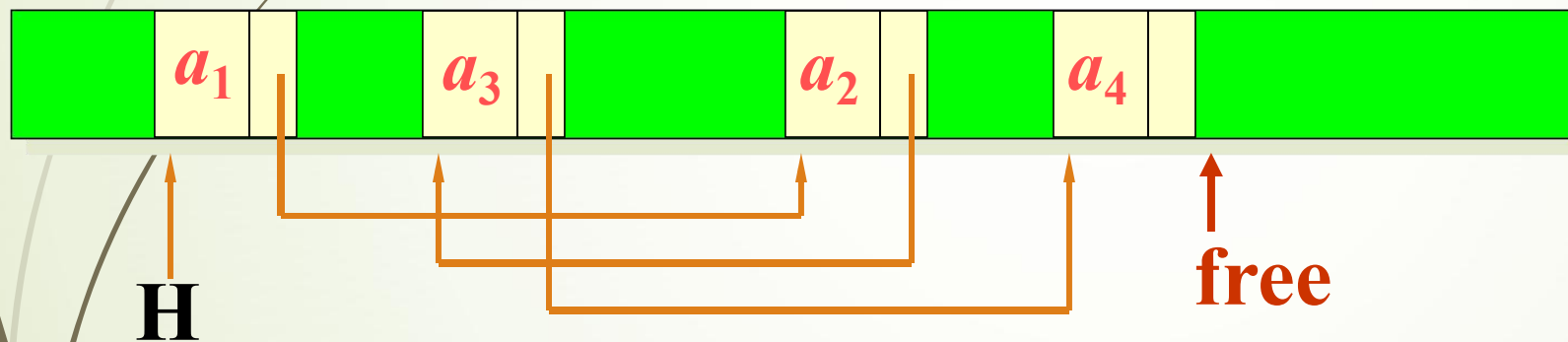
通过**指针**来实现

单链表的存储映像



↑
free

(a) 可利用存储空间



(b) 经过一段运行后的无头结点的单链表结构

110↔	a5↔	200↔	↔
...↔	↔	...↔	↔
150↔	a2↔	190↔	↔
160↔	a1↔	150↔	↔
...↔	↔	...↔	↔
190↔	a3↔	210↔	↔
200↔	a6↔	260↔	↔
210↔	a4↔	110↔	↔
...↔	↔	...↔	↔
240↔	a8↔	NULL↔	↔
...↔	↔	...↔	↔
260↔	a7↔	240↔	↔

链式存储结构 ↔

H↔
160↔

链表（链式存储结构）的特点

- (1) 结点在存储器中的位置是任意的，即逻辑上相邻的数据元素在物理上不一定相邻
- (2) 访问时只能通过头指针进入链表，并通过每个结点的指针域向后扫描其余结点，所以寻找第一个结点和最后一个结点所花费的时间不等

这种存取元素的方法被称为**顺序存取法**

链表的优缺点

优点

- 数据元素的个数可以自由扩充
- 插入、删除等操作不必移动数据，只需修改链接指针，修改效率较高

缺点

- 存储密度小
- 查找时效率不高，必须采用顺序存取，即查找数据元素时，只能按链表的某种顺序进行访问（顺藤摸瓜）

练习

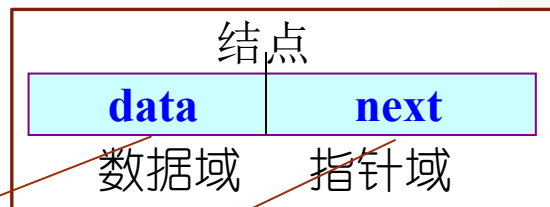
- ✗ 1. 链表的每个结点中都恰好包含 **一个或多个指针**
- ✗ 2. **链表** 结构适宜于进行顺序存取，而 **顺序表** 适宜于进行随机存取。
- ✗ 3. 顺序存储方式的优点是存储密度大，且插入、删除运算 **效率低**。
- ✗ 4. 线性表若采用链式存储时，结点之间 **任意** **内部** 的存储空间都是可以不连续的。
- ✗ 5. 线性表的每个结点 **可以** 是一个简单类型，**也可以是复杂类型**，而链表的每个结点可以是一个复杂类型

2.3 线性表的链式存储与实现

结点的c语言类型定义如下:

```
typedef struct LNode
{
    ElemType data;
    struct LNode *next;
} LNode, *LinkList;
```

// LNode 这个结构体类型, 是结点的类型
// LinkList 这个指针类型, 是结点指针类型
// LinkList 等同于 LNode * 类型



//声明结点类型的变量:

```
LNode node;
```

//声明结点指针类型的变量:

```
LinkList pL1=NULL;
```

```
LNode * pL2=NULL;
```

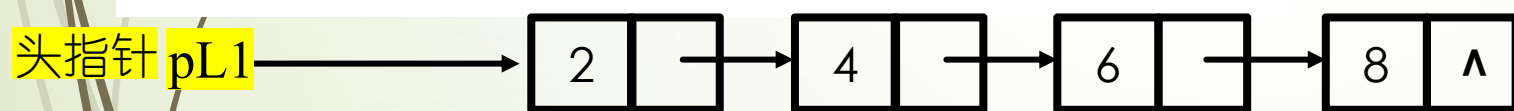
// pL1,pL2均为指针类型变量

//动态申请结点内存:

```
pL1=(LNode *)malloc(sizeof(LNode));
```

//动态释放一个已使用的内存:

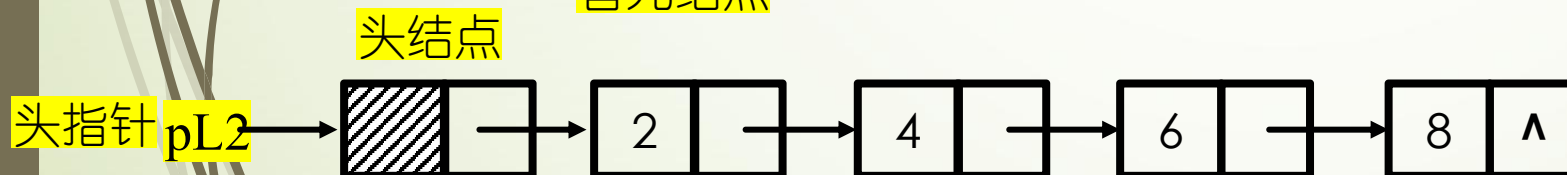
```
if (pL1) free(pL1);
```



首元结点

空表:
pL1==NULL

不带头结点单链表



头结点

首元结点

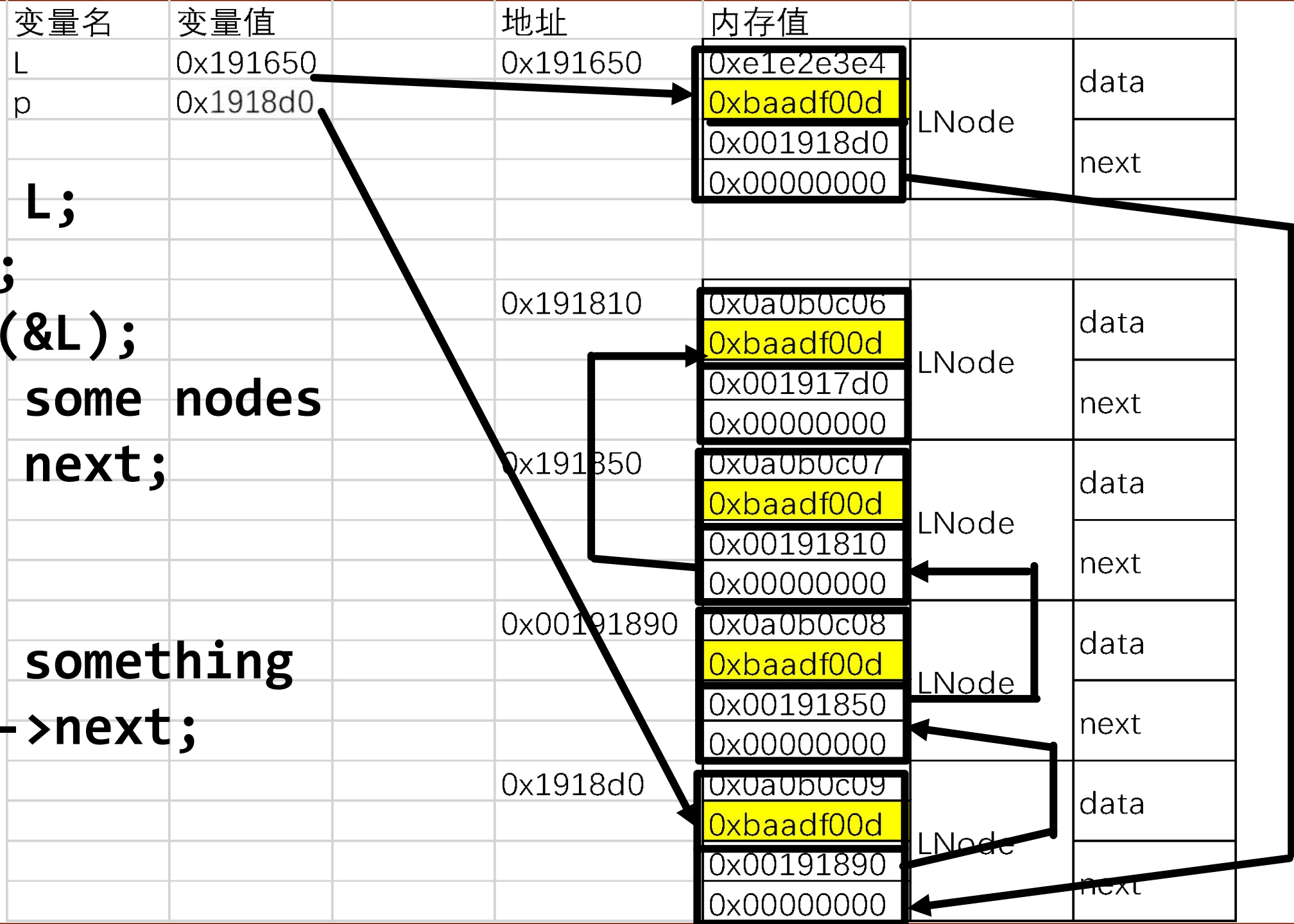
空表:
pL2->next
==NULL

带头结点单链表

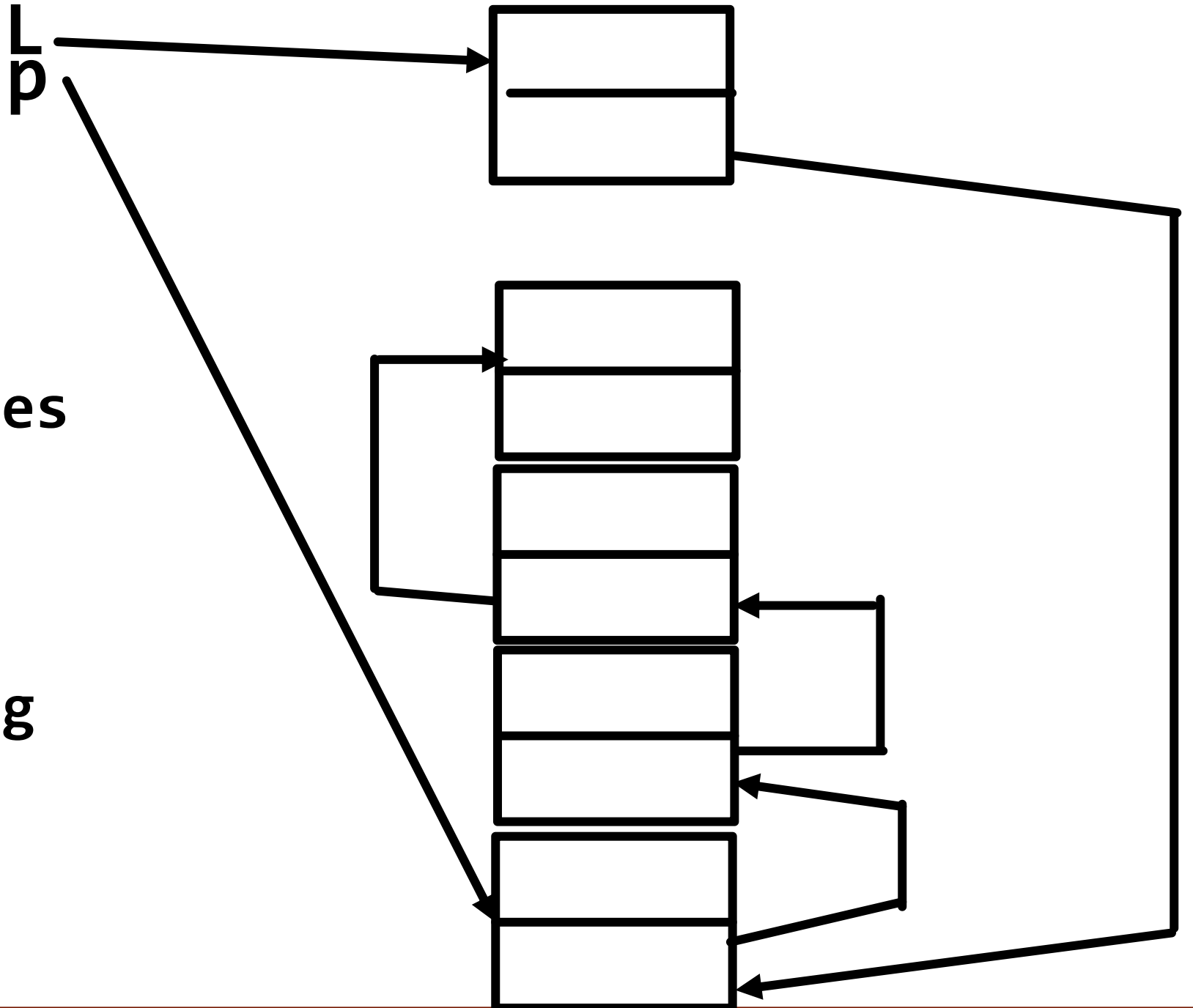
```

LinkedList L;
lNode *p;
InitList(&L);
//Insert some nodes
p = L -> next;
While(p)
{
    // do something
    p = p->next;
}

```



```
LinkedList L;  
lNode *p;  
InitList(&L);  
//Insert some nodes  
p = L -> next;  
While(p)  
{  
    // do something  
    p = p->next;  
}
```



线性表的ADT定义

➡ 基本操作：

- ➡ (1) 线性表初始化： InitList(&L)
- ➡ (2) 销毁线性表： DestroyList(&L)
- ➡ (3) 将线性表置空： ClearList(&L)
- ➡ (4) 判断线性表是否为空： ListEmpty(L)
- ➡ (5) 求线性表的长度： ListLength (L)
- ➡ (6) 按位置取表中某个元素： GetElem(L,i,&e)

线性表的ADT定义

基本操作：

- ➡ (7)按值查找: `LocateElem(L,e,compare())`
- ➡ (8)前驱: `PriorElem(L,cur_e, &pre_e)`
- ➡ (9)后继: `NextElem(L,cur_e,&next_e)`
- ➡ (10)插入操作: `ListInsert (&L,i,e)`
- ➡ (11) 删除操作: `ListDelete (&L,i,&e)`
- ➡ (12) 遍历操作: `ListTraverse (L,visit())`

在遍历操作中，用visit()函数
指针作为参数
即遍历到某个结点时，由
visit()函数来具体对结点的数
据项进行操作

带头结点的单链表的基本运算

带头结点的
单链表

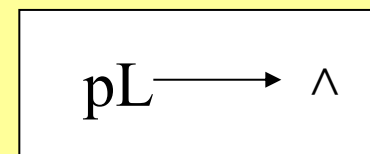
(1) 构造一个空的线性单链表，要求输入的

pL是空指针或未经初始化的指针

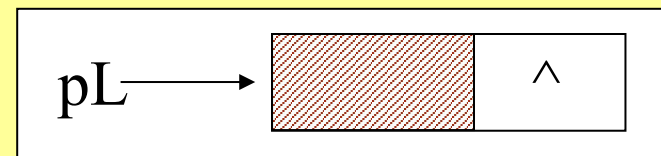
```
Status InitList_L(LinkList &pL)
{
    pL = (LinkList)malloc(sizeof(LNode));
    if (!pL) return OVERFLOW;
    pL->next= NULL;
    return OK;
} //InitList_L O(1)
```

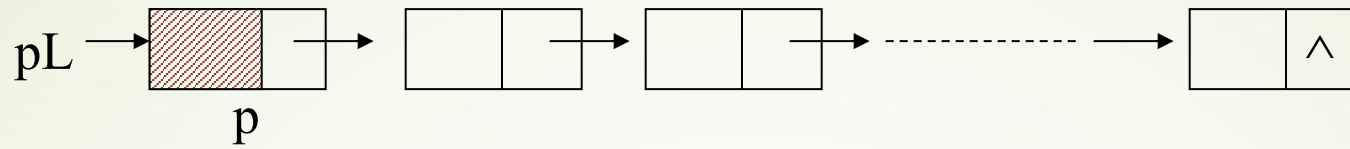
```
int main()
{
    LinkList pL=NULL;
    if(InitList_L(pL)!=OK)
        exit(0)
    //....
    DestroyList_L(pL);
    return 1;
} //使用InitList_L示例
```

调用InitList_L之前:



调用InitList_L之后:





带头结点的单链表

(2)销毁已存在线性单链表，参数是指向头结点的指针变量pL

```
void DestroyList_L(LinkList &pL)
```

```
//销毁已存在线性单链表pL
```

```
{
```

```
    if(!pL) return;
```

```
    LNode *p= pL;
```

```
    while (p) {
```

```
        LNode * q= p->next;
```

```
        free(p);
```

```
        p=q;
```

```
    }
```

```
    pL=NULL;
```

```
} // DestroyList_L    O(n)
```

带头结点的单链表的基本运算

带头结点的
单链表

(3) 构造一个空的线性单链表，要求输入的**L是头结点变量**

```
Status InitList_L(LNode &L)
```

```
{
```

```
    L.next= NULL;
```

```
    return OK;
```

```
}//InitList_L  O(1)
```

```
int main()
```

```
{
```

```
    LNode H;
```

```
    if(InitList_L(H)!=OK)
```

```
        exit(0)
```

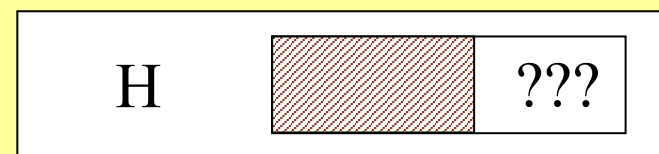
```
    //....
```

```
    DestroyList_L(H);
```

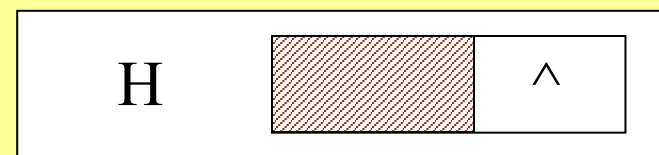
```
    return 1;
```

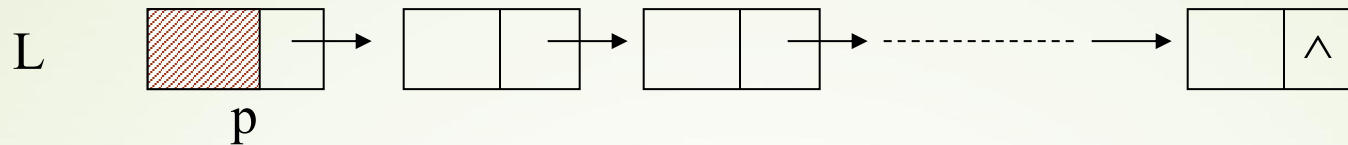
```
}//使用InitList_L示例
```

调用InitList_L之前:



调用InitList_L之后:





带头结点的 单链表

(4)销毁已存在线性单链表，参数是指向头结点的结构体变量 L

```
void DestroyList_L(LNode &L)
```

```
//销毁已存在线性单链表L
```

```
{
```

```
    LNode *p= L.next;
```

```
    while (p) {
```

```
        LNode * q= p->next;
```

```
        free(p);
```

```
        p=q;
```

```
    }
```

```
    L.next=NULL;
```

```
} // DestroyList_L     $O(n)$ 
```

两种销毁函数，逐行对比着看看

```
void DestroyList_L(LinkList &pL)
```

```
//销毁已存在线性单链表pL
```

```
{
```

```
    if(!pL) return;
```

```
    LNode *p= pL;
```

```
    while (p) {
```

```
        LNode * q= p->next;
```

```
        free(p);
```

```
        p=q;
```

```
    }
```

```
    pL=NULL;
```

```
} // DestroyList_L    O(n)
```

```
void DestroyList_L(LNode &L)
```

```
//销毁已存在线性单链表L
```

```
{
```

```
    LNode *p= L.next;
```

```
    while (p) {
```

```
        LNode * q= p->next;
```

```
        free(p);
```

```
        p=q;
```

```
    }
```

```
    L.next=NULL;
```

```
} // DestroyList_L    O(n)
```

两种初始化函数，逐行对比着看看

```
Status InitList_L(LinkList &pL)
```

```
{
```

```
    pL = (LinkList)malloc(sizeof(LNode));
```

```
    if (!pL) return OVERFLOW;
```

```
    pL->next= NULL;
```

```
    return OK;
```

```
}//InitList_L  O(1)
```

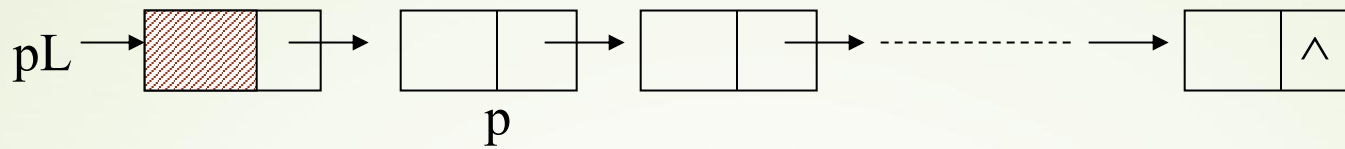
```
Status InitList_L(LNode &L)
```

```
{
```

```
    L.next= NULL;
```

```
    return OK;
```

```
}//InitList_L  O(1)
```



带头结点的 单链表

(5)清空已存在线性单链表pL

```
void ClearList_L(LinkList &pL)
```

```
//清空已存在的线性单链表，参数是指向头结点的指针变量pL
```

```
{
```

```
    if(!pL) return;
```

```
    LNode *p= pL->next;
```

```
    while (p) {
```

```
        LNode *q= p->next;
```

```
        free(p);
```

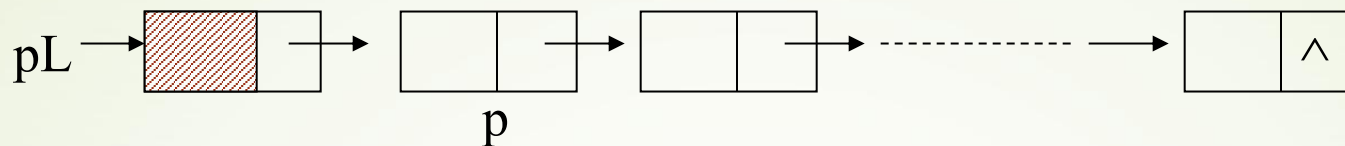
```
        p=q;
```

```
    }
```

```
    pL->next=NULL;
```

```
} // ClearList_L    0(n)
```

带头结点的单链表



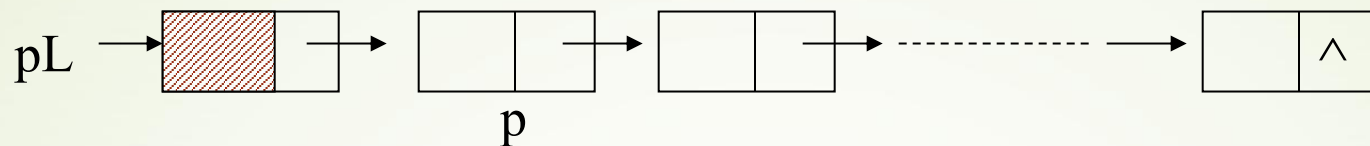
(6)判定已存在线性单链表pL是否为空

```
bool ListEmpty_L(LinkList pL)
//判定已存在线性单链表L是否为空
{
    if(!pL) return FALSE;
    if(pL->next) return FALSE;
    return TRUE;
} // ListEmpty_L O(1)
```

这是有头结点的单链表，计数时从下一个结点开始。

(7)求线性单链表pL中数据元素的个数

```
int ListLength_L(LinkList pL)
//求线性单链表L中数据元素的个数
{
    if(!pL) return 0;
    int len=0;
    LNode *p= pL->next;
    while (p) {
        ++len;
        p= p->next;
    }
    return len;
} // ListLength_L O(n)
```



```
void someListFunc_L(LinkedList &pL)
```

```
//参数是指向头结点的指针变量pL，常用的代码结构 - while(pL)
{
```

```
    if(!pL) return;
```

```
    LNode *p= pL->next;
```

```
    while (p) {
```

```
        //对p指向的结点进行一些操作。
```

```
        p=p->next; //
```

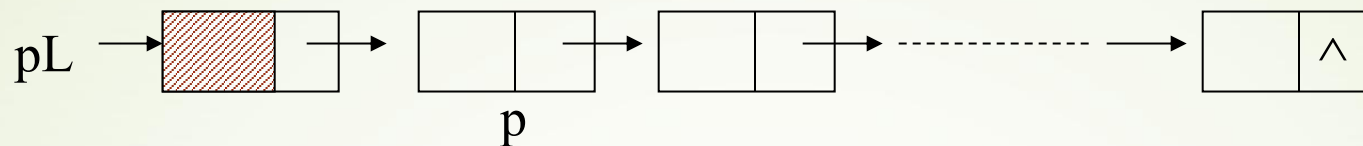
```
    }
```

```
    pL->next=NULL;
```

```
} // 基于遍历的操作实现，实际复杂度O(n)
```

- 1、这是有头结点的单链表，实际的数据元素结点从头结点的下一个结点开始。
- 2、要注意链表遍历时要养成习惯：另外声明一个变量p，让p去从L->next开始向后移动，去遍历其他元素，在这个过程中完成需要操作
- 3、不要让pL=pL->next，这样就丢失了原链表的头指针值了。

带头结点的单链表



(8)取线性单链表L中的第*i*个元素e的值

```
Status GetElem_L(LinkList pL,int i,ElemType &e)
```

```
//取线性单链表L中的第i个元素e的值
```

```
{
    if(!pL) return ERROR;
    LNode *p= pL->next;
    int j=1;
    while ( p && j<i ) {
        p= p->next; ++j;
    }
    if ( !p || j>i ) return ERROR;
    e= p->data;
    return OK;
} // GetElem_L    O(n) P29-2.8
```

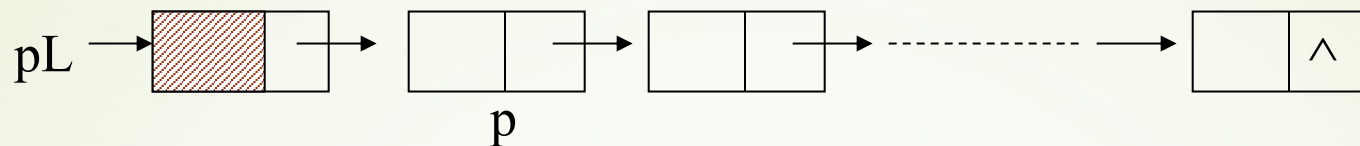
(9)取线性单链表L中的第*i*个结点的地址

```
LNode*GetElem_L(LinkList pL,int i,ElemType &e)
```

```
//取线性单链表L中的第i个结点的地址
```

```
{
    if(!pL) return NULL;
    LNode *p= pL->next;
    int j=1;
    while ( p && j<i ) {
        p= p->next; ++j;
    }
    if ( !p || j>i ) return NULL;
    e= p->data;
    return p;
} // GetElem_L    O(n)
```

带头结点的单链表



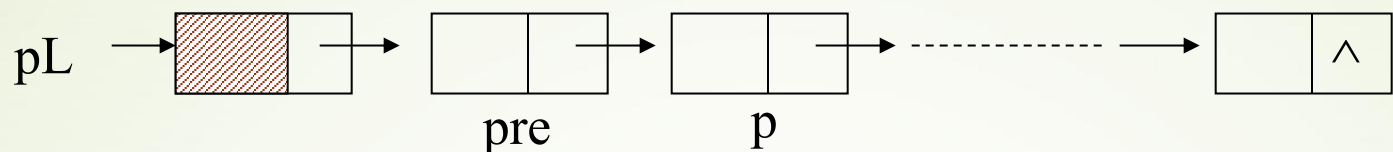
(10) 查找第1个值相等的数据元素的序号

```
int LocateElem_L(LinkList pL, ElemType e,
    Status (*compare)(ElemType, ElemType) )
//在线性单链表L中查找第1个值与e满足
//compare ( ) 的元素位序
{
    if(!pL) return 0;
    LNode * p= pL->next;  int i=1;
    while (p) {
        if ( (*compare)(p->data,e)==OK )
            return i;
        p= p->next; ++i;
    }
    return 0;           //未找到
} // LocateElem_L      O(n)
```

(11) 查找第1个值相等的数据元素的结点地址

```
(LNode *)LocateElem_L(LinkList pL, ElemType e,
    Status (*compare)(ElemType, ElemType) )
//在线性单链表L中查找第1个值与e满足
//compare ( ) 的元素地址指针
{
    if(!pL) return 0;
    LNode * p= pL->next;
    while (p) {
        if ( (*compare)(p->data,e)==OK )
            return p;
        p= p->next;
    }
    return NULL;        //未找到
} // LocateElem_L      O(n)
```

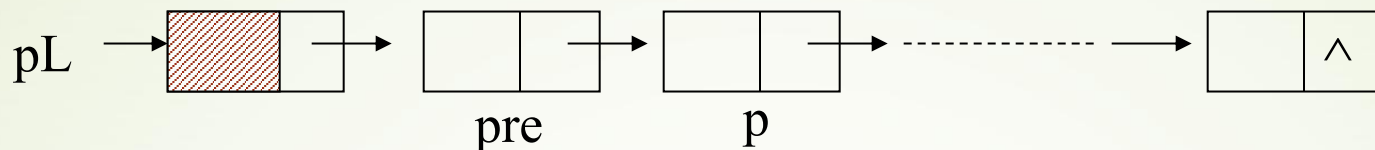
带头结点的
单链表



(12)若值为cur_e的结点是线性表L的元素且不是第一个，返回它的前驱

```
(LNode*) PriorElem_L(LinkList pL, ElemType cur_e,   ElemaType &pre_e)
{
    if(!pL) return NULL;
    LNode * pre=pL->next;
    if ( !pre  ||  !pre->next  )  return NULL;
    LNode *p=pre->next;
    while ( p ) {
        if ( p->data==cur_e ) {
            pre_e= pre->data;    //取前驱元素的值
            return pre ;        //返回前驱元素的指针
        }
        pre=p;  p= p->next;
    }
    return NULL;
} //PriorElem_L  O(n)
```

带头结点的单链表



(13) 若值为`cur_e`的结点是线性表`L`的元素且不是最后一个，返回它的后继的值和地址

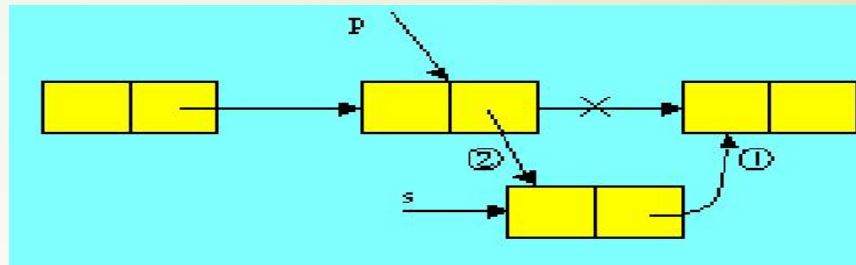
```
(LNode*) NextElem_L(LinkList pL, ElemType cur_e,   ElemaType &next_e)
```

```
{  
    if(!pL) return NULL;  
    LNode * p=pL->next;  
    while ( p  && p->data != cur_e)    p=p->next;  
    if ( p && p->next ) {  
        next_e= p->next->data ;           //取后继元素的值  
        return  p->next ;                 //返回后继元素的指针  
    }  
    return NULL;  
} //NextElem_L  O(n)
```

对比一下两种向后移动指针的方法

```
while ( p ) {  
    if ( p->data==cur_e ) {  
        pre_e= pre->data;   //取前驱元素的值  
        return  pre ;       //返回前驱元素的指针  
    }  
    pre=p;  p= p->next;  
}
```

带头结点的单链表



(14)在单链线性表L中第*i*个位置之前插入元素e

```
Status ListInsert_L(LinkList &pL, int i, ElemType e)
{
    if(!pL) return ERROR;
    LNode *p=pL;    int j=0; // 考虑到有空表的可能性, 从0开始计数
    //将p从头节点移动到i-1结点, 在i-1结点后插结点
    while (p && j<i-1)    {
        p=p->next;    j++    // 每移动1个节点, j++, 直到j==i-1
    }
    if ( !p || j>i-1 ) return ERROR ;
    LNode * s = (LNode *)malloc(sizeof(LNode));
    if(!s) reurn ERROR;
    s->data= e ;

    s->next= p->next;    p->next= s;

    return OK ;
} // ListInsert_L    O(n)    P30-2.9
```

插入结点实际上都是在某结点之后插:

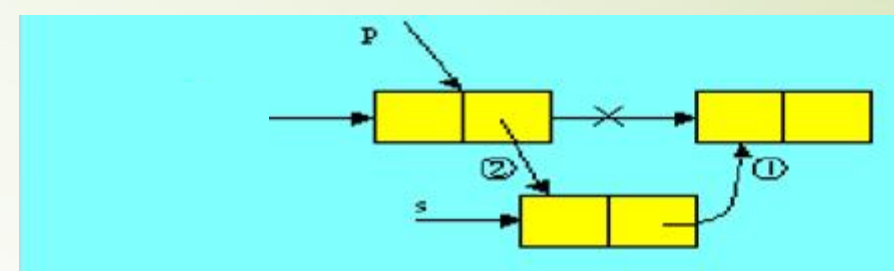
设p指向单链表中某结点, s指向待插入的值为x的新结点, 将*s插入到*p的后面。

操作如下:

- 1、创建新结点s
- 2、s->next = p->next
- 3、p->next = s;

注意: 两个指针的操作顺序不能交换 (先连上, 再打断原链)

带头结点的 单链表



(15)在单链线性表L的头部插入元素e，头插入

```
Status ListInsertHeader_L(LinkList &pL, ElemType e)
```

```
{
```

```
    if(!pL) return ERROR;
```

```
    LNode * s = (LNode *)malloc(sizeof(LNode));
```

```
    if(!s) reurn ERROR;
```

```
    s->data= e ;
```

```
    LNode * p = pL; //头结点，头插法就是在头结点之后插新结点
```

```
    s->next= p ->next;    p->next = s;
```

```
    return OK ;
```

```
}// ListInsert_L    0(1)
```

插入结点实际上都是在某结点之后插：

设p指向单链表中某结点，s指向待插入的值为x的新结点，将*s插入到*p的后面。

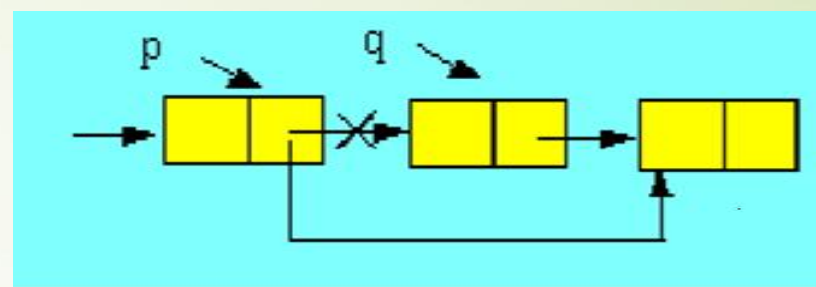
操作如下：

1、创建新结点s

2、s->next = p->next

3、p->next = s;

注意：两个指针的操作顺序不能交换（先连上，再打断原链）



(16)在单链线性表L中删除第i个元素，并由e返回其值

```
Status ListDelete_L(LinkList &pL, int i, ElemType &e)
{
    if(!pL) return ERROR;
    LNode *p=pL;   int j=0;  // 考虑到有空表的可能性，从0开始计数
    //将p移动到i-1位置，在i-1结点后删除结点
    while ( p->next  &&  j<i-1){
        p=p->next;    j++
    }
    if( !(p->next)  ||  j>i-1 ) return ERROR ;

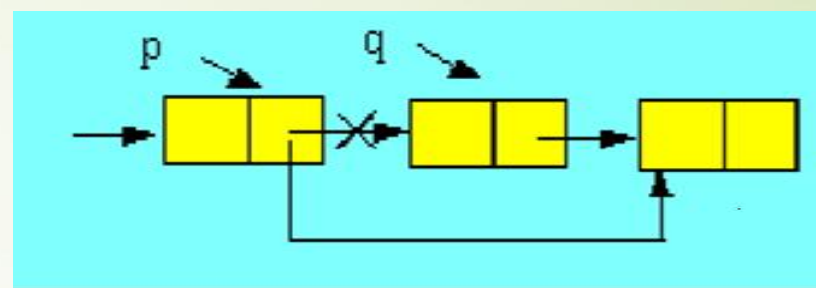
    LNode * q= p->next;
    p->next= q->next;
    e= q->data;
    free(q);
    return OK ;
} // ListDelete_L  O(n)    30-2.10
```

后删除结点：

设p指向单链表中某结点，计划删除p后面的节点，操作如下：

- 1、 q=p->next;
- 2、 p->next = q->next
- 3、 //对q指向的节点操作
- 4、 free(q);

注意：先用临时变量q保存p后面的结点；再把p和q后面的结点连上，最后释放q。指针操作顺序不能交换（先连上，再释放）



(17)在单链线性表L中删除第i个元素，并由e返回其值

```
Status ListDelete_L(LinkList &pL, int i, ElemType &e)
{
    LNode *p;
    p = GetElem_L(pL, i-1, e);
    if(!p) // i-1结点 不存在
        return ERROR;
    if(!p->next) // i结点 不存在
        return ERROR;

    LNode * q= p->next;
    p->next= q->next;
    e= q->data;
    free(q);
    return OK ;
} // ListDelete_L O(n)
```

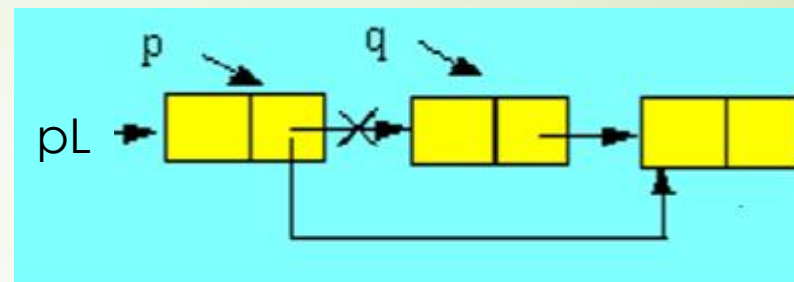
后删除结点：

设p指向单链表中某结点，计划删除p后面的节点，操作如下：

- 1、 $q = p \rightarrow next$;
- 2、 $p \rightarrow next = q \rightarrow next$
- 3、//对q指向的节点操作
- 4、 $free(q)$;

注意：先用临时变量q保存p后面的结点；再把p和q后面的结点连上，最后释放q。指针操作顺序不能交换（先连上，再释放）

带头结点的 单链表



(18)在单链线性表L中删除第1个元素，并由e返回其值，**头删除**

```
Status ListHeaderDelete_L(LinkList &pL, ElemType &e)
{
    if(!L) return ERROR;

    LNode *p = pL;
    if(!p->next) // 首元结点不存在
        return ERROR;

    LNode * q= p->next;
    p->next= q->next;
    e= q->data;
    free(q);
    return OK ;
} // ListDelete_L O(1)
```

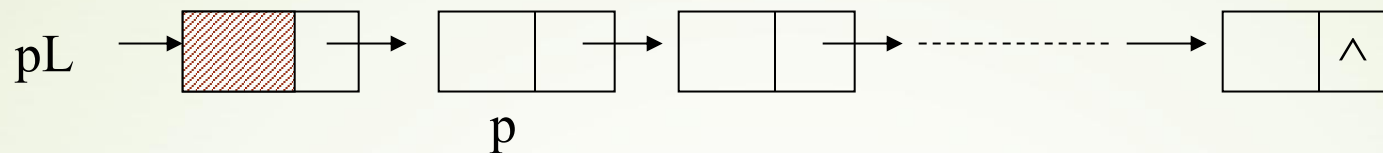
后删除结点：

设p指向单链表中某结点，计划删除p后面的节点，操作如下：

- 1、q=p->next;
- 2、p->next = q->next
- 3、//对q指向的节点操作
- 4、free(q);

注意：先用临时变量q保存p后面的结点；再把p和q后面的结点连上，最后释放q。指针操作顺序不能交换（先连上，再释放）

带头结点的单链表



(19)在单链线性表L中遍历，即访问全部元素一遍

```
Status ListTraverse_L(LinkList pL, Status (*visit)(ElemType) )
//依次对已存在线性表L中每个数据元素调用函数visit( )
{
    if(!pL) return ERROR;
    LNode *p= pL->next;
    while (p) {
        if (!(*visit)(p->data) ) return ERROR;    //有调用失败
        p= p->next;
    }
    return OK;    //全部被遍历
} // ListTraverse_L    O(n)
```

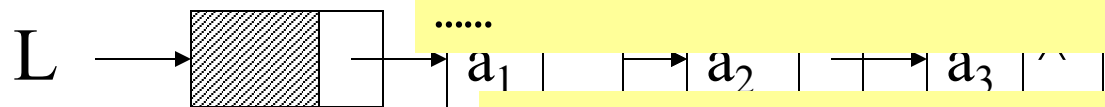
单链表的最基本函数框架：
用while循环，将p指针移动到最后一个元素

[生成带头结点的单链线性表] 初始化表+反复插入元素

设线性表数据元素为

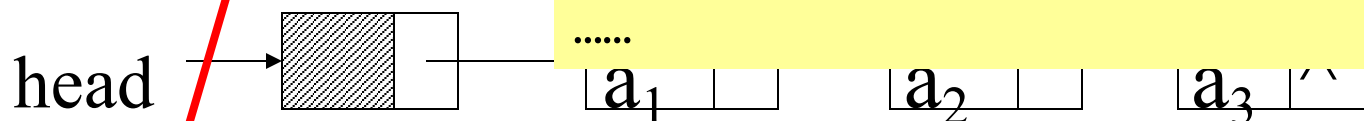
1) 头插法：插入

```
.....  
InitList_L(L);  
for (i=n-1; i>=0; i--)  
    ListInsert_L(L, 1, a[i]);
```



2) 尾插法：插入

```
.....  
InitList_L(head);  
for (i=0; i<n; i++)  
    ListInsert_L(head, i+1, a[i]);
```



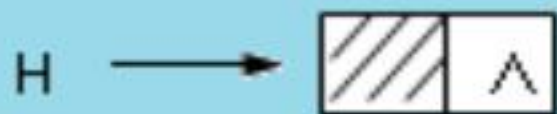
提示我们，是不是可以增加一个尾指针指向最后一个元素来提高尾插法的性能。

头结点问题的说明：

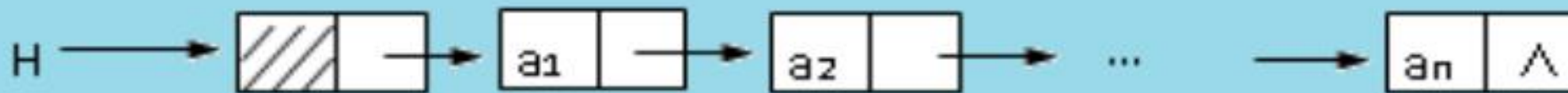
头结点的加入完全是为了运算的方便（注：另外一方面有人认为是增加了逻辑的复杂性），它的数据域无定义，指针域中存放的是第一个数据结点的地址，空表时空。

其方便性在于：单链表永远有至少一个结点，无需判断H是否空指针。

其复杂性在于：几乎每个单链表的操作，都是从H->next开始的，我们总是会想一下为什么不从第1个开始呢？



(a) 带头结点的空单链表



(b) 带头结点的非空单链表

带头结点的单链表

对带头结点和不带头结点单链表 求表长

(1) 设L是带头结点的单链表

【算法2-10(a)】 求带头结点的单链表表长

```
int Length_LinkList1 (LinkList L)
{ LNode *p=L;          /*p指向头结点*/
  int j=0;
  while (p->next)
  { p=p->next; j++; }    /*p所指的是第j个结点*/
  return j;
}
```

对比一下有头结点和无头结点情况下的遍历单链表while循环框架，看上去差别不大。

(2) 设L是不带头结点的单链表。

【算法2-10(b)】 求单链表表长

```
int Length_LinkList2 (LinkList L)
{ LNode *p=L;          /*非空表情况下，p所指的是第一个结点*/
  int j=0;
  while (p)
  { j++; p=p->next; }
  return j;
}
```

在**无头节点**的单链线性表L中删除第i个元素，并由e返回其值

无头节点的
单链表

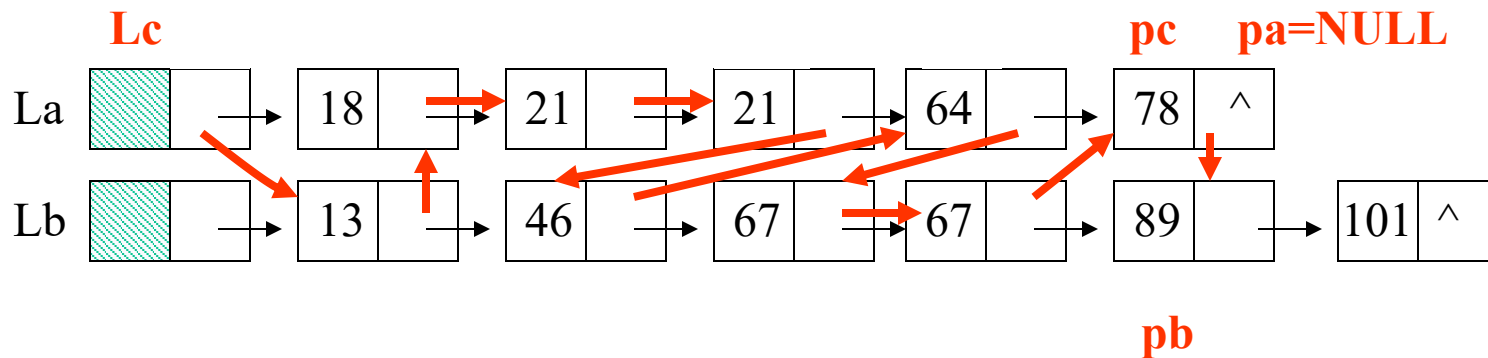
```
Status ListDelete_L(LinkList &pL, int i, ElemType &e)
{
    //L 是不带头结点的单链表;
    LNode *q = NULL;
    if(!pL) return ERROR;        // 空表情况。
    if(i ==1) // 删除第一个节点，需特殊处理。
    {
        q=L; L=L->next;
    }
    else
    { //将p移动到i-1位置，在i-1结点
        Lnode *p = L; int j=1;
        while ( p && j<i-1){
            p=p->next;    j++;
        }
        if( !(p->next) || j>i-1 ) return ERROR;
        q=p->next; p->next= q->next;
    }
    e= q->data;    free(q);
    return OK ;
}// ListDelete_L O(n)
```

对比一下有头结点和无头结点情况下的删除某个位置的节点的实现，
无头结点的单链表，要单独处理特殊情况，即删除第1个位置的结点。

[例2-4] 两个有序单链线性表的合并

```
void MergeList_L(LinkList &La, LinkList &Lb, LinkList &Lc)
{
    pa = La->next;  pb = Lb->next;  Lc = pc = La;
    while ( pa && pb ) {
        if ( pa->data <= pb->data ) {
            pc->next = pa;  pc = pa;  pa = pa->next;
        }
        else {  pc->next = pb;  pc = pb;  pb = pb->next; }
    }
    pc->next = pa ? pa : pb ;
    free(Lb);
} // MergeList_L   O(n+m)   就地合并
```

P31-2.12



静态链表 用数组模拟可利用空间表

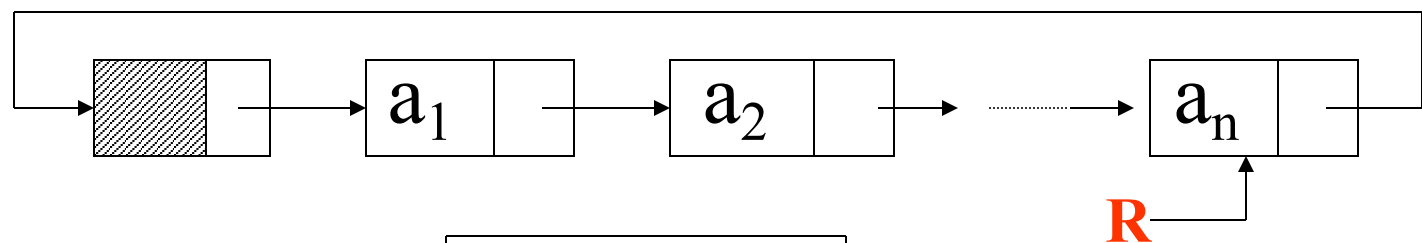
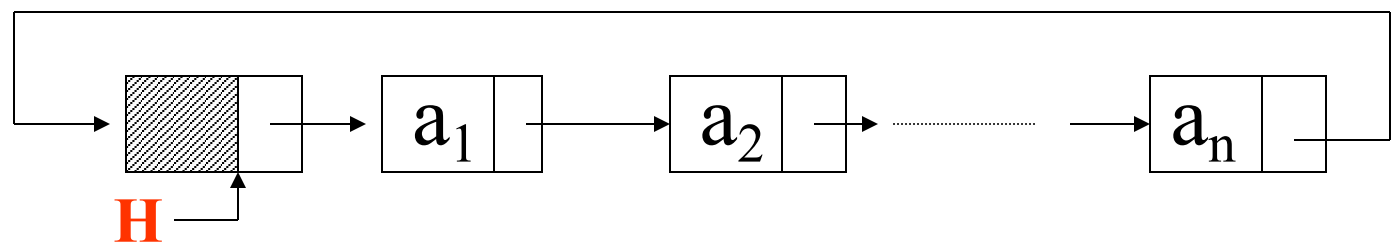
0	1	2	3	4	5	6	7	8	9	
3	8		1			-1		6		cur(游标)
	a ₂		a ₁			a ₄		a ₃		data

↑

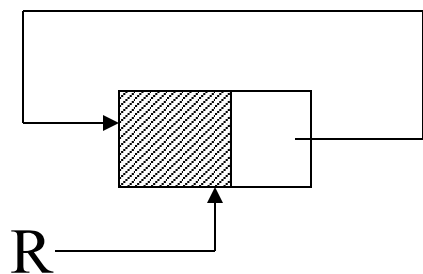
```
# define MAXSIZE 10           //链表的最大长度
typedef struct {
    ElemType data;
    int cur;
}componet, SLinkList[MAXSIZE];
```

第1个元素的下标: SLinkList[0].cur

2.3.2 循环链表

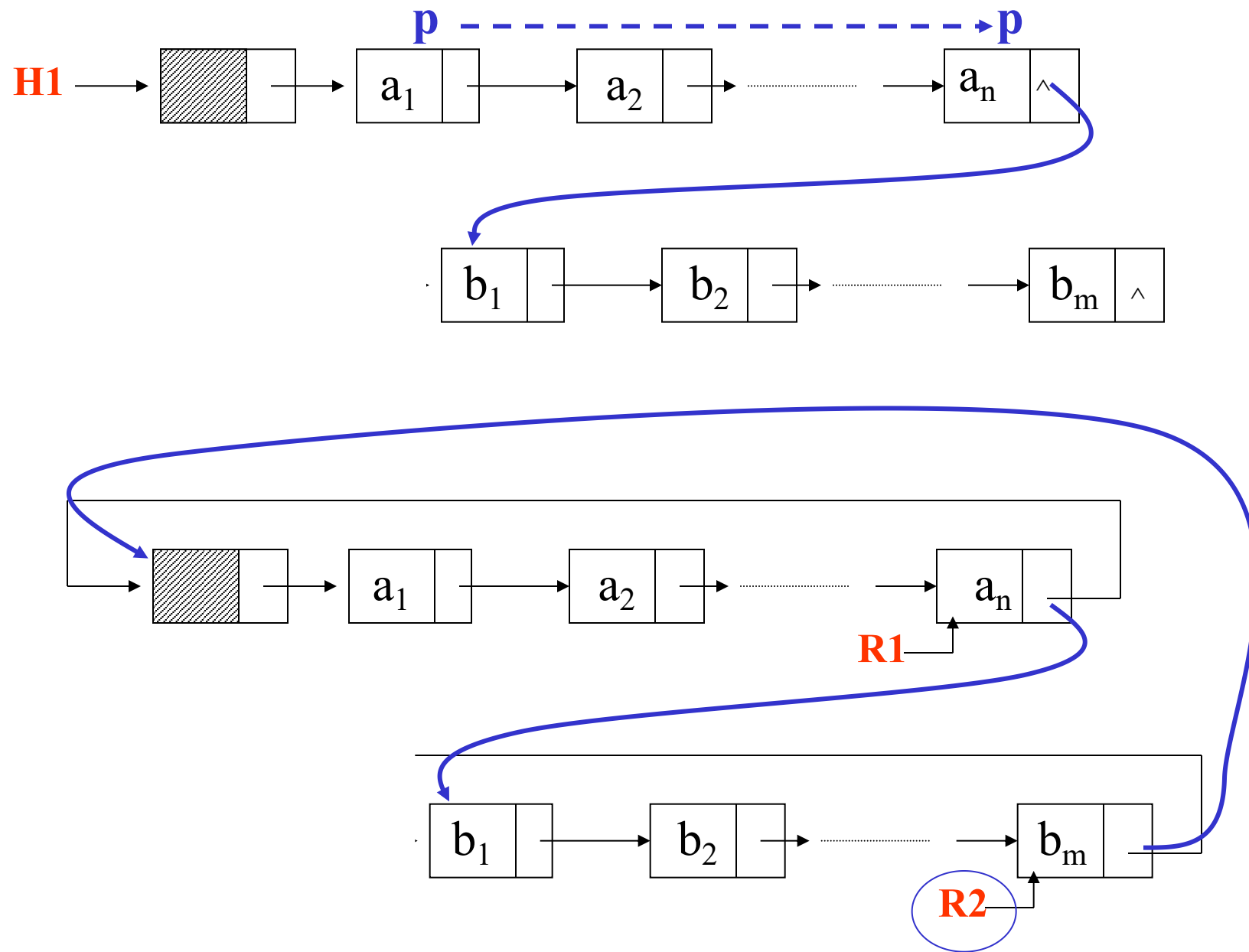


空循环链表



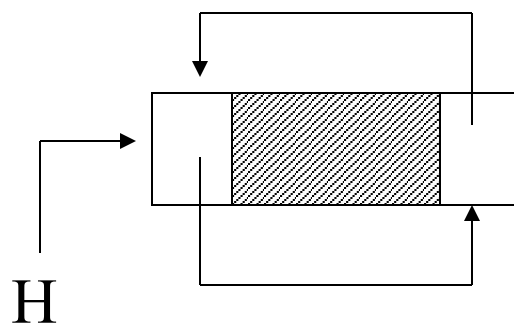
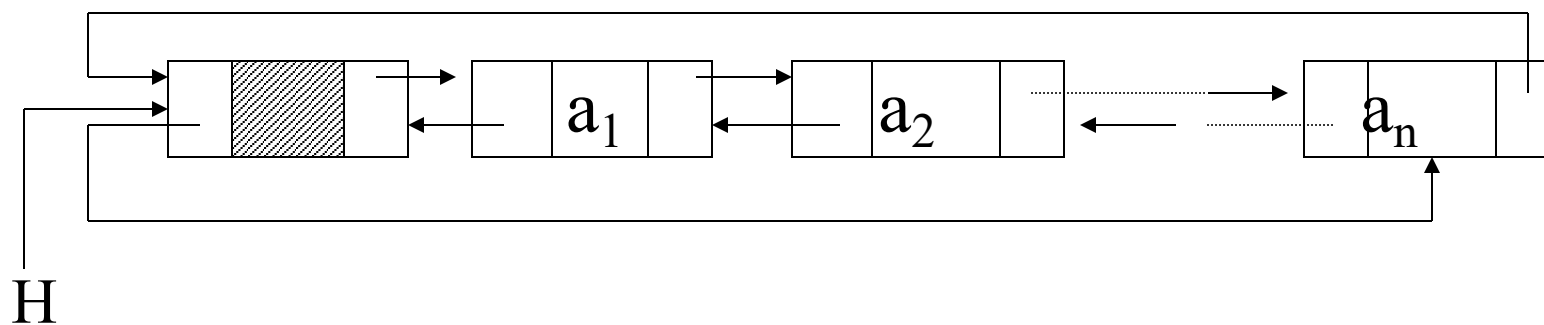
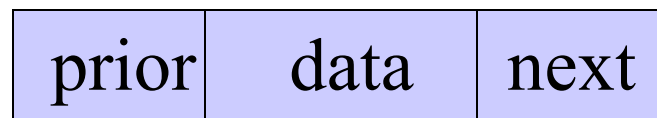
$R \rightarrow \text{next} = R$

循环链表类型定义与单链表相同



2.3.3 双向链表

结点的物理表示



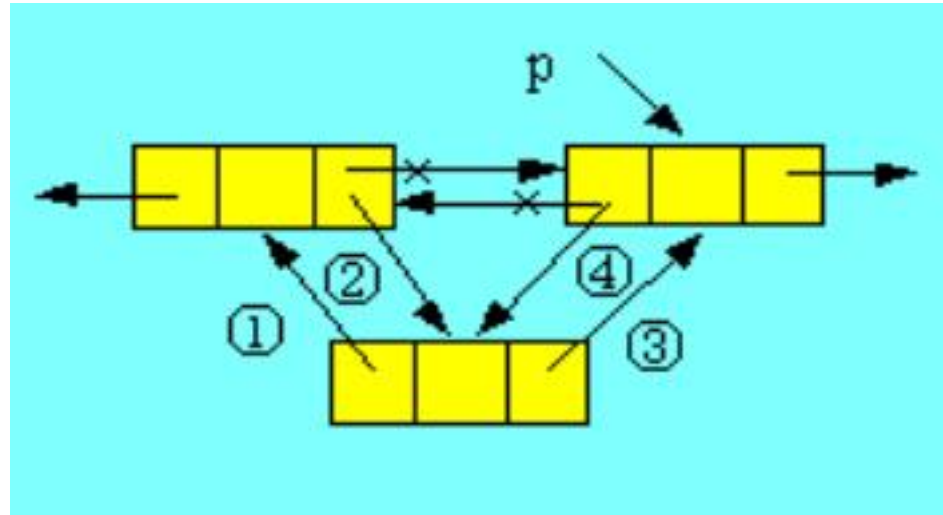
$H \rightarrow \text{next} = H \rightarrow \text{prior} = H$

思考：能否以 $H \rightarrow \text{next} = H \rightarrow \text{prior}$ 判定双向循环链表为空？

在双向链表中插入一个结点：

设p指向双向链表中某结点，s指向待插入的值为x的新结点，将*s插入到*p的前面。操作如下：

- ① $s \rightarrow \text{prior} = p \rightarrow \text{prior};$
- ② $p \rightarrow \text{prior} \rightarrow \text{next} = s;$
- ③ $s \rightarrow \text{next} = p;$
- ④ $p \rightarrow \text{prior} = s;$



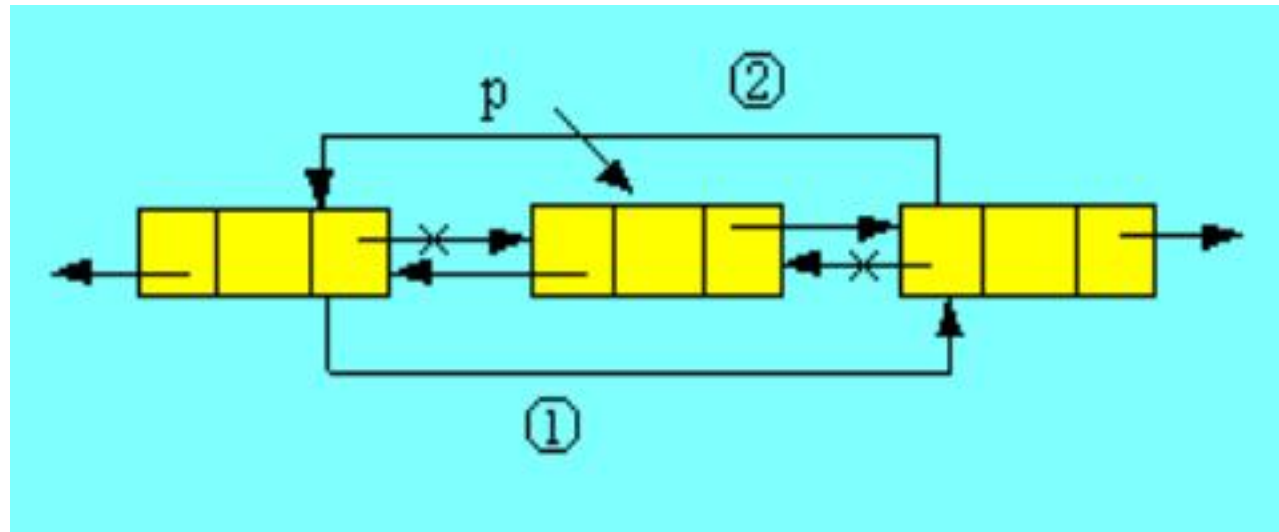
上面指针操作的顺序不是唯一的，但也不是任意的，操作①必须要放到操作④的前面完成，否则*p的前驱结点的指针就丢掉了。

在双向链表中删除指定结点：

设p指向双向链表中某结点，删除*p。

操作如下：

- ① $p \rightarrow \text{prior} \rightarrow \text{next} = p \rightarrow \text{next};$
- ② $p \rightarrow \text{next} \rightarrow \text{prior} = p \rightarrow \text{prior};$
- ③ $\text{free}(p);$



类型定义 表头指针

```
typedef struct DuLNode {  
    ElemType data;  
    struct DuLNode *prior;  
    struct DuLNode *next;  
} DuLNode, *DuLinkList;
```

小结 顺序表和链表的比较

- 空间

- 存储空间分配时难以准确估计
- 存储密度高($d=1$)
(不考虑空闲区)

- 时间

- 是随机存取结构，可以根据序号直接定位
- 插入/删除结点操作时数据元素要移动

- 空间

- 存储空间动态分配，可以按需要使用
- 每一结点附加指针域($d<1$)

- 时间

- 是非随机存取结构，定位需从头指针扫描
- 插入/删除结点操作时通常只要修改指针

顺序表和链表的比较



存储结构 比较项目		顺序表	链表
空间	存储空间	预先分配，会导致空间闲置或溢出现象	动态分配，不会出现存储空间闲置或溢出现象
	存储密度	不用为表示结点间的逻辑关系而增加额外的存储开销，存储密度等于1	需要借助指针来体现元素间的逻辑关系，存储密度小于1
时间	存取元素	随机存取，按位置访问元素的时间复杂度为 $O(1)$	顺序存取，按位置访问元素时间复杂度为 $O(n)$
	插入、删除	平均移动约表中一半元素，时间复杂度为 $O(n)$	不需移动元素，确定插入、删除位置后，时间复杂度为 $O(1)$
适用情况		① 表长变化不大，且能事先确定变化的范围 ② 很少进行插入或删除操作，经常按元素位置序号访问数据元素	① 长度变化较大 ② 频繁进行插入或删除操作

练习

- ✗ 1. 链表的每个结点中都恰好包含 **一个或多个指针**
- ✗ 2. **链表** 结构适宜于进行顺序存取，而 **顺序表** 适宜于进行随机存取。
- ✗ 3. 顺序存储方式的优点是存储密度大，且插入、删除运算 **效率低**。
- ✗ 4. 线性表若采用链式存储时，结点之间 **任意** **内部** 的存储空间都是可以是不连续的。
- ✗ 5. 线性表的每个结点 **可以** 是一个简单类型，**也可以是复杂类型**，而链表的每个结点可以是一个复杂类型

知识点小结

- 线性表的概念
- 线性表的顺序存储
- 顺序存储线性表的操作
- 线性表的链式存储
- 链式存储线性表的操作
- 头结点及其作用
- 单链表
- 循环链表
- 双向链表
- 顺序存储线性表的特点
- 链式存储线性表的特点

借助画图的方式来编写链表程序

使用画图方法建立一个双向链表

- 0、思考一下，结点类型如何定义：

- 先把结点画出来：

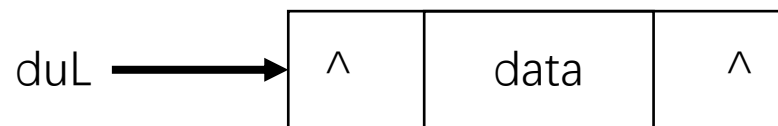
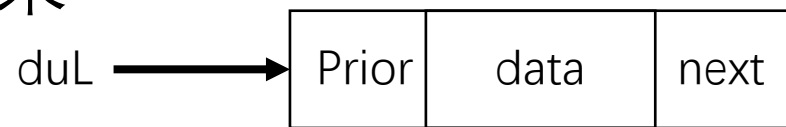


- 参考教材和PPT中的类型声明，试着写一下：

- `typedef struct DuLNode`
- `{`
- `char data;`
- `struct DuLNode *next, *prior;`
- `} DuLNode, *DuLinkList;`

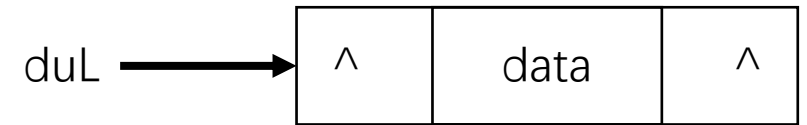
使用画图方法建立一个双向链表

- 1、思考一下，是否要头结点，仿照教材，采用带头结点的双向链表，使用指针类型的变量duL指向头结点，先把duL画出来
- 2、该考虑一下空表情况了，：
- 3、就是要思考一下：duL->next应该是多少？ duL->prior应该是多少？
- 4、当不使用循环链表时，
 - duL->next，看上去是NULL为好；
 - duL->prior，看上去也是NULL为好；
 - 画出带头结点的双向链表的空表。
- 5、空表有了，试着写出空表的判断条件：
 - duL->prior，永远是NULL，没有用。
 - **duL->next == NULL** 这个表达式可以。



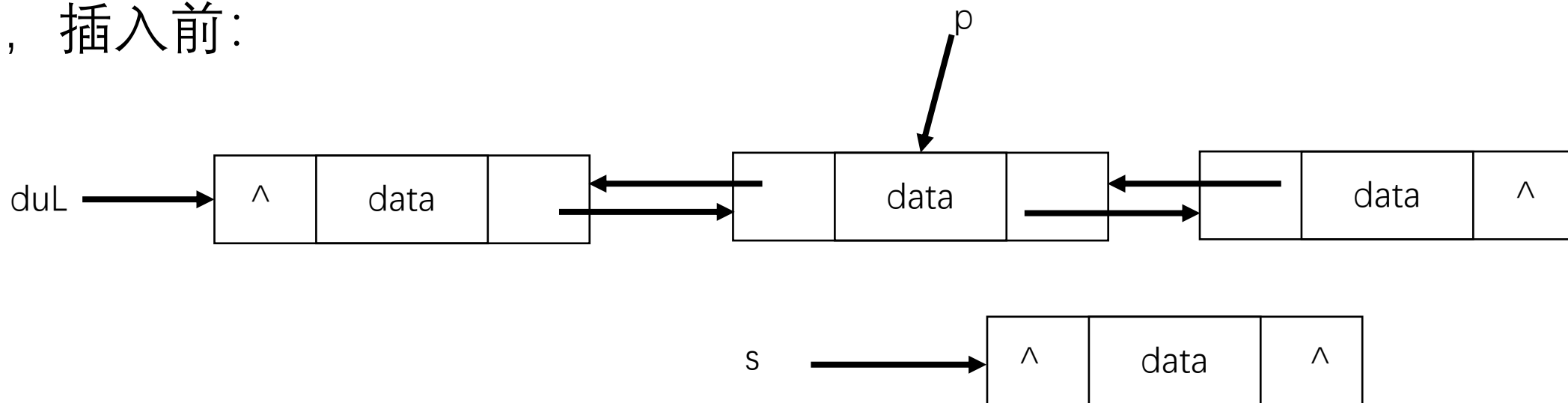
使用画图方法建立一个双向链表

- 6、按画图的结点情况，试着写一下初始化函数：
- `bool InitDuList(DuLinkList &duL)`
- `{`
- `duL = (DuLinkList)malloc(sizeof(DuLNode));`
- `if(!duL) return false;`
- `duL->prior = NULL;`
- `duL->next = NULL;`
- `return true;`
- `}`



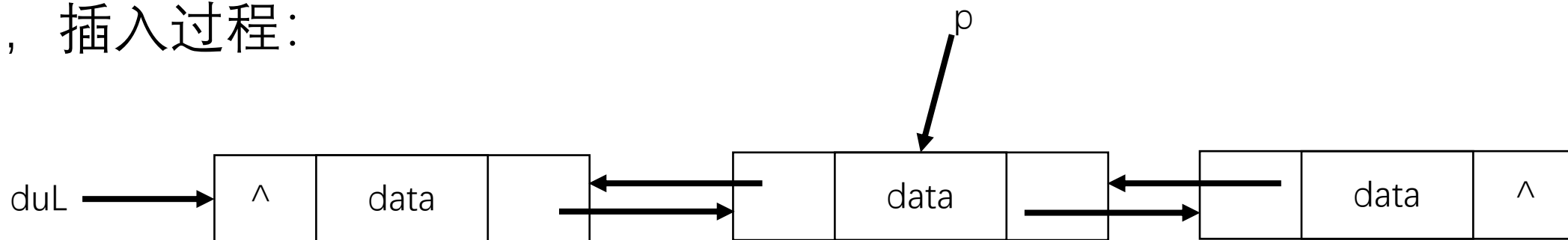
使用画图方法建立一个双向链表

- 7、试着写一下插入函数，先画出来：
- 假设2个结点的线性表，在位置1后插入，准备好工作指针p和新结点指针s，插入前：

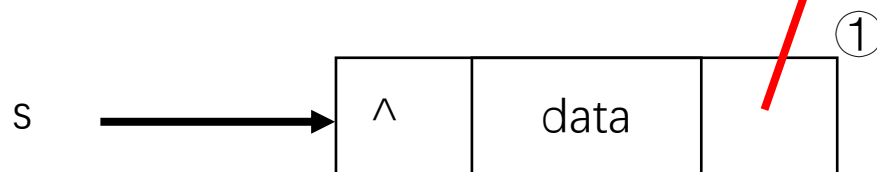


使用画图方法建立一个双向链表

- 7、试着写一下插入函数，先画出来：
- 假设2个结点的线性表，在位置1后插入，准备好工作指针p和新结点指针s，插入过程：



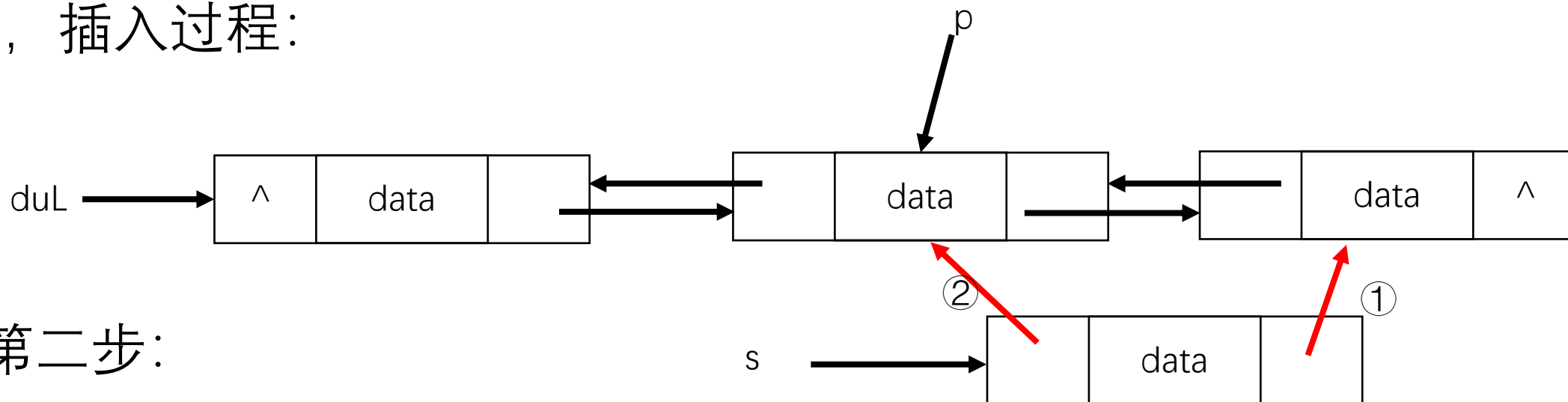
- 第一步：



使用画图方法建立一个双向链表

- 7、试着写一下插入函数，先画出来：

- 假设2个结点的线性表，在位置1后插入，准备好工作指针p和新结点指针s，插入过程：

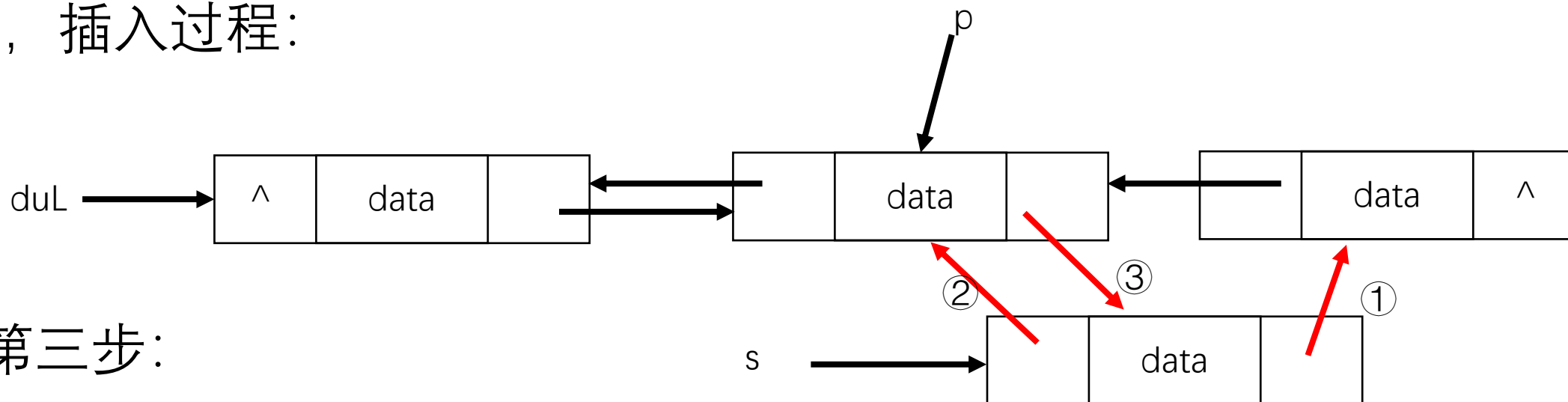


- 第二步：

使用画图方法建立一个双向链表

- 7、试着写一下插入函数，先画出来：

- 假设2个结点的线性表，在位置1后插入，准备好工作指针p和新结点指针s，插入过程：

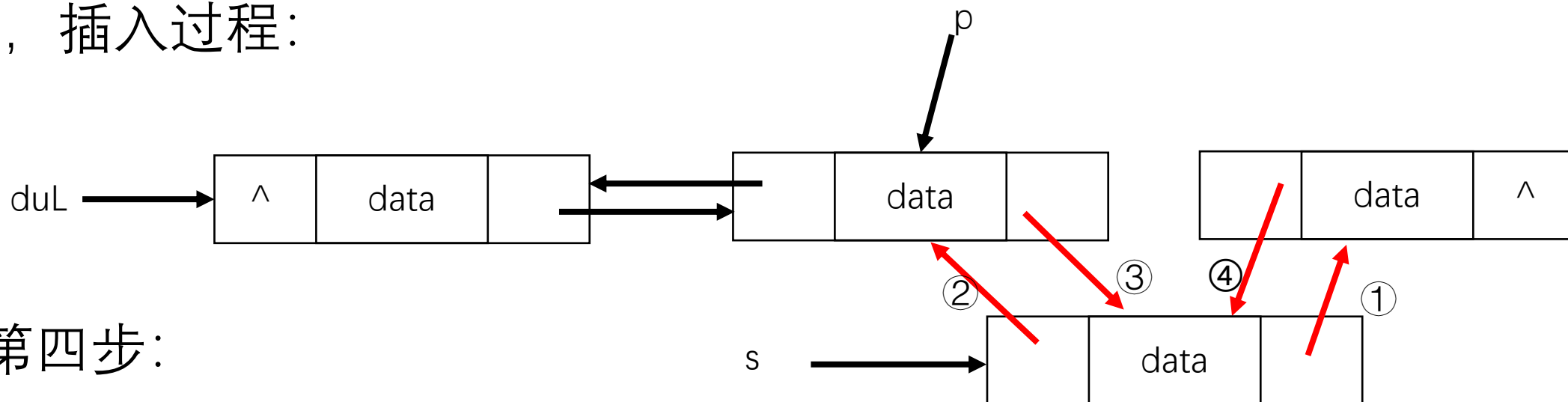


- 第三步：

使用画图方法建立一个双向链表

- 7、试着写一下插入函数，先画出来：

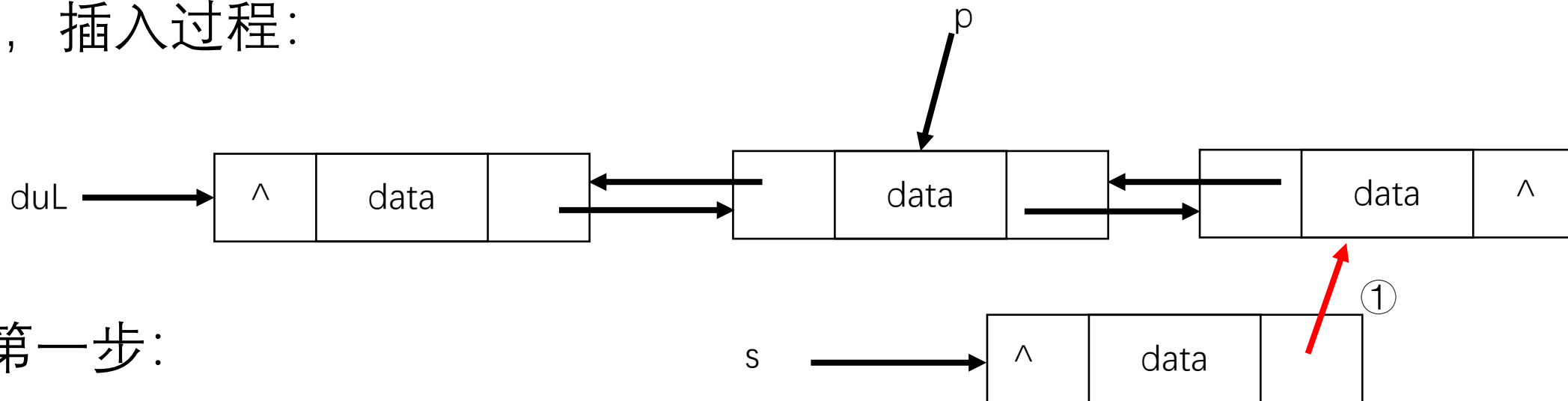
- 假设2个结点的线性表，在位置1后插入，准备好工作指针p和新结点指针s，插入过程：



- 第四步：

使用画图方法建立一个双向链表

- 7、试着写一下插入函数，先画出来：
- 假设2个结点的线性表，在位置1后插入，准备好工作指针p和新结点指针s，插入过程：

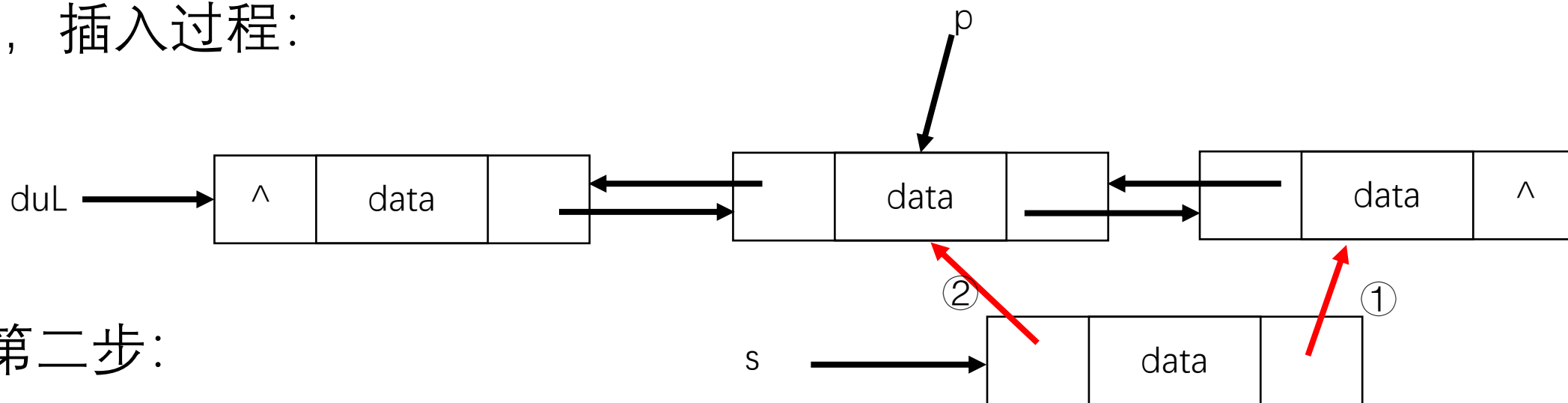


- 第一步：
- `s->next = p->next;`

对照图写代码

使用画图方法建立一个双向链表

- 7、试着写一下插入函数，先画出来：
- 假设2个结点的线性表，在位置1后插入，准备好工作指针p和新结点指针s，插入过程：

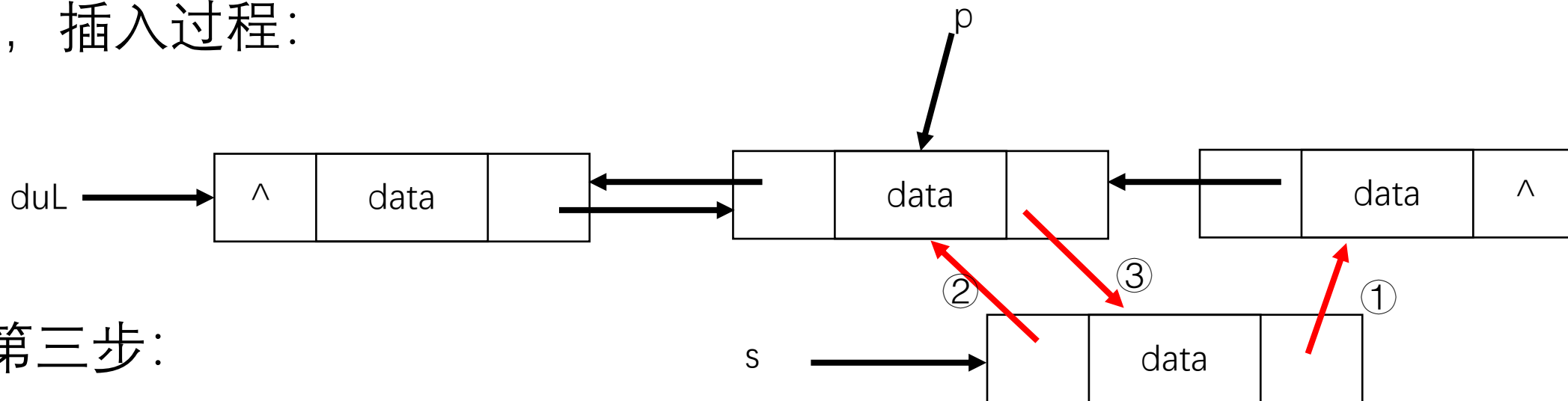


- 第二步：
- `s->next = p->next;`
- `s->prior = p;`

对照图写代码

使用画图方法建立一个双向链表

- 7、试着写一下插入函数，先画出来：
- 假设2个结点的线性表，在位置1后插入，准备好工作指针p和新结点指针s，插入过程：

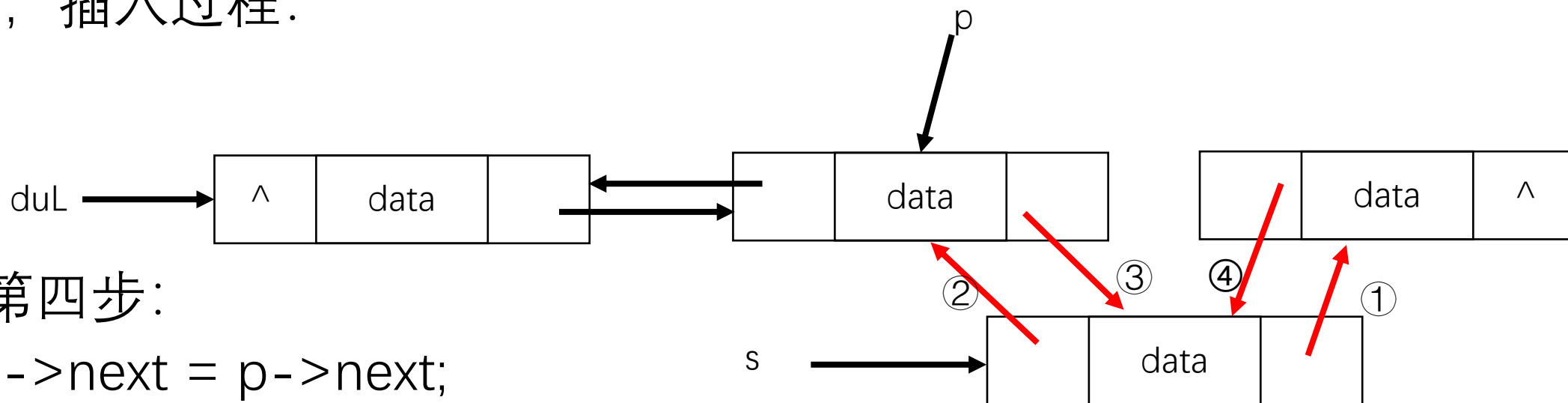


- 第三步：
- `s->next = p->next;`
- `s->prior = p;`
- `p->next = s;`

对照图写代码

使用画图方法建立一个双向链表

- 7、试着写一下插入函数，先画出来：
- 假设2个结点的线性表，在位置1后插入，准备好工作指针p和新结点指针s，插入过程：



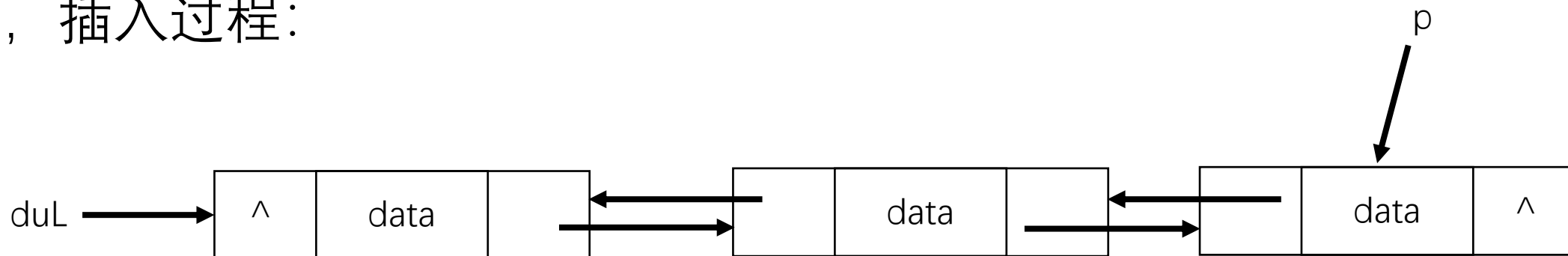
- 第四步：
- `s->next = p->next;`
- `s->prior = p;`
- `p->next = s;`
- `s->next->prior = s;`

对照图写代码

使用画图方法建立一个双向链表

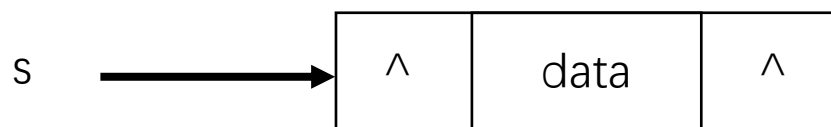
- 8、试试最后p是一个结点时插入代码是否正确： 对照代码画图

- 假设2个结点的线性表，在位置2后插入，准备好工作指针p和新结点指针s，插入过程：



- 对照代码修改图：

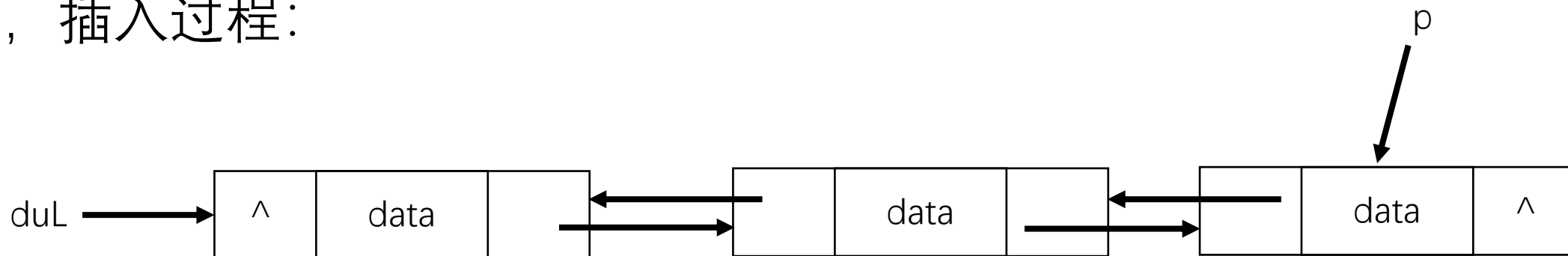
- $s \rightarrow \text{next} = p \rightarrow \text{next};$
- $s \rightarrow \text{prior} = p;$
- $p \rightarrow \text{next} = s;$
- $s \rightarrow \text{next} \rightarrow \text{prior} = s;$



使用画图方法建立一个双向链表

- 8、试试最后p是一个结点时插入代码是否正确： 对照代码画图

- 假设2个结点的线性表，在位置2后插入，准备好工作指针p和新结点指针s，插入过程：



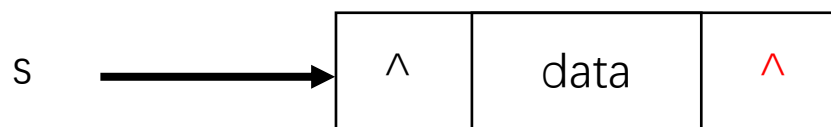
- 对照代码修改图：

- $s \rightarrow next = p \rightarrow next;$

- $s \rightarrow prior = p;$

- $p \rightarrow next = s;$

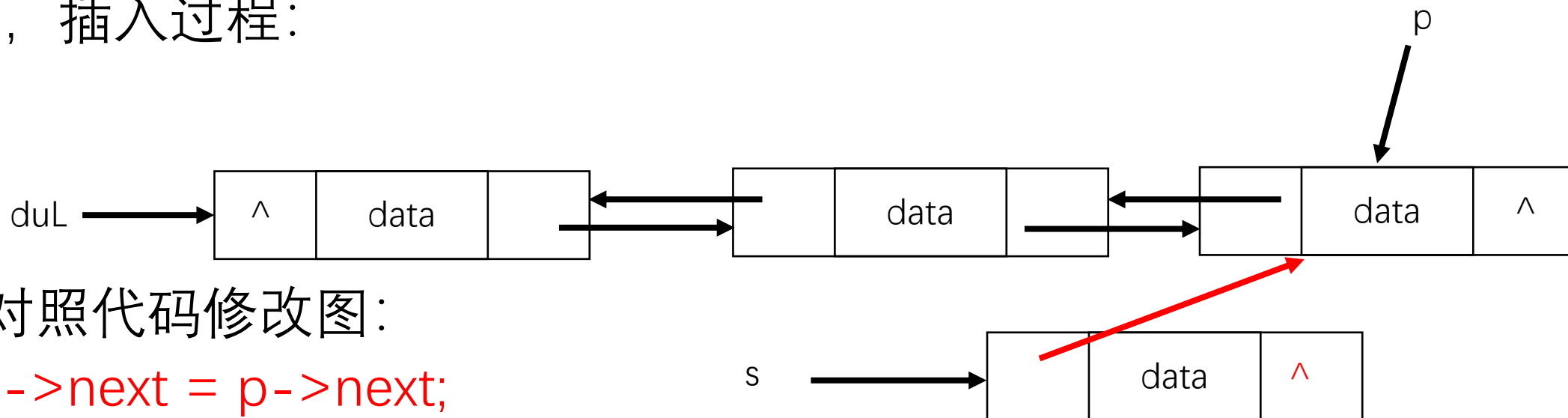
- $s \rightarrow next \rightarrow prior = s;$



使用画图方法建立一个双向链表

- 8、试试最后p是一个结点时插入代码是否正确： 对照代码画图

- 假设2个结点的线性表，在位置2后插入，准备好工作指针p和新结点指针s，插入过程：



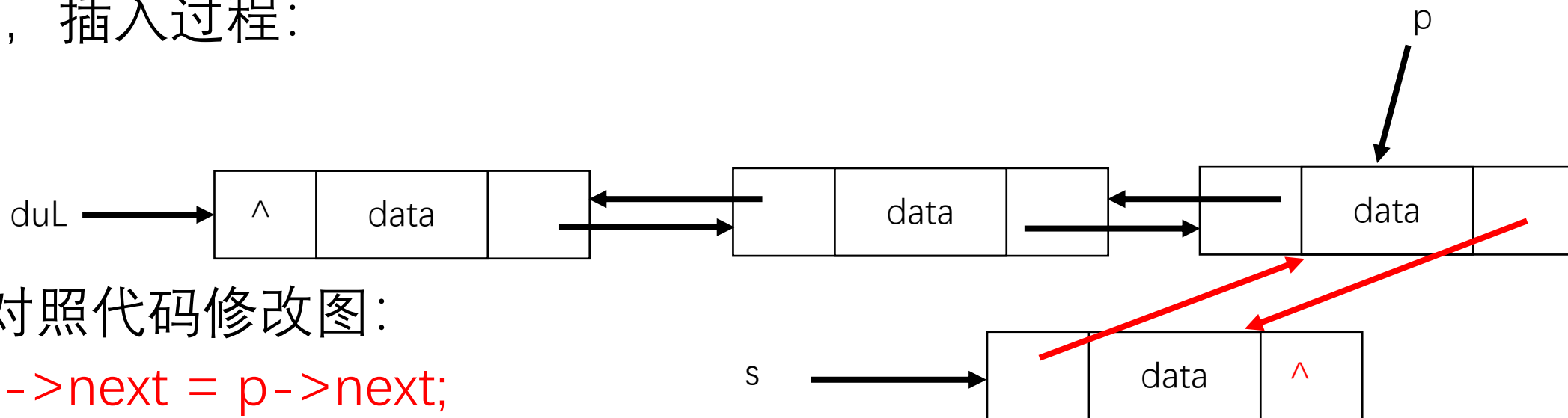
- 对照代码修改图：

- $s \rightarrow next = p \rightarrow next;$
- $s \rightarrow prior = p;$
- $p \rightarrow next = s;$
- $s \rightarrow next \rightarrow prior = s;$

使用画图方法建立一个双向链表

- 8、试试最后p是一个结点时插入代码是否正确： 对照代码画图

- 假设2个结点的线性表，在位置2后插入，准备好工作指针p和新结点指针s，插入过程：



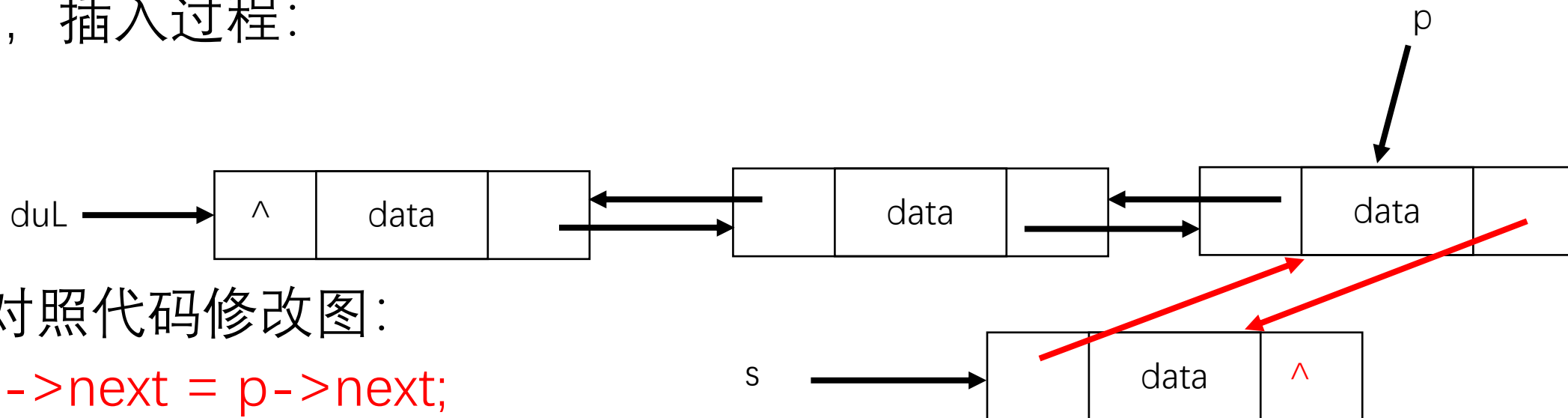
- 对照代码修改图：

- $s \rightarrow next = p \rightarrow next;$
- $s \rightarrow prior = p;$
- $p \rightarrow next = s;$
- $s \rightarrow next \rightarrow prior = s;$

使用画图方法建立一个双向链表

- 8、试试最后p是一个结点时插入代码是否正确：对照代码画图

- 假设2个结点的线性表，在位置2后插入，准备好工作指针p和新结点指针s，插入过程：



- 对照代码修改图：

- `s->next = p->next;`
- `s->prior = p;`
- `p->next = s;`
- `s->next->prior = s;`

出错啦!!!

S->next是NULL

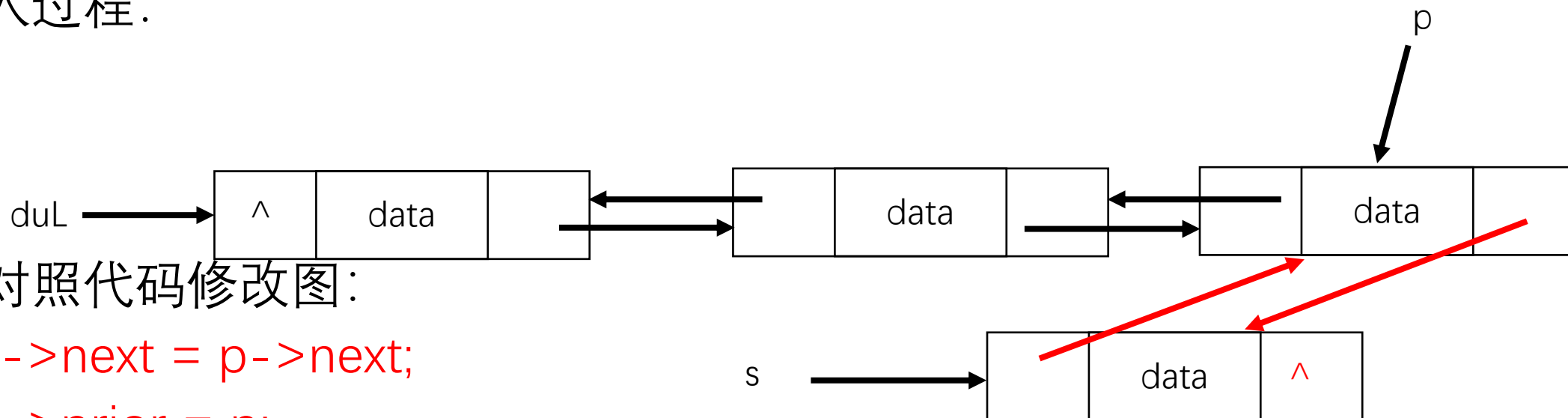
S->next->prior这样会产生运行时的空值错!!

使用画图方法建立一个双向链表

- 8、试试最后p是一个结点时插入代码是否正确：

对照代码画图

- 假设2个结点的线性表，在位置2后插入，准备好工作指针p和新结点指针s，插入过程：



- 对照代码修改图：

- `s->next = p->next;`
- `s->prior = p;`
- `p->next = s;`
- `if(s->next)`
- `s->next->prior = s;`

出错啦!!!

S->next是NULL

S->next->prior这样会产生运行时的空值错!!!

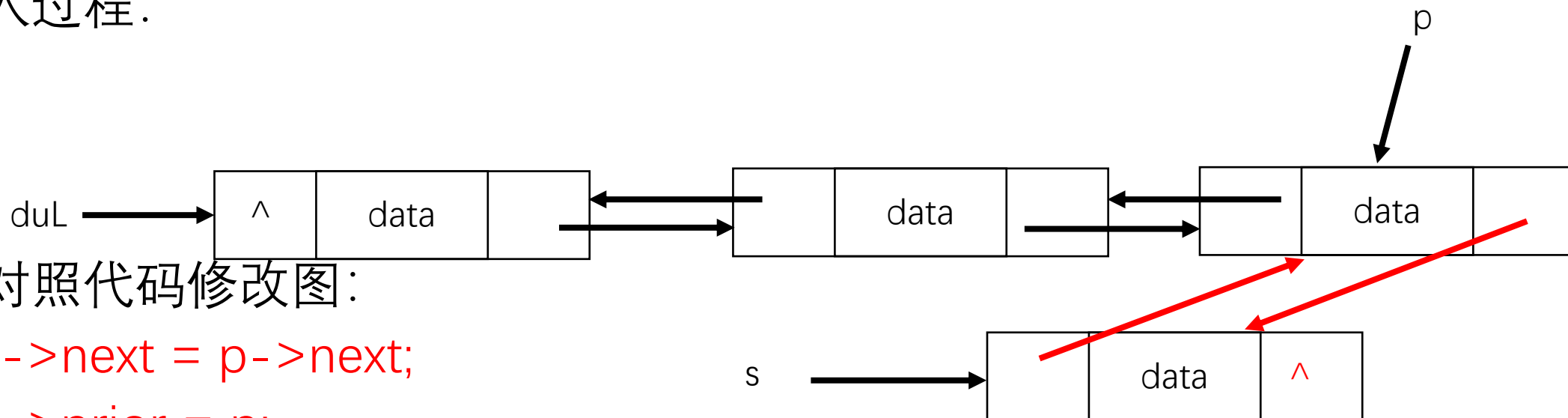
修改代码，判断一下s->next是否为空后再修改

使用画图方法建立一个双向链表

- 8、试试最后p是一个结点时插入代码是否正确：

对照代码画图

- 假设2个结点的线性表，在位置2后插入，准备好工作指针p和新结点指针s，插入过程：



- 对照代码修改图：

- `s->next = p->next;`
- `s->prior = p;`
- `p->next = s;`
- `if(s->next)`
- `s->next->prior = s;`

出错啦!!!

S->next是NULL

S->next->prior这样会产生运行时的空值错!!!

修改代码，判断一下s->next是否为空后再修改

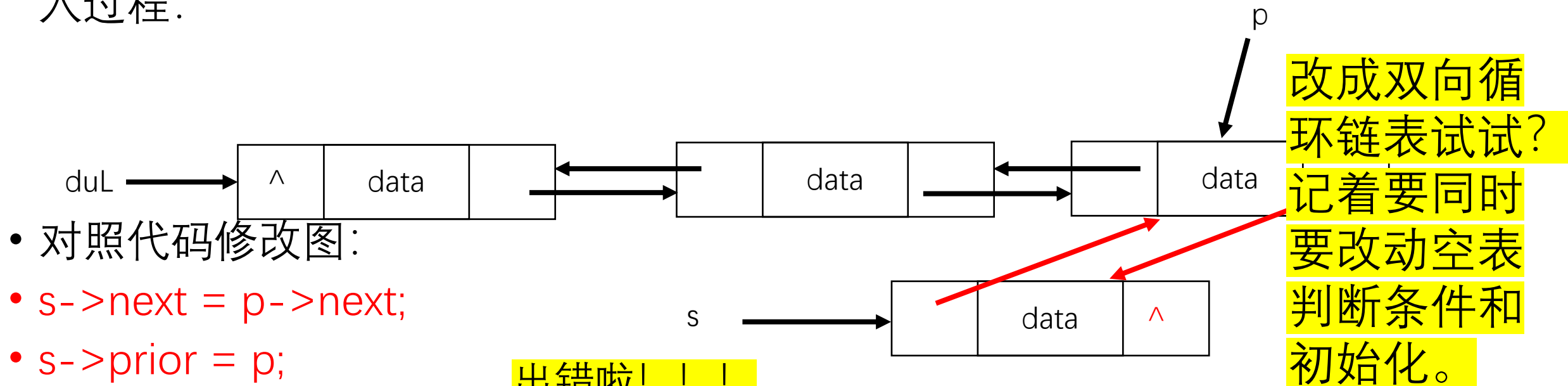
回过头来再想想看，这个错误能否通过调整修改指针的顺序避免掉？能否不用最后这个if判断？

使用画图方法建立一个双向链表

- 8、试试最后p是一个结点时插入代码是否正确：

对照代码画图

- 假设2个结点的线性表，在位置2后插入，准备好工作指针p和新结点指针s，插入过程：



- 对照代码修改图：

- `s->next = p->next;`
- `s->prior = p;`
- `p->next = s;`
- `if(s->next)`
- `s->next->prior = s;`

出错啦！！！！

S->next是NULL

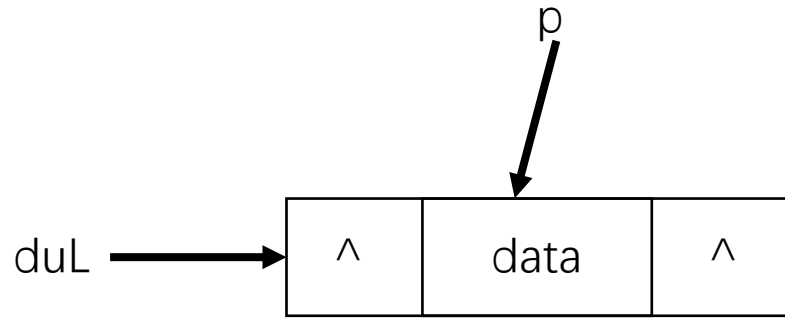
S->next->prior这样会产生运行时的空值错！！！！

修改代码，判断一下s->next是否为空后再修改

回过头来再想想看，这个错误能否通过调整修改指针的顺序避免掉？能否不用最后这个if判断？

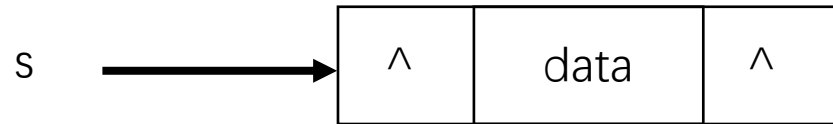
使用画图方法建立一个双向链表

- 9、试试空表情况：
- 假设空表时，插入s指向的结点过程：



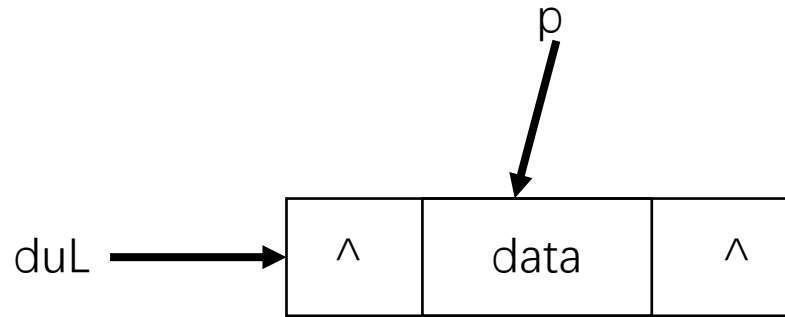
- 对照代码修改图：

- `s->next = p->next;`
- `s->prior = p;`
- `p->next = s;`
- `if(s->next)`
- `s->next->prior = s;`



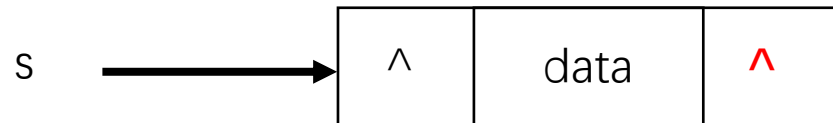
使用画图方法建立一个双向链表

- 9、试试空表情况：
- 假设空表时，插入s指向的结点过程：



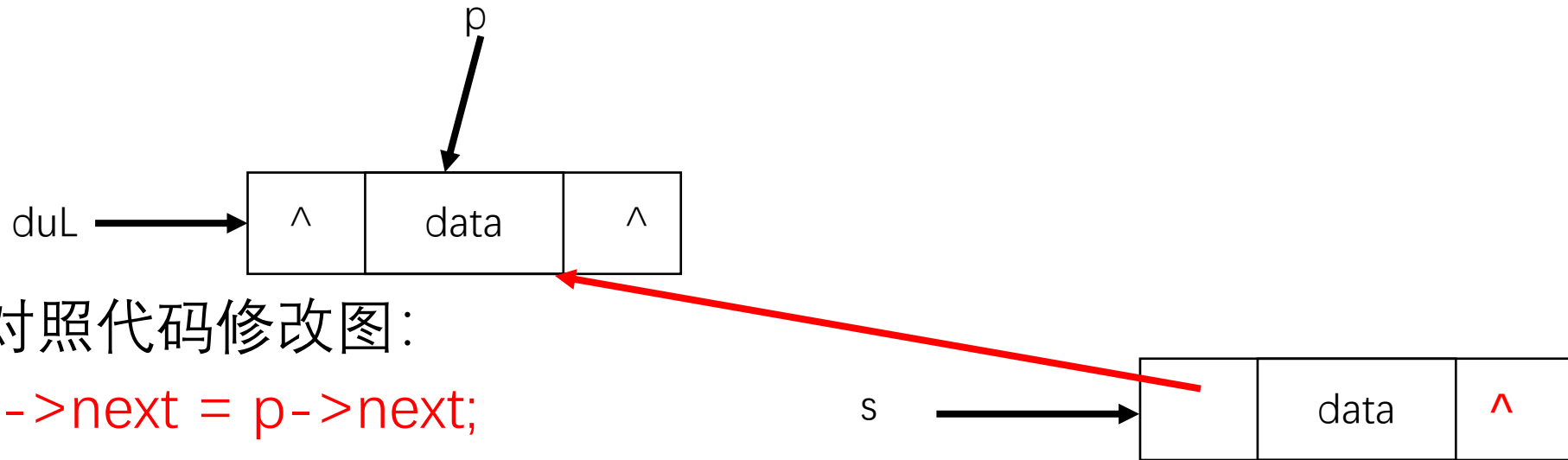
- 对照代码修改图：

- $s \rightarrow next = p \rightarrow next;$
- $s \rightarrow prior = p;$
- $p \rightarrow next = s;$
- `if(s->next)`
- $s \rightarrow next \rightarrow prior = s;$



使用画图方法建立一个双向链表

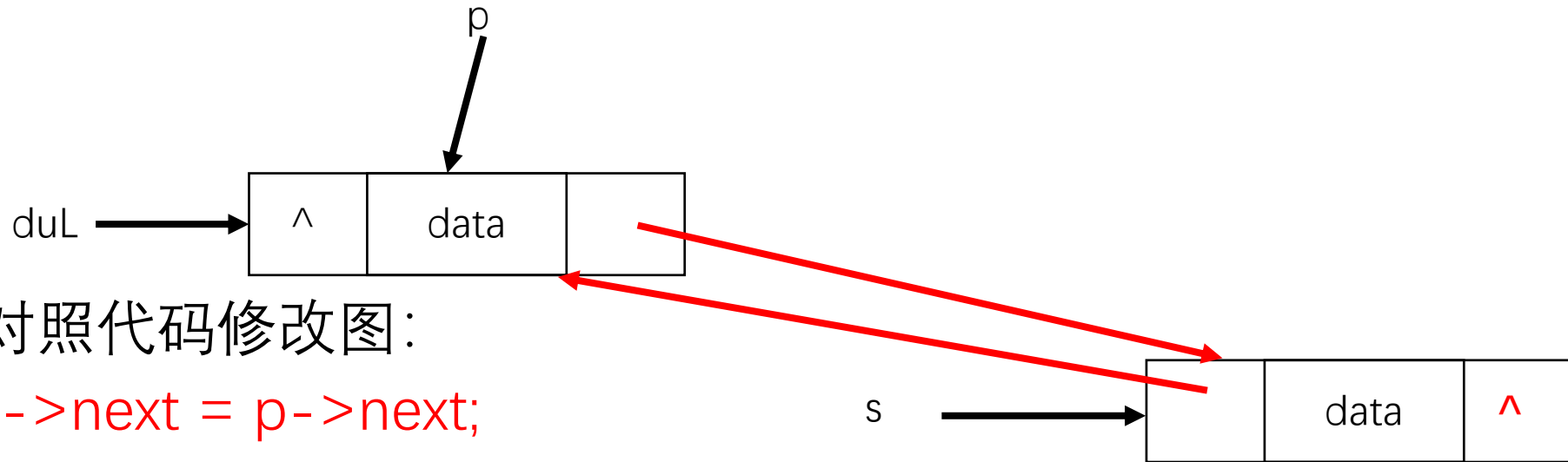
- 9、试试空表情况：
- 假设空表时，插入s指向的结点过程：



- 对照代码修改图：
- $s \rightarrow next = p \rightarrow next;$
- $s \rightarrow prior = p;$
- $p \rightarrow next = s;$
- `if(s->next)`
- `s->next->prior = s;`

使用画图方法建立一个双向链表

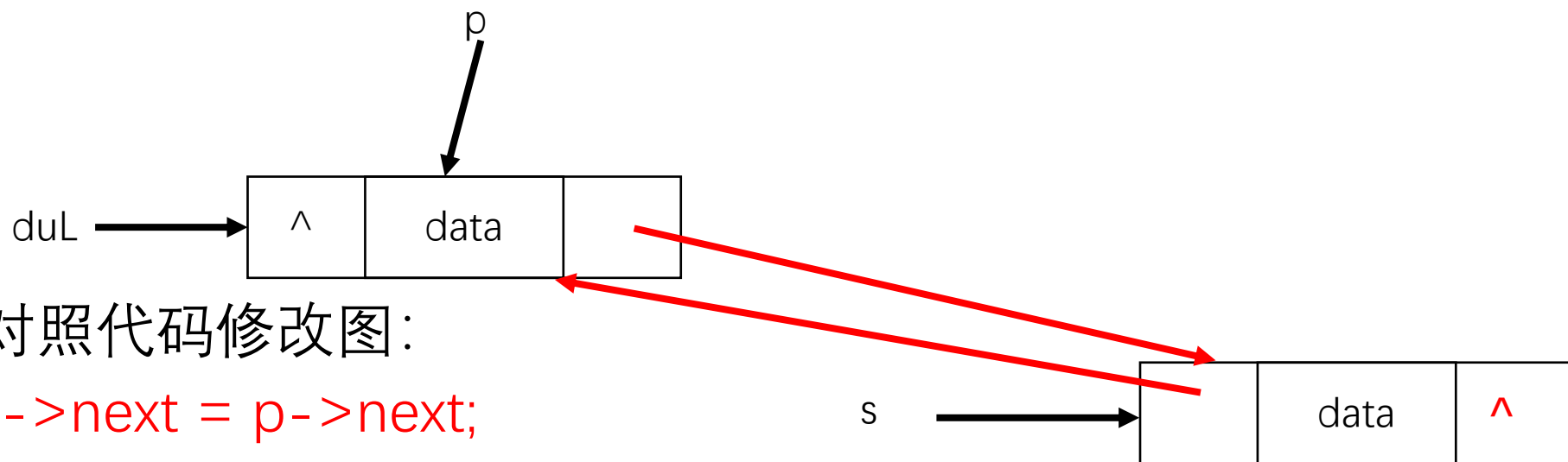
- 9、试试空表情况：
- 假设空表时，插入s指向的结点过程：



- 对照代码修改图：
- $s \rightarrow next = p \rightarrow next;$
- $s \rightarrow prior = p;$
- $p \rightarrow next = s;$
- `if(s->next)`
- `s->next->prior = s;`

使用画图方法建立一个双向链表

- 9、试试空表情况：
- 假设空表时，插入s指向的结点过程：



- 对照代码修改图：

- `s->next = p->next;`
- `s->prior = p;`
- `p->next = s;`
- `if(s->next)`
- `s->next->prior = s;`

成功了!!!

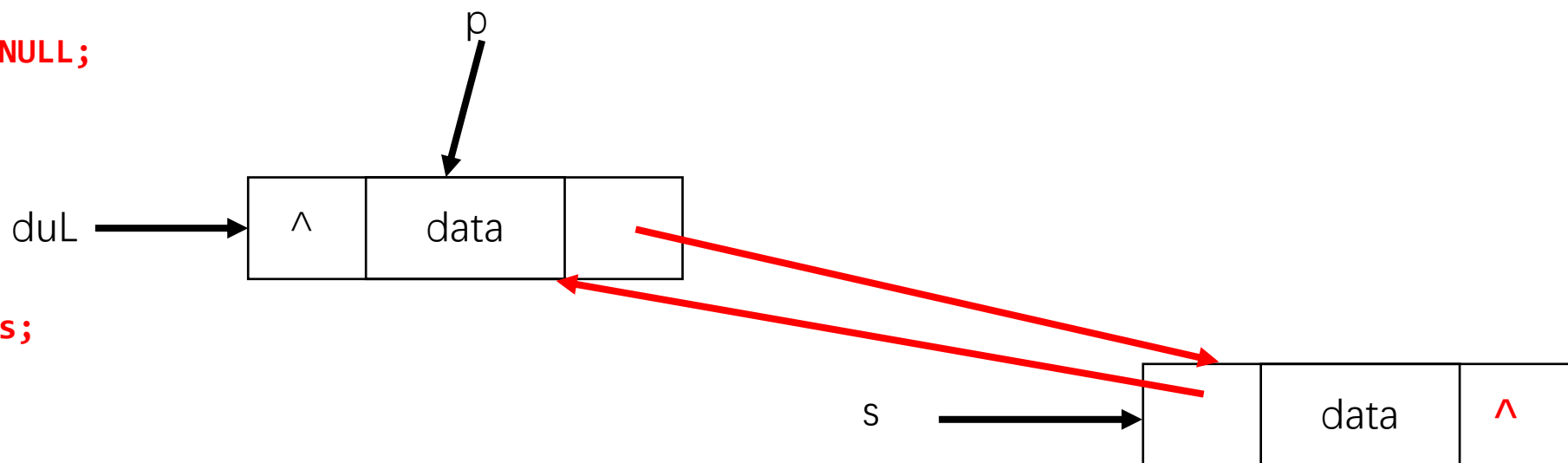
使用画图方法建立一个双向链表

- 10、把插入函数代码补充完成，参考ppt和教材中的单链表的代码：

```
bool DuListInsert(DuLinkList &duL, int i, char e)
{
    if(!duL) return false;
    DuLNode *p=duL;    int  j=0; // 考虑到有空表的可能性，从0开始
    //将p从头节点移动到i-1结点，在i-1结点后插结点
    while (p && j<i-1)    {
        p=p->next;    j++; // 每移动1个节点，j++，直到j==i-1
    }
    if ( !p || j>i-1 ) return false ;
    DuLNode * s = (DuLNode *)malloc(sizeof(DuLNode));
    if(!s) return false;
    s->data= e ;
    s->next = s->prior = NULL;

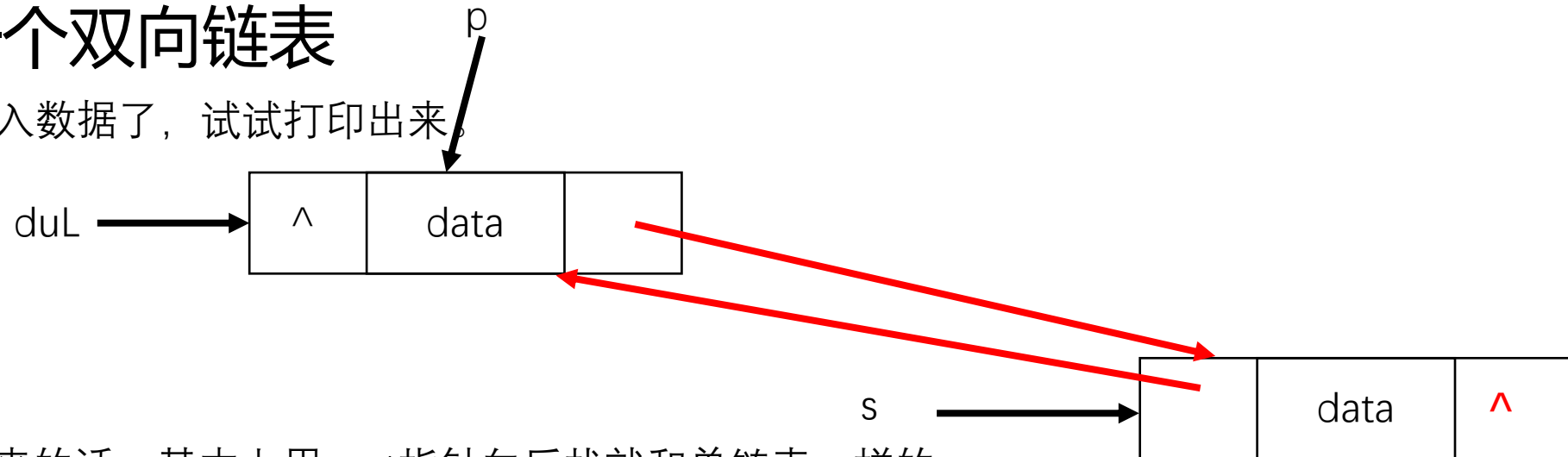
    s->next = p->next;
    s->prior = p;
    p->next = s;
    if(s->next)
        s->next->prior = s;

    return true ;
}//
```



使用画图方法建立一个双向链表

- 11、有点意思了，似乎能插入数据了，试试打印出来。



- 按照这个图，要打印结点出来的话，基本上用next指针向后找就和单链表一样的

```
bool PrintList(DuLinkList duL)
//打印双向链表的每个结点
{
    if(!duL) return false;
    DuLNode *p= duL->next;
    while ( p ) {
        printf("%c\n", p->data);
        p= p->next;
    }
    return true;
}
```

使用画图方法建立一个双向链表

- 12、写个main函数试试。

```
01: int main()
02: {
03:     DuLinkedList duL=NULL;
04:     if(!InitDuList(duL))
05:         return -1;
06:
07:     char ch;
08:     printf("请输入一个字符: ");
09:     scanf("%c", &ch);
10:     getchar();
11:
12:     if( DuListInsert(duL, 1, ch) != true)
13:         return -2;
14:     if(PrintList(duL))
15:         return 1;
16:     else
17:         return -3;
18: }
```

第03行程序后:

duL → ^

第07行程序前:



第14行程序前, 自己画一画:

第18行程序前, 自己画一画:

打印出来了1个结点, 成功了!!!

嗯, 还有线性表的销毁没有做, malloc的内存没有释放。

可以按照打印的函数写一个释放函数, 参考单链表的销毁函数

使用画图方法建立一个双向链表

- 13、写个销毁函数试试，拿单链表的改，似乎也不需要大改，改改变量类型就行了。

```
void DestroyDuList (DuLinkList &duL)
```

```
//销毁已存在线性单链表L
```

```
{
```

```
    if(!duL) return;
```

```
    DuLNode *p= duL;
```

```
    while (p) {
```

```
        DuLNode * q= p->next;
```

```
        free(p);
```

```
        p=q;
```

```
    }
```

```
    duL=NULL;
```

```
} // DestroyDuList    O(n)
```

这个函数的执行情况，自己画一画：

使用画图方法建立一个双向链表

- 14、修改一下main函数将DestroyDuList加进去试试。

```
int main()
{
    DuLNode *duL=NULL;
    if(!InitDuList(duL))
        return -1;

    char ch;
    printf("请输入一个字符: ");
    scanf("%c", &ch);
    getchar();

    int ret = 1;
    if( DuListInsert(duL, 1, ch) != true){
        ret = -2;
    }
    else if(!PrintList(duL))
        ret = -3;
    DestroyDuList(duL);
    return ret;
}
```

main函数的执行过程中duL的每行之前的情况，自己画一画：

打印出来了1个结点，
Main函数返回值是1，成功了！！
改改main函数多加几个结点试试

使用画图方法建立一个双向链表

- 15、修改一下main函数试试在表头多插入几个结点。

```
int main()
{
    DuLNode *duL=NULL;
    if(!InitDuList(duL))
        return -1;

    int ret = 1;
    for(int i=0; i<5;i++){
        char ch;
        printf("请输入一个字符: ");
        scanf("%c", &ch);
        getchar();
        if( DuListInsert(duL, 1, ch) != true){
            ret = -2;
            break;
        }
    }
    if(!PrintList(duL))
        ret = -3;
    DestroyDuList(duL);
    return ret;
}
```

main函数的执行过程中duL的每行之前的情况，自己画一画：

打印出来了5个结点，

Main函数返回值是1，成功了！！！！

这时都是头插入，改改main函数试试尾插入。

使用画图方法建立一个双向链表

- 16、修改一下main函数在不同位置加入结点试试。

```
int main()
{
    DuLNode *duL=NULL;
    if(!InitDuList(duL))
        return -1;

    int ret = 1;
    for(int i=0; i<5;i++){
        char ch;
        printf("请输入一个字符: ");
        scanf("%c", &ch);
        getchar();
        if( DuListInsert(duL, i+1, ch) != true){
            ret = -2;
            break;
        }
    }
    if(!PrintList(duL))
        ret = -3;
    DestroyDuList(duL);
    return ret;
}
```

main函数的执行过程中duL的每行之前的情况，自己画一画：

打印出来了5个结点，

尾插入也成功了！！！！

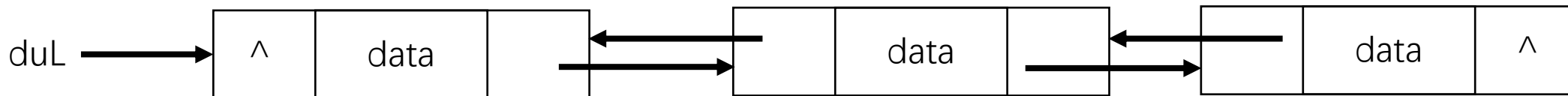
Main函数返回值是1，成功了！！！！

还有一个小疑惑，print和destroy都是用的next的链，prior的链对不对呢

修改printList函数，从前向后打印一遍，再从后向前打印一遍

使用画图方法建立一个双向链表

- 17、修改一下printList函数，从前向后打印一遍，再从后向前打印一遍。



- ```
bool PrintList(DuLinkList duL)
```

//打印双向链表的每个结点

```
{
 if(!duL) return false;

 DuLNode *p= duL;
 printf("从前向后输出:\n");
 while (p->next) {
 printf("%c\n", p->next->data);
 p= p->next;
 }
```

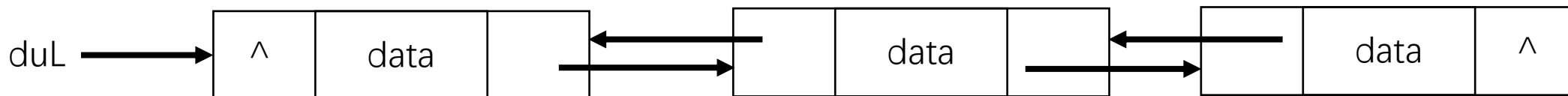
```
 printf("从后向前输出:\n");
 while (p->prior) {
 printf("%c\n", p->data);
 p= p->prior;
 }
 return true;
}
```

为什么这样就能打印出来？

试试把p画在链表上，画画看看。

# 使用画图方法建立一个双向链表

- 18、试着写一个删除第*i*个位置结点的DuListDelete函数，也是先画出来，再按画的步骤写代码：



- `bool DuListInsert(DuLinkList &duL, int i, char &e )`

//删除双向链表中第*i*个位置的结点，并将其值用*e*返回；删除成功函数返回true，删除失败函数返回false

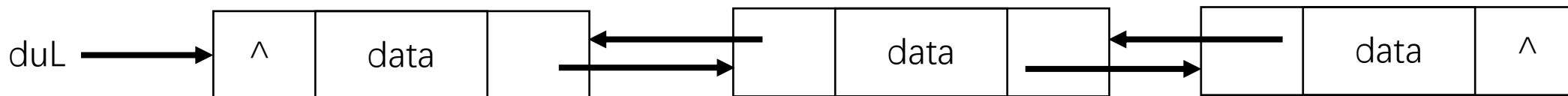
{

画画看看。  
写写代码  
调试调试

}

# 使用画图方法建立一个双向链表

- 19、试着写一个头插入的DuListHeadInsert函数，也是先画出来，再按画的步骤写代码：



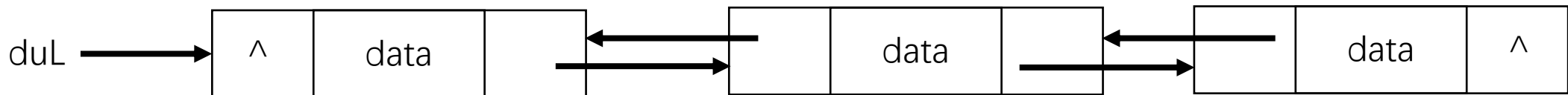
- ```
bool DuListHeadInsert(DuLinkList &duL, char e )  
//将e插入到双向链表中第1个位置；插入成功函数返回true，插入失败函数返回false  
{
```

画画看看。
写写代码
调试调试

}

使用画图方法建立一个双向链表

- 20、试着写一个头删除的DuListHeadDelete函数，也是先画出来，再按画的步骤写代码：



- `bool DuListHeadDelete(DuLinkList &duL, char &e )`

//将双向链表中第1个位置的结点删除，将被删除结点值用e返回；删除成功函数返回true，删除失败函数返回false

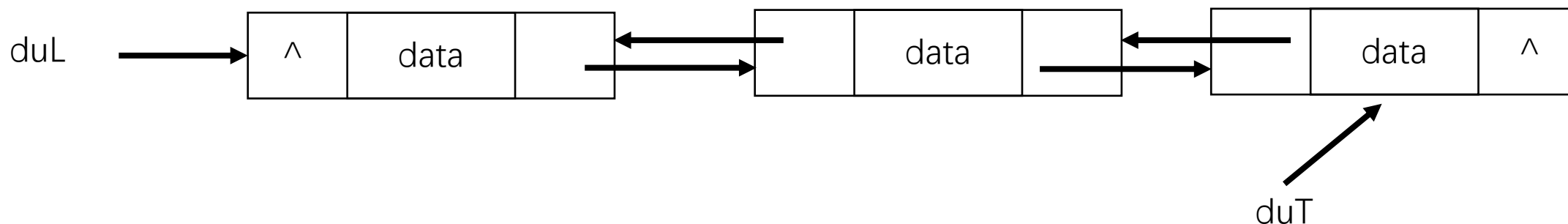
{

画画看看。
写写代码
调试调试

}

使用画图方法建立一个双向链表

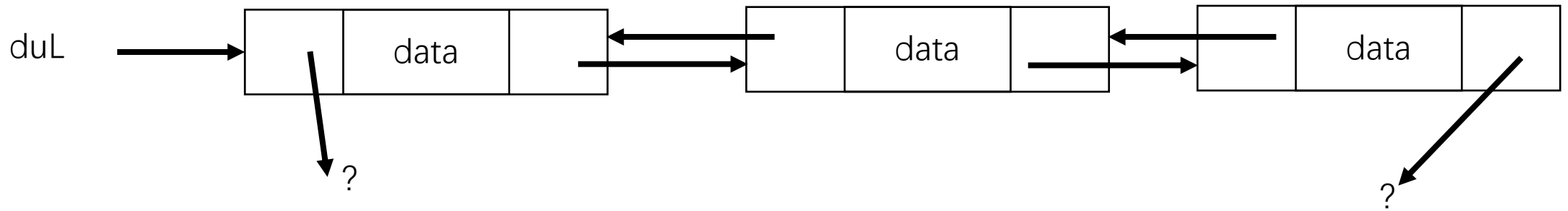
- 21、在main函数中，除了duL之外，再声明一个指针，duT，指向双向链表的尾结点，试着写写尾插入和尾删除函数，先画画，再按画的步骤写代码：



画画看看。
写写代码
调试调试

使用画图方法建立一个双向链表

- 23、试着将duT去掉，让双向链表循环起来：
- 给下图中的两个“？”找找答案，修改前面的所有函数，建立一个循环的双向链表，先画画，再按画的步骤写代码：



画画看看。
写写代码
调试调试

线性表小结

知识点小结

- 线性表的概念
- 线性表的顺序存储
- 顺序存储线性表的操作
- 线性表的链式存储
- 链式存储线性表的操作
- 头结点及其作用
- 单链表
- 循环链表
- 双向链表
- 顺序存储线性表的特点
- 链式存储线性表的特点

线性表的一种链式存储实现方案（第8页片子的的方案4）： 有头尾指针的单链表

```
typedef struct LNode    //结点的数据类型定义
{
    ElemType data;    //结点的数据域
    struct LNode * next; //结点的指针域
} LNode , *Link, *Position;

typedef struct LinkList    //单链表的数据类型定义
{
    LNode * head;
    LNode * tail;
    unsigned int uLength;
}LinkList;
```

- ➡ 特点：
- ➡ 1、单独为链表定义了一个数据类型，包含了链表的长度。（和顺序存储的线性表的类型对照比较，采用了类似的方式）
- ➡ 2、在链表的数据类型定义中，使用了两个指针变量：head，指向链表的头结点；tail，指向链表尾结点。便于进行尾插入操作（头插入、尾插入是常用操作，时间复杂度都降低到 $O(1)$ ）。

线性表的另一种链式存储实现方案（第8页片子的的方案16）： 只有尾指针的单向循环链表

```
typedef struct LNode    //结点的数据类型定义
{
    ElemType data;      //结点的数据域
    struct LNode * next; //结点的指针域
} LNode , *Link, *Position;

typedef struct LinkList //单链表的数据类型定义
{
    LNode * tail;
    unsigned int uLength;
}LinkList;
```

- 特点：
- 1、单独为链表定义了一个数据类型，包含了链表的长度。（和顺序存储的线性表的类型对照比较，采用了类似的方式）
- 2、在链表的数据类型定义中，使用了一个指针变量：tail，指向链表尾结点；
- 便于进行尾插入操作（因为尾插入是一个常用操作）。
- 3、在代码实现时，采用循环链表方式实现，这样tail的next就是head；头插入、遍历都可以方便的实现。

线性表在存储结构上的实现

➡ 顺序存储

- ➡ 用数组方式直接定义线性表的实现
- ➡ 用指向一个数组的指针方式定义线性表的实现

➡ 链式存储

- ➡ 带头结点的单链表
- ➡ 带头结点的循环链表
- ➡ 带头结点的双向循环链表

线性表在存储结构上的实现

➤ 顺序存储

- 用数组方式直接定义线性表的实现
- 用指向一个数组的指针方式定义线性表的实现

➤ 链式存储

- 带头结点的单链表
- 带头结点的循环链表
- 带头结点的双向循环链表

➤ 需重点掌握的实现方法：

- 判表空
- 判表尾
- 遍历（从表头到表尾）
 - 按位置查找，链表以此为基础
 - 按值查找，以遍历为基础
- 插入
 - 表头插入
 - 某结点后插入（前插入和尾插入均以此为基础）
- 删除
 - 表头删除
 - 某个结点后删除（某结点前删除和尾删除均以此为基础）

线性表在存储结构上的实现

➡ 顺序存储

- ➡ 用数组方式直接定义线性表的实现
- ➡ 用指向一个数组的指针方式定义线性表的实现

➡ 链式存储

- ➡ 带头结点的单链表
- ➡ 带头结点的循环链表
- ➡ 带头结点的双向循环链表

➡ 需重点掌握的实现方法：

- ➡ 判表空
- ➡ 判表尾
- ➡ 遍历（从表头到表尾）
 - ➡ 按位置查找，链表以此为基础
 - ➡ 按值查找，以遍历为基础
- ➡ 插入
 - ➡ 表头插入
 - ➡ 某结点后插入（前插入和尾插入均以此为基础）
- ➡ 删除
 - ➡ 表头删除
 - ➡ 某个结点后删除（某结点前删除和尾删除均以此为基础）

顺序存储	数组方式，编译时分配内存，顺序存储的线性表大小在编程时固定				1			
	指向数组的指针方式，运行时分配内存，顺序存储的线性表大小在运行时确定				2			
链式存储		不循环（最后一个结点的next指向NULL）			循环方式（用头结点时，最后一个结点的next指向头结点；不用头结点时，最后一个结点的next指向第一个结点）			
	单向链表	有头结点	链表指针指向头结点		3	有头结点	链表指针指向头结点	11
			链表指针指向头结点，尾指针指向最后一个结点		4		链表指针指向头结点，尾指针指向最后一个结点	12
							只有尾指针指向最后一个结点	13
		无头结点	链表指针指向第一个结点		5	无头结点	链表指针指向第一个结点	14
			链表指针指向第一个结点，尾指针指向最后一个结点		6		链表指针指向第一个结点，尾指针指向最后一个结点	15
							只有尾指针指向最后一个结点	16
	双向链表	有头结点	链表指针指向头结点		7	有头结点	链表指针指向头结点	17
			链表指针指向头结点，尾指针指向最后一个结点		8		链表指针指向头结点，尾指针指向最后一个结点	18
							只有尾指针指向最后一个结点	19
		无头结点	链表指针指向第一个结点		9	无头结点	链表指针指向第一个结点	20
			链表指针指向第一个结点，尾指针指向最后一个结点		10		链表指针指向第一个结点，尾指针指向最后一个结点	21
							只有尾指针指向最后一个结点	22

顺序存储	数组方式，编译时分配内存，顺序存储的线性表大小在编程时固定				1
	指向数组的指针方式，运行时分配内存，顺序存储的线性表大小在运行时确定				2
链式存储		不循环（最后一个结点的next指向NULL）		循环方式（用头结点时，最后一个结点的next指向头结点；不用头结点时，最后一个结点的next指向第一个结点）	
	结点内有一个指针域Next	有头结点	链表指针指向头结点	3	有头结点
			链表指针指向头结点，尾指针指向最后一个结点	4	链表指针指向头结点，尾指针指向最后一个结点
			只有尾指针指向最后一个结点		12
			链表指针指向第一个结点	5	只有尾指针指向最后一个结点
	教材中， 顺序存储以方式2，为例子 链式存储以方式3为例子，链式存储的ADT定义使用方式4		链表指针指向第一个结点	6	只有尾指针指向最后一个结点
			只有尾指针指向最后一个结点	7	16
			链表指针指向头结点	8	17
			链表指针指向头结点，尾指针指向最后一个结点	9	18
			只有尾指针指向最后一个结点	10	19
			链表指针指向第一个结点		20
			链表指针指向第一个结点，尾指针指向最后一个结点		21
			只有尾指针指向最后一个结点		22

顺序表和链表的比较



存储结构 比较项目		顺序表	链表
空间	存储空间	预先分配，会导致空间闲置或溢出现象	动态分配，不会出现存储空间闲置或溢出现象
	存储密度	不用为表示结点间的逻辑关系而增加额外的存储开销，存储密度等于1	需要借助指针来体现元素间的逻辑关系，存储密度小于1
时间	存取元素	随机存取，按位置访问元素的时间复杂度为 $O(1)$	顺序存取，按位置访问元素时间复杂度为 $O(n)$
	插入、删除	平均移动约表中一半元素，时间复杂度为 $O(n)$	不需移动元素，确定插入、删除位置后，时间复杂度为 $O(1)$
适用情况		① 表长变化不大，且能事先确定变化的范围 ② 很少进行插入或删除操作，经常按元素位置序号访问数据元素	① 长度变化较大 ② 频繁进行插入或删除操作